{ JS }

The Last JavaScript Notes You'll Ever Need

Read once. Understand everything. Crack any interview.

From complete beginner to interview-ready — in one book.

19 Chapters · 180+ Topics · 18 Polyfills · 150+ Interview Q&A · 50+ Coding Challenges

Everything inside this book

Variables & Scope · Closures · this Keyword · Prototypes & OOP · Async / Promises
Event Loop · DOM & Events · ES6+ Modern JS · Polyfills · Machine Coding Round

Written by **Syed Shaaz Akhtar**

2025 Edition · Simple English · Interview Ready

One Book. Zero Confusion. Full Confidence.

Table of Contents

Level 1 — Foundations

Chapter 1: Introduction to JavaScript

- What is JavaScript?
- Weakly Typed Language
- The Console
- Variables — Containers for Storing Data
- Data Types
- Operators
- Conditionals
- Loops

Chapter 2: Strings

- Declaring Strings
- Strings are Immutable
- String Length
- Accessing Characters
- String Concatenation
- Case Conversion
- Trim — Remove Extra Spaces
- slice() vs substring()
- replace() and replaceAll()
- split() — String to Array
- Search Methods
- repeat(), padStart(), padEnd()
- Converting String to Number and Vice Versa
- Spread Operator with Strings
- String Interview Questions with Solutions
- Quick Reference — All String Methods

Chapter 3: Functions

- What is a Function?
- 4 Ways to Write a Function
- Parameters vs Arguments
- Default Parameters
- Rest Parameters
- The return Statement
- First-Class Functions
- Higher-Order Functions
- Pure vs Impure Functions
- Callback Functions
- Function Scope
- Function Interview Questions with Solutions
- Quick Reference — All Function Types

Chapter 4: Arrays

› What is an Array?
› Creating Arrays
› Array Length and Accessing Last Element
› Adding and Removing — push, pop, shift, unshift
› slice() vs splice()
› Searching in Arrays
› map(), filter(), reduce() — The Power Trio
› forEach() vs map()
› Sorting Arrays
› Other Useful Methods
› Spread Operator and Destructuring
› Array Interview Questions with Solutions
› Quick Reference — All Array Methods

Chapter 5: JavaScript Objects

› What is an Object?
› 4 Ways to Create an Object
› Accessing and Modifying Properties
› Methods — Functions Inside Objects
› Object Destructuring
› Spread Operator and Rest with Objects
› Looping Through an Object
› Checking Properties in an Object
› Object.freeze() vs Object.seal()
› Shallow Copy vs Deep Copy
› Optional Chaining (?.) and Nullish Coalescing (??)
› Computed Property Names
› Object Interview Questions with Solutions
› Quick Reference — All Object Concepts

Level 2 — How JavaScript Really Works

Chapter 6: Scope in JavaScript

› What is Scope?
› The 3 Types of Scope
› var vs let vs const — The Scope Difference
› Scope Chain — How JavaScript Finds Variables
› Lexical Scope — Where You WRITE the Code Matters
› Hoisting — Variables and Functions Move to the Top
› Variable Shadowing — Inner Hides Outer
› Scope Interview Questions with Solutions
› Quick Reference — Scope at a Glance

Chapter 7: The 'this' Keyword

› The Story That Makes 'this' Crystal Clear Forever
› Context 1 — 'this' in Global Scope
› Context 2 — 'this' Inside an Object Method
› Context 3 — 'this' in a Regular Standalone Function
› Context 4 — 'this' in Arrow Functions
› Context 5 — 'this' with new Keyword (Constructor)
› Context 6 — 'this' in ES6 Classes

› Context 7 — 'this' in Event Listeners
› The Complete 'this' Summary — All Contexts
› Tricky 'this' Scenarios — Interview Traps
› 'this' Keyword Interview Questions with Solutions
› Quick Reference — 'this' in Every Context

Chapter 8: Closures

› The Story That Will Make You Never Forget Closures
› What Actually Happens in Memory
› How to Spot a Closure in Code — 3 Signs
› Private Variables — The Most Important Use of Closures
› Maintaining State Between Calls
› Function Factories — Create Customised Functions
› Closure in Loops — The Most Famous Interview Trap
› Memoization — Caching Results with Closures
› The Module Pattern — Closures in Real Projects
› Closure Interview Questions with Solutions
› Quick Reference — Closures at a Glance

Chapter 9: Call, Apply, and Bind

› The Problem — Why Do call, apply, bind Exist?
› Quick Understanding of 'this'
› call() — Borrow and Run Immediately
› apply() — Same as call() but Arguments in an Array
› bind() — Prepare Now, Run Later
› call vs apply vs bind — Side by Side
› Real World Examples
› Call, Apply, Bind Interview Questions with Solutions
› Quick Reference — call, apply, bind at a Glance

Level 3 — Browser & Practical

Chapter 10: Understanding the DOM

› What is the DOM?
› Selecting Elements — 5 Ways
› Reading and Changing Content
› classList — The Cleanest Way to Manage CSS Classes
› Events and Event Listeners
› Event Bubbling, Capturing, and Delegation
› Creating, Inserting, and Removing Elements
› DOM Traversal — Moving Through the Tree
› DOM Performance — Reflow, Repaint, DocumentFragment
› DOM Interview Questions with Solutions
› Quick Reference — All DOM Methods

Chapter 11: Asynchronous JavaScript

› Synchronous vs Asynchronous — The Big Difference
› Why Does Asynchronous Code Exist?
› How JavaScript Actually Works — Event Loop
› Callbacks — The Old Way to Handle Async
› Promises — The Better Way

› async / await — The Cleanest and Modern Way
› Promise.all, Promise.race, Promise.allSettled, Promise.any
› fetch() — Getting Data from an API
› setTimeout and setInterval
› Async JavaScript Interview Questions with Solutions
› Quick Reference — All Async Concepts

Chapter 12: Error Handling

› Why Error Handling Exists
› try / catch / finally — The Main Tool
› The Error Object — What You Get in catch
› Types of Errors in JavaScript
› throw — Create Your Own Errors
› Error Handling in Async Code
› Error Handling Interview Questions

Chapter 13: Storing Data in the Browser

› Why Do We Store Data in the Browser?
› localStorage — Permanent Storage
› sessionStorage — Temporary Storage
› Cookies — The Bridge Between Browser and Server
› localStorage vs sessionStorage vs Cookies — Full Comparison
› JSON.stringify() and JSON.parse()
› Browser Storage Interview Questions with Solutions
› Quick Reference — All Storage Methods

Level 4 — Advanced

Chapter 14: Object-Oriented Programming (OOP)

› What is OOP? The Story That Makes It Click Forever
› Class and Constructor — The Foundation
› The 4 Pillars — Encapsulation, Inheritance, Polymorphism, Abstraction
› Prototypes — How JavaScript OOP Actually Works Under the Hood
› static Methods — Class Level, Not Object Level
› Getters and Setters — Controlled Property Access
› Complete Real World Example — School Management System
› OOP Interview Questions with Solutions
› Quick Reference — OOP at a Glance

Chapter 15: Advanced JavaScript Concepts

› Destructuring — Unpack Values Easily
› Spread (...) and Rest (...) — Same Symbol, Different Jobs
› Short-Circuit Evaluation — &&, ||, ?? Smart Tricks
› Template Literals — String Power
› Symbol — Unique Keys That Never Clash
› Generators — Pause and Resume Functions
› WeakMap and WeakSet — Memory-Friendly Collections
› Advanced Concepts Interview Questions

Chapter 16: Must-Know Modern JavaScript

› Microtask Queue vs Macrotask Queue

› Type Coercion — == vs === and the Tricky Outputs
› Map and Set — Modern Data Structures
› ES Modules — import and export
› Regular Expressions (Regex) — Pattern Matching
› Prototype Chain — How JavaScript Inheritance Really Works
› Iterators and for...of — Custom Iteration
› Proxy and Reflect — Intercept Object Operations
› Newer Array Methods — flat, flatMap, at, findLast, Array.from
› Modern JavaScript Interview Questions with Solutions

Chapter 17: Advanced Patterns

› Debouncing — Wait Until User Stops
› Throttling — Run at Most Once Per Interval
› Currying — One Argument at a Time
› IIFE — Immediately Invoked Function Expression
› Mixin Pattern — Add Features Without Inheritance
› Advanced Patterns Interview Questions

Chapter 18: Polyfills — Write It From Scratch

› Polyfill 1: Array.prototype.map()
› Polyfill 2: Array.prototype.filter()
› Polyfill 3: Array.prototype.reduce()
› Polyfill 4: Array.prototype.forEach()
› Polyfill 5: Array.prototype.find() and findIndex()
› Polyfill 6: Array.prototype.every() and some()
› Polyfill 7: Array.prototype.flat()
› Polyfill 8: Function.prototype.bind()
› Polyfill 9: Function.prototype.call()
› Polyfill 10: Function.prototype.apply()
› Polyfill 11: Promise.all()
› Polyfill 12: Promise.allSettled()
› Polyfill 13: Array.from()
› Polyfill 14: Object.assign()
› Polyfill 15: Deep Clone — structuredClone()
› Polyfill 16: pipe() and compose()
› Polyfill 17: once() — Run Function Only Once
› Polyfill 18: memoize() — Cache Function Results
› Polyfill Interview Questions with Solutions

Level 5 — Machine Coding Practice

Chapter 19: Machine Coding Round — Practice Questions

› Section A — Output Based Questions (Q1–Q20)
› A1: var, let, const and Hoisting (Q1–Q5)
› A2: Closures Output (Q6–Q8)
› A3: 'this' Keyword Output (Q9–Q11)
› A4: Promises and Async/Await Output (Q12–Q15)
› A5: Prototype and OOP Output (Q16–Q17)
› A6: Type Coercion Traps (Q18–Q20)
› Section B — Function Implementation Questions
› B1: Array and String Manipulation — flatten, groupBy, chunk, uniqueBy, anagram, deepEqual

› B2: Closure-Based — counter, limitCalls, retry
› B3: Object Transformation — flattenObj, unflatten, pick, omit
› B4: Async — sequential promises, timeout wrapper, allSettled
› B5: DOM — event delegation, infinite scroll
› Section C — Full Machine Coding Challenges
› C1: Build an Event Emitter (on/off/emit/once)
› C2: Build an LRU Cache
› C3: Search Bar with Debounce
› C4: Task Queue with Concurrency Limit
› C5: Implement lodash _.get()
› C6: Build a Rate Limiter
› C7: Virtual DOM Diffing Function
› C8: Simple Reactive Store (like Redux)
› Section D — Data Structure Challenges
› D1: Stack + Balanced Brackets
› D2: Queue using Two Stacks
› D3: Binary Search
› D4: Linked List — append, reverse
› Section E — Logic and Algorithm Questions
› E1: Two Sum (Map approach)
› E2: Valid Palindrome
› E3: Maximum Subarray — Kadane's Algorithm
› E4: Fibonacci with Memoization
› E5: All Permutations of a String
› Section F — Tricky JS-Specific Questions
› F1: map using reduce
› F2: Object.create from scratch
› F3: Observable / Reactive Variable
› F4: Promisify a callback function
› F5: pipe with Error Handling
› Quick Reference Checklist — What to Know Cold

Chapter 1: Introduction to JavaScript

Topic 1: What is JavaScript?

JavaScript is a programming language that makes your website interactive. Think of it this way:

- HTML is the skeleton of a webpage
- CSS is the design and styling
- JavaScript gives life to that skeleton — it makes things move, react, and work

For example, when you click a button and something happens, or when you type in a search box and suggestions appear — that is all JavaScript!

Topic 2: Weakly Typed Language

JavaScript is called a 'weakly typed' language. This means you can store different types of data in the same variable. JavaScript does not force you to follow strict type rules, which makes it beginner-friendly.

```
let name = 'Shaaz';   // This is a string
name = 5;            // Now it is a number
name = { age: 23 }; // Now it is an object
```

Note: In languages like Java or C++, you must declare the type. JavaScript handles this automatically.

Topic 3: The Console

To see the output of your JavaScript code on screen, we use console.log(). This shows the result in the 'Console' section of your browser.

```
let name = "Shaaz";
console.log(name);  // Output: Shaaz
```

How to Open the Console?

- Open your browser (Chrome is recommended)
- Right-click on the page → click "Inspect" → go to the "Console" tab

Tip: console.log() is your best friend while learning. Use it to check the value of any variable at any point.

Topic 4: Variables — Containers for Storing Data

A variable is a container where you can store data. In JavaScript, there are 3 ways to declare variables:

- var — Old method, function-scoped, can be overwritten anytime
- let — Modern method, block-scoped, value can be changed
- const — Block-scoped, value CANNOT be changed after assignment

```
var score = 10;       // old way
let name = "Shaaz";  // modern way
const pi = 3.14;     // constant, never changes
```

Tip: Use 'const' by default. Use 'let' only when the value needs to change. Avoid 'var' in modern code.

✅Remember: var is function-scoped. let and const are block-scoped. This difference matters inside loops and if-blocks.

Topic 5: Data Types

JavaScript has 7 main data types. Knowing these is very important for interviews:

1. Number — for all numeric values

```
let age = 25;
let price = 99.99;
```

2. String — for text

```
let name = "Shaaz";
let city = 'Delhi';
```

3. Boolean — true or false only

```
let isLoggedIn = true;
let isAdmin = false;
```

4. Null — intentionally empty

Use null when you want to say 'this variable exists but has no value on purpose'.

```
let emptyValue = null;
```

5. Undefined — not assigned yet

When you declare a variable but do not give it a value, it becomes undefined automatically.

```
let noValue;
console.log(noValue); // Output: undefined
```

6. Object — key-value pairs

Objects store multiple related values together.

```
let person = { name: "Shaaz", age: 25, city: "Delhi" };
console.log(person.name); // Output: Shaaz
```

7. Array — ordered list of values

Arrays store multiple values in a sequence. Each value has an index starting from 0.

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Output: Apple
```

✅Remember: null means empty on purpose. undefined means not yet assigned. They are NOT the same!

Tip: Use typeof to check the data type of any variable: console.log(typeof 'hello'); // Output: string

Topic 6: Operators

Arithmetic Operators — for math

```
let a = 10, b = 3;
console.log(a + b);  // 13  — addition
console.log(a - b);  // 7   — subtraction
console.log(a * b);  // 30  — multiplication
console.log(a / b);  // 3.33 — division
console.log(a % b);  // 1   — remainder (modulus)
```

Comparison Operators — always return true or false

```
console.log(5 == "5");   // true  — loose equality (only checks value)
console.log(5 === "5");  // false — strict equality (checks value AND type)
console.log(5 != 3);    // true
console.log(5 > 3);     // true
```

```
console.log(5 <= 5);    // true
```

✅**Remember: Always use === instead of ==. It is safer because it also checks the data type.**

Logical Operators — combine multiple conditions

- && (AND) — both conditions must be true
- || (OR) — at least one condition must be true
- ! (NOT) — flips the value: true becomes false, false becomes true

```
let age = 20;
console.log(age > 18 && age < 30); // true
console.log(age < 18 || age > 60); // false
console.log(!true);                // false
```

Topic 7: Conditionals

if / else if / else

Used to run different blocks of code depending on a condition.

```
let marks = 75;
if (marks >= 90) {
  console.log("Grade: A");
} else if (marks >= 70) {
  console.log("Grade: B");
} else if (marks >= 50) {
  console.log("Grade: C");
} else {
  console.log("Grade: Fail");
}
// Output: Grade: B
```

Switch Statement

A cleaner way to check one variable against many possible values.

```
let day = "Monday";
switch(day) {
  case "Monday":
    console.log("Start of the week!");
    break;
  case "Friday":
    console.log("Weekend is near!");
    break;
  default:
    console.log("A regular day.");
}
```

Tip: Always add 'break' inside each case. Without it, JavaScript will keep running the next cases — this is called 'fall-through'.

Topic 8: Loops

Loops are used when you need to repeat a task multiple times. Instead of writing the same line 100 times, use a loop.

for Loop — when you know how many times to repeat

```
for (let i = 1; i <= 5; i++) {
  console.log("Count: " + i);
```

```
}
// Output: Count: 1, Count: 2, Count: 3, Count: 4, Count: 5
```

Note: The for loop has 3 parts: (start; condition; step). It runs as long as the condition is true.

while Loop — when you don't know how many times upfront

```
let i = 1;
while (i <= 5) {
  console.log("Count: " + i);
  i++;
}
```

do...while Loop — runs at least once, then checks condition

```
let i = 1;
do {
  console.log("Count: " + i);
  i++;
} while (i <= 5);
```

for...of Loop — best for looping through Arrays

```
let fruits = ["Apple", "Banana", "Mango"];
for (let fruit of fruits) {
  console.log(fruit);
}
// Output: Apple  Banana  Mango
```

for...in Loop — best for looping through Object keys

```
let person = { name: "Shaaz", age: 23, city: "Delhi" };
for (let key in person) {
  console.log(key + ': ' + person[key]);
}
// Output: name: Shaaz   age: 23   city: Delhi
```

break and continue

break stops the loop completely. continue skips the current step and moves to the next one.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue; // Skip 3
  if (i === 5) break;    // Stop at 5
  console.log(i);
}
// Output: 1, 2, 4
```

✅Remember: for...of is for arrays. for...in is for objects. Do not mix them up — it is a common interview trick!

Chapter 2: Strings

How to Define in an Interview

"A String in JavaScript is a sequence of characters enclosed in single quotes, double quotes, or backticks. Strings are primitive data types and they are immutable — meaning once a string is created, it cannot be changed. Any method that appears to modify a string actually returns a brand new string. Strings are used to represent text data like names, messages, and URLs."

Example you can say in interview:

```
let name = "Shaaz";   // double quotes
let city = 'Delhi';   // single quotes
let msg  = `Hello, ${name}!`; // template literal — can embed variables
```

Strings are one of the most commonly tested topics in JavaScript interviews. Make sure you understand all the methods clearly.

Topic 1: Declaring Strings

A string is a sequence of characters. You can write it in 3 ways:

```
let single   = 'Hello';                      // Single quotes
let double   = "Hello";                      // Double quotes
let template = `My name is ${single}`;       // Template literal (backticks)
```

Tip: Template literals are the modern and preferred way. You can put any expression inside ${ } — variables, math, function calls, etc.

```
let a = 5, b = 10;
console.log(`Sum is ${a + b}`); // Output: Sum is 15
```

Topic 2: Strings are Immutable (Very Important for Interviews!)

Immutable means you CANNOT change a string once it is created. If you try to change a character, it simply does not work — no error, but no change either.

```
let str = "Hello";
str[0] = 'J';
console.log(str); // Output: Hello  (NOT changed!)
```

To 'change' a string, you must create a NEW string.

```
let str = "Hello";
let newStr = "J" + str.slice(1);
console.log(newStr); // Output: Jello
```

✅Remember: Strings are immutable in JavaScript. Every string method returns a NEW string — it never changes the original.

Topic 3: String Length

The .length property gives you the total number of characters in a string.

```
let str = "JavaScript";
console.log(str.length); // Output: 10
```

Note: length is a property, not a method — do NOT write str.length(). No brackets!

Topic 4: Accessing Characters

You can access any character using its index. Index always starts from 0.

```
let str = "JavaScript";
```

```
console.log(str[0]);          // Output: J
console.log(str[4]);          // Output: S
console.log(str.charAt(4));   // Output: S
console.log(str[str.length - 1]); // Output: t  (last character)
```

Tip: To get the last character, always use str[str.length - 1]. This is a very common interview question.

Topic 5: String Concatenation (Joining Strings)

Old way — using + operator

```
let firstName = "Shaaz";
let lastName  = "Akhtar";
console.log(firstName + " " + lastName); // Output: Shaaz Akhtar
```

Modern way — using Template Literals

```
console.log(`Hello, ${firstName} ${lastName}!`); // Output: Hello, Shaaz Akhtar!
```

✅Remember: Always prefer template literals over + for joining strings. Cleaner, easier to read.

Topic 6: Case Conversion

```
let str = "Hello World";
console.log(str.toLowerCase()); // Output: hello world
console.log(str.toUpperCase()); // Output: HELLO WORLD
```

Tip: Very commonly used in interviews to compare strings without worrying about uppercase/lowercase.

```
let input = "SHAAZ";
console.log(input.toLowerCase() === "shaaz"); // Output: true
```

Topic 7: Trim — Remove Extra Spaces

Used to clean up user input (form fields often have accidental spaces).

```
let messy = "   Hello World!   ";
console.log(messy.trim());      // Output: "Hello World!"  (both sides)
console.log(messy.trimStart()); // Output: "Hello World!   " (left side only)
console.log(messy.trimEnd());   // Output: "   Hello World!" (right side only)
```

Topic 8: slice() vs substring() — Interview Favourite!

Both are used to extract a part of a string. slice() is more powerful and preferred.

```
let str = "JavaScript";
//         0123456789  (index positions)

console.log(str.slice(0, 4));      // Output: Java  (from 0, up to but NOT including
4)
console.log(str.slice(4));         // Output: Script (from index 4 to end)
console.log(str.slice(-6));        // Output: Script (last 6 characters!)
console.log(str.substring(0, 4));  // Output: Java
```

Note: slice() accepts NEGATIVE indexes — it counts from the end. substring() does NOT support negative indexes.

✅Remember: slice(-1) gives the last character. slice(-3) gives the last 3 characters. Very useful!

Topic 9: replace() and replaceAll()

Used to find and replace text inside a string.

```
let str = "I love cats. Cats are great.";
console.log(str.replace("cats", "dogs"));    // Output: I love dogs. Cats are great.
console.log(str.replaceAll("cats", "dogs")); // Output: I love dogs. Cats are great.
```

Note: replace() only replaces the FIRST match. replaceAll() replaces ALL matches. Also note: it is case-sensitive!

```
// Case-insensitive replace using regex:
console.log(str.replace(/cats/gi, "dogs")); // Output: I love dogs. dogs are great.
```

Topic 10: split() — String to Array

Splits a string into an array of parts based on a separator. Very commonly asked!

```
let colors = "red,green,blue";
console.log(colors.split(","));   // Output: ["red", "green", "blue"]

let sentence = "Hello World";
console.log(sentence.split(" ")); // Output: ["Hello", "World"]

let str = "abc";
console.log(str.split(""));      // Output: ["a", "b", "c"]  (split each character)
```

Tip: split('') with an empty string splits the string into individual characters — very useful for reversing strings!

Topic 11: Search Methods

```
let str = "Hello World Hello";

console.log(str.indexOf("Hello"));      // Output: 0  (first occurrence)
console.log(str.lastIndexOf("Hello"));  // Output: 12 (last occurrence)
console.log(str.includes("World"));     // Output: true
console.log(str.includes("world"));     // Output: false (case-sensitive!)
console.log(str.startsWith("Hello"));   // Output: true
console.log(str.endsWith("Hello"));     // Output: true
```

✅Remember: indexOf() returns -1 if the value is NOT found. Always check for -1 when using indexOf in interviews.

```
let str = "JavaScript";
if (str.indexOf("Java") !== -1) {
  console.log("Found!"); // Output: Found!
}
```

Topic 12: repeat(), padStart(), padEnd()

repeat() — repeat a string N times

```
console.log("ha".repeat(3));   // Output: hahaha
console.log("*".repeat(5));    // Output: *****
```

padStart() — add padding at the beginning

Very useful for formatting numbers like dates, times, and IDs.

```
console.log("5".padStart(3, "0"));   // Output: 005
console.log("42".padStart(5, "0"));  // Output: 00042
```

padEnd() — add padding at the end

```
console.log("Hi".padEnd(5, "."));    // Output: Hi...
```

Tip: padStart() is commonly asked in interviews for tasks like formatting clock time — e.g., showing 09:05 instead of 9:5.

Topic 13: Converting String to Number and Vice Versa

String → Number

```
console.log(Number("42"));        // Output: 42
console.log(parseInt("42px"));    // Output: 42  (stops at first non-number)
console.log(parseFloat("3.14")); // Output: 3.14
console.log(Number("hello"));     // Output: NaN  (Not a Number)
```

Number → String

```
console.log(String(42));          // Output: "42"
console.log((42).toString());     // Output: "42"
console.log((255).toString(16)); // Output: "ff"  (converts to hexadecimal!)
```

✅Remember: NaN means 'Not a Number'. Use isNaN() to check: isNaN(Number('hello')) returns true.

Topic 14: Spread Operator with Strings

You can spread a string into individual characters using the spread operator (...).

```
let str = "Hello";
let chars = [...str];
console.log(chars); // Output: ["H", "e", "l", "l", "o"]
```

Topic 15: Common String Interview Questions with Solutions

Q1: Reverse a String

```
function reverseString(str) {
  return str.split("").reverse().join("");
}
console.log(reverseString("hello")); // Output: olleh
```

Note: split("") turns string to array, .reverse() flips the array, .join("") turns it back to string.

Q2: Check if a String is a Palindrome

A palindrome reads the same forwards and backwards. Example: 'racecar', 'madam'.

```
function isPalindrome(str) {
  let reversed = str.split("").reverse().join("");
  return str === reversed;
}
console.log(isPalindrome("racecar")); // Output: true
console.log(isPalindrome("hello"));   // Output: false
```

Q3: Count Occurrences of a Character

```
function countChar(str, char) {
  let count = 0;
  for (let c of str) {
    if (c === char) count++;
  }
  return count;
}
console.log(countChar("javascript", "a")); // Output: 2
```

Q4: Check if Two Strings are Anagrams

Anagrams are words with the same characters in different order. Example: 'listen' and 'silent'.

```
function isAnagram(str1, str2) {
  let sort = s => s.toLowerCase().split("").sort().join("");
  return sort(str1) === sort(str2);
}
console.log(isAnagram("listen", "silent")); // Output: true
console.log(isAnagram("hello", "world"));   // Output: false
```

Q5: Find the Most Repeated Character

```
function mostRepeated(str) {
  let count = {};
  for (let char of str) {
    count[char] = (count[char] || 0) + 1;
  }
  return Object.keys(count).reduce((a, b) => count[a] > count[b] ? a : b);
}
console.log(mostRepeated("javascript")); // Output: a
```

Q6: Capitalize First Letter of Every Word

```
function titleCase(str) {
  return str.split(" ").map(word => word[0].toUpperCase() + word.slice(1)).join(" ");
}
console.log(titleCase("hello world from shaaz")); // Output: Hello World From Shaaz
```

Q7: Remove All Spaces from a String

```
let str = "Hello World How Are You";
console.log(str.replaceAll(" ", "")); // Output: HelloWorldHowAreYou
// OR using split & join:
console.log(str.split(" ").join("")); // Output: HelloWorldHowAreYou
```

Q8: Check if String Contains Only Numbers

```
function isNumeric(str) {
  return !isNaN(str) && str.trim() !== '';
}
console.log(isNumeric("12345")); // Output: true
console.log(isNumeric("12a45")); // Output: false
```

Topic 16: Quick Reference — All String Methods at a Glance

str.length — total number of characters

str[0] / str.charAt(0) — access a character by index

str.toUpperCase() / str.toLowerCase() — change case

str.trim() / trimStart() / trimEnd() — remove spaces

str.slice(start, end) — extract part of string (supports negative index)

str.substring(start, end) — extract part of string (no negative index)

str.replace(old, new) — replace first match

str.replaceAll(old, new) — replace all matches

str.split(separator) — split string into array

str.indexOf(val) — find first position (-1 if not found)

str.lastIndexOf(val) — find last position

str.includes(val) — true/false check

str.startsWith(val) / endsWith(val) — check beginning/end

str.repeat(n) — repeat string n times

str.padStart(len, char) / padEnd(len, char) — add padding

str.concat(str2) — join two strings (same as +)

> ✅**Remember: All string methods return a NEW string. The original string is NEVER changed because strings are immutable.**

Chapter 3: Functions

How to Define in an Interview

"A Function in JavaScript is a reusable block of code that is written once and can be executed multiple times. A function can accept inputs called parameters, perform some logic on them, and optionally return an output. Functions are first-class citizens in JavaScript — meaning they can be stored in variables, passed as arguments to other functions, and returned from other functions. JavaScript supports multiple types of functions: function declarations, function expressions, arrow functions, and IIFEs."

Example you can say in interview:

```
function add(a, b) {   // declaration — hoisted, reusable
  return a + b;        // returns output
}
const multiply = (a, b) => a * b; // arrow function — short, modern
console.log(add(2, 3));       // Output: 5
console.log(multiply(2, 3)); // Output: 6
```

Functions are the heart of JavaScript. Almost every interview question involves functions in some way. So read this chapter very carefully.

Topic 1: What is a Function? (Very Simple)

Think of a function like a machine. You put something IN, the machine does its work, and gives something OUT.

Example in real life: A juice machine. You put fruits IN → machine makes juice → juice comes OUT.

In code, a function is a block of code that you write ONCE and can use it again and again whenever you need.

```
function makeJuice(fruit) {      // 'fruit' goes IN
  return fruit + ' juice ready!'; // result comes OUT
}

console.log(makeJuice("Mango"));  // Output: Mango juice ready!
console.log(makeJuice("Apple"));  // Output: Apple juice ready!
```

Tip: Write once, use many times. That is the whole point of a function.

Topic 2: 4 Ways to Write a Function — Must Know for Interviews!

This is one of the most asked topics. You must know all 4 types and their differences.

Way 1 — Function Declaration (Classic Way)

```
function greet(name) {
  return `Hello, ${name}!`;
}
console.log(greet("Shaaz")); // Output: Hello, Shaaz!
```

Special thing: You can call this function BEFORE writing it. This is called Hoisting.

```
console.log(greet('Shaaz')); // This works even before the function is defined!

function greet(name) {
  return `Hello, ${name}!`;
}
```

✅Remember: Function Declaration is HOISTED — you can call it before you write it. This is unique to this type only.

Way 2 — Function Expression (Stored in a Variable)

```
const greet = function(name) {
  return `Hello, ${name}!`;
};
console.log(greet("Shaaz")); // Output: Hello, Shaaz!
```

Special thing: This is NOT hoisted. You CANNOT call it before writing it.

```
console.log(greet('Shaaz')); // ERROR! Cannot access before initialization
const greet = function(name) { return `Hello, ${name}!`; };
```

Note: Function Expression behaves like a variable. Just like you cannot use a variable before declaring it, you cannot call this function before writing it.

Way 3 — Arrow Function (Short and Modern Way)

```
const greet = (name) => `Hello, ${name}!`;
console.log(greet("Shaaz")); // Output: Hello, Shaaz!
```

Arrow functions are the shortest and most modern way. Used everywhere in real projects.

```
// If only ONE parameter — brackets are optional:
const double = n => n * 2;
console.log(double(5)); // Output: 10

// If NO parameters — empty brackets required:
const sayHi = () => 'Hi there!';
console.log(sayHi()); // Output: Hi there!

// If multiple lines — use curly braces and return:
const add = (a, b) => {
  let sum = a + b;
  return sum;
};
```

✅Remember: Arrow functions do NOT have their own 'this'. They take 'this' from their parent. Very important for interviews!

Way 4 — IIFE (Immediately Invoked Function Expression)

IIFE is a function that runs immediately as soon as it is written. You do not need to call it separately.

```
(function() {
  console.log("I run immediately!");
})();
// Output: I run immediately!

// Arrow function IIFE:
(() => {
  console.log("Arrow IIFE runs too!");
})();
```

Tip: IIFE is used to avoid polluting the global scope. Variables inside IIFE are private — they cannot be accessed from outside.

```
(function() {
  let secret = 'hidden value';  // Cannot be accessed outside
})();
console.log(secret); // ERROR! secret is not defined
```

Topic 3: Parameters vs Arguments — Interview Question!

Many people confuse these two words. Here is the simple explanation:

- Parameter — the name written in the function definition (like a placeholder)
- Argument — the actual value you pass when calling the function

```
function add(a, b) {      // 'a' and 'b' are PARAMETERS
  return a + b;
}
add(5, 10);               // 5 and 10 are ARGUMENTS
```

✅**Remember: Parameter = placeholder in definition. Argument = actual value during calling.**

Topic 4: Default Parameters

Sometimes a user might not pass a value. In that case, you can set a default value so the function does not break.

```
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}
console.log(greet());          // Output: Hello, Guest!   (no argument passed)
console.log(greet("Shaaz"));   // Output: Hello, Shaaz!   (argument passed)
```

Topic 5: Rest Parameters — When You Don't Know How Many Arguments

Sometimes you do not know in advance how many values someone will pass. Use rest parameters (...) to collect them all into an array.

```
function sum(...numbers) {   // ...numbers collects ALL arguments into an array
  let total = 0;
  for (let n of numbers) total += n;
  return total;
}
console.log(sum(1, 2, 3));         // Output: 6
console.log(sum(10, 20, 30, 40));  // Output: 100
```

Tip: Rest parameter must always be the LAST parameter. You cannot write (a, ...b, c) — this is an error.

Topic 6: The return Statement

The return statement does two things: it gives back a value AND it stops the function immediately.

```
function checkAge(age) {
  if (age < 18) {
    return "Too young!";   // Function stops here if age < 18
  }
  return "Welcome!";       // Only reaches here if age >= 18
}
console.log(checkAge(15)); // Output: Too young!
console.log(checkAge(21)); // Output: Welcome!
```

Note: A function without a return statement automatically returns undefined.

```
function doNothing() {}
console.log(doNothing()); // Output: undefined
```

Topic 7: First-Class Functions — Very Important Concept!

In JavaScript, functions are treated like any other value — like a number or a string. This means:

- You can store a function in a variable
- You can pass a function as an argument to another function
- You can return a function from another function

This is what people mean when they say 'functions are first-class citizens in JavaScript'.

```
// Store in a variable:
const sayHi = () => 'Hello!';

// Pass as an argument:
function runTwice(fn) {
  fn();
  fn();
}
runTwice(() => console.log('Running!')); // Prints: Running! Running!

// Return from a function:
function multiplier(x) {
  return (n) => n * x;  // Returns a NEW function!
}
const triple = multiplier(3);
console.log(triple(5));  // Output: 15
console.log(triple(10)); // Output: 30
```

Topic 8: Higher-Order Functions

A Higher-Order Function is simply a function that either:

- Takes another function as an input (argument), OR
- Returns a function as its output

```
// Example 1: Takes a function as argument
function doMath(a, b, operation) {
  return operation(a, b);  // calls whatever function is passed
}
const add      = (a, b) => a + b;
const multiply = (a, b) => a * b;

console.log(doMath(5, 3, add));      // Output: 8
console.log(doMath(5, 3, multiply)); // Output: 15
```

Tip: map(), filter(), and reduce() are all Higher-Order Functions — they take a function as an argument. You will use these every day in real projects.

Topic 9: Pure vs Impure Functions — Asked in Almost Every Interview!

Pure Function

A pure function follows TWO simple rules:

- Rule 1: Same input will ALWAYS give the same output
- Rule 2: It does NOT change anything outside the function (no side effects)

```
// PURE FUNCTION — only uses its own inputs, touches nothing outside
function add(a, b) {
  return a + b;
}
console.log(add(2, 3)); // Always 5. Every single time. No matter what.
```

This is pure because: the same inputs (2, 3) always give the same output (5), and nothing outside the function is changed.

Impure Function

An impure function breaks at least one of those two rules — either it gives different results for the same input, or it changes something outside.

```
// IMPURE FUNCTION — changes a variable outside the function (side effect)
let total = 0;
function addToTotal(n) {
  total += n;   // This changes 'total' which is OUTSIDE the function
  return total;
}
console.log(addToTotal(5)); // Output: 5
console.log(addToTotal(5)); // Output: 10  (same input, different output!)
```

This is impure because the same input (5) gives different output each time. Also, it modifies the outside variable 'total'.

```

// Another example of impure — uses random or date (unpredictable output):
function getRandomNumber() {
  return Math.random();  // Different output every time!
}
```

✅Remember: Pure = predictable + no side effects. Impure = unpredictable OR changes something outside. Pure functions are easier to test and debug.

Side Effects — What Are They?

A side effect is when a function changes or interacts with something OUTSIDE of itself. Examples of side effects:

- Changing a variable that is outside the function
- Printing to the console (console.log)
- Fetching data from an API
- Writing to a file or database
- Changing the DOM (the webpage)

Note: Side effects are not always bad — they are necessary in real apps. But pure functions are preferred for logic and calculations because they are predictable and easy to test.

Topic 10: Callback Functions — Very Commonly Asked!

A callback is just a function that you GIVE to another function, and that other function calls it later.

Real life example: You order food at a restaurant and give your phone number. When food is ready, the restaurant CALLS you back. Your phone number is the callback!

```
function doWork(task, callback) {
  console.log("Doing: " + task);
  callback();   // Now call the function that was passed in
}

doWork("Cooking", function() {
  console.log("Work is done!");
});
// Output: Doing: Cooking
//         Work is done!
```

Callbacks are used a lot with array methods like forEach, map, filter — and with async operations like setTimeout and API calls.

```
// Real example with setTimeout:
setTimeout(function() {
  console.log("This runs after 2 seconds");
}, 2000);
```

Tip: Any time you pass a function as an argument, it is a callback. You see this everywhere in JavaScript.

Topic 11: Function Scope — What Stays Inside, Stays Inside

Variables created INSIDE a function cannot be used OUTSIDE that function. They are private to that function.

```
function myFunction() {
  let secret = "I am inside!";
  console.log(secret); // Works fine inside
}
myFunction();
console.log(secret); // ERROR! secret is not defined outside
```

But a function CAN read variables from OUTSIDE:

```
let name = "Shaaz";  // outside variable
function greet() {
  console.log(`Hello, ${name}!`); // can READ outside variable
}
greet(); // Output: Hello, Shaaz!
```

Topic 12: Common Function Interview Questions with Solutions

Q1: What is the difference between function declaration and function expression?

```
// Function Declaration — hoisted, can be called before writing
function greet() { return 'Hello!'; }

// Function Expression — NOT hoisted, cannot call before writing
const greet = function() { return 'Hello!'; };
```

✅Remember: The key difference is HOISTING. Declaration is hoisted, expression is NOT.

Q2: What is the difference between arrow function and regular function?

- Arrow function does NOT have its own 'this' — it borrows from parent
- Arrow function cannot be used as a constructor (no 'new' keyword)
- Arrow function does NOT have 'arguments' object
- Regular function has all of these

```
// Regular function — has its own 'this'
function Person(name) {
  this.name = name;
  setTimeout(function() {
    console.log(this.name); // undefined! 'this' is lost here
  }, 1000);
}

// Arrow function — borrows 'this' from parent
function Person(name) {
```

```
  this.name = name;
  setTimeout(() => {
    console.log(this.name); // Shaaz! 'this' is correct here
  }, 1000);
}
```

Q3: Write a pure function to calculate area of a rectangle

```
function getArea(length, width) {
  return length * width;  // Same input = same output. Nothing changed outside.
}
console.log(getArea(5, 3));  // Output: 15
console.log(getArea(5, 3));  // Output: 15  (always same!)
```

Q4: What is a Higher-Order Function? Give an example.

A function that takes another function as an argument OR returns a function.

```
function applyTax(price, taxFn) {   // taxFn is a function passed as argument
  return taxFn(price);
}
const addGST = price => price * 1.18;
console.log(applyTax(1000, addGST)); // Output: 1180
```

Q5: What will this output? (Tricky Hoisting Question)

```
console.log(square(4));    // What happens here?

var square = function(n) {
  return n * n;
};
```

Answer: ERROR — TypeError: square is not a function

Why? Because 'var square' is hoisted as undefined, not as a function. So calling undefined() throws an error.

> **✅Remember: Function declarations are fully hoisted. But var variables are hoisted as 'undefined' — not as their value.**

Q6: Create a function that can be called like: add(2)(3) — gives 5

This is called a curried function. The function returns another function.

```
function add(a) {
  return function(b) {
    return a + b;
  };
}
console.log(add(2)(3));  // Output: 5

// Arrow function version:
const add = a => b => a + b;
console.log(add(2)(3));  // Output: 5
```

Q7: What is the difference between pure and impure functions?

Pure function — same input always gives same output, no side effects.

Impure function — output can change even with same input, or it changes something outside.

```
// Pure:
const multiply = (a, b) => a * b;
```

```
// Impure:
let count = 0;
const increment = () => ++count; // changes external variable
```

Q8: Write a function that runs another function N times

```
function runNTimes(fn, n) {
  for (let i = 0; i < n; i++) {
    fn();
  }
}
runNTimes(() => console.log("Hello!"), 3);
// Output: Hello! Hello! Hello!
```

Topic 13: Quick Reference — All Function Types at a Glance

Function Declaration — hoisted, can call before writing, has own 'this'
Function Expression — NOT hoisted, stored in a variable
Arrow Function — shortest syntax, NO own 'this', NOT hoisted
IIFE — runs immediately, variables are private inside
Callback Function — a function passed as an argument to another function
Higher-Order Function — takes a function as argument OR returns a function
Pure Function — same input = same output, no side effects
Impure Function — output can vary, or it changes something outside
Rest Parameters (...) — collect unlimited arguments into an array
Default Parameters — fallback value if no argument is passed

> ✅**Remember: The most asked interview topics in Functions: Pure vs Impure, Hoisting difference, Arrow vs Regular, Callbacks, and HOF.**

Chapter 4: Arrays

How to Define in an Interview

"An Array in JavaScript is an ordered collection of values stored in a single variable. Arrays are zero-indexed — meaning the first element is at index 0. They can hold any type of data including numbers, strings, booleans, objects, and even other arrays. Arrays are mutable, meaning their content can be changed after creation. JavaScript provides many built-in methods on arrays like push, pop, map, filter, and reduce which make working with lists of data very easy."

Example you can say in interview:

```
let fruits = ["Apple", "Banana", "Mango"];  // ordered list, zero-indexed
console.log(fruits[0]);        // Output: Apple  (index starts at 0)
console.log(fruits.length);    // Output: 3
fruits.push('Grapes');         // add to end — arrays are mutable
console.log(fruits);           // Output: ["Apple","Banana","Mango","Grapes"]
```

Arrays are used in almost every program you write. They are also one of the most tested topics in JavaScript interviews. So understand each method clearly with examples.

Topic 1: What is an Array? (Super Simple)

Think of an array like a train. Each coach (compartment) holds one item. The first coach is number 0, second is number 1, and so on.

An array stores MULTIPLE values in ONE variable — in order, one after another.

```
let fruits = ["Apple", "Banana", "Mango"];
//              index 0    index 1    index 2
console.log(fruits[0]); // Output: Apple
console.log(fruits[1]); // Output: Banana
console.log(fruits[2]); // Output: Mango
```

Tip: Index always starts from 0, NOT from 1. This is one of the most common beginner mistakes.

Arrays can hold ANY type of data — numbers, strings, booleans, objects, even other arrays:

```
let mixed = [42, "hello", true, null, { name: "Shaaz" }];
```

Topic 2: Creating Arrays

Way 1 — Square Brackets (Most Common Way)

```
let colors = ["Red", "Green", "Blue"];
```

Way 2 — Array Constructor (Rarely Used)

```
let colors = new Array("Red", "Green", "Blue");
```

Empty Array — Start Empty, Add Later

```
let basket = [];
basket.push("Apple");
basket.push("Mango");
console.log(basket); // Output: ["Apple", "Mango"]
```

Topic 3: Array Length and Accessing Last Element

```
let fruits = ["Apple", "Banana", "Mango", "Grapes"];
console.log(fruits.length);            // Output: 4
```

```
console.log(fruits[fruits.length - 1]); // Output: Grapes (last element)
console.log(fruits[fruits.length - 2]); // Output: Mango  (second last)
```

Tip: fruits[fruits.length - 1] is the standard way to get the last item. Very commonly asked in interviews.

Topic 4: Adding and Removing Elements — push, pop, shift, unshift

Think of the array as a queue at a shop. You can add or remove people from either end.

push() — Add to the END

```
let arr = [1, 2, 3];
arr.push(4);
console.log(arr); // Output: [1, 2, 3, 4]
```

pop() — Remove from the END

```
let removed = arr.pop();
console.log(arr);     // Output: [1, 2, 3]
console.log(removed); // Output: 4  (pop returns the removed item)
```

unshift() — Add to the BEGINNING

```
arr.unshift(0);
console.log(arr); // Output: [0, 1, 2, 3]
```

shift() — Remove from the BEGINNING

```
let first = arr.shift();
console.log(arr);   // Output: [1, 2, 3]
console.log(first); // Output: 0  (shift also returns the removed item)
```

✅Remember: push/pop work at the END. unshift/shift work at the BEGINNING. pop and shift return the removed element.

Topic 5: slice() vs splice() — Most Confused in Interviews!

These two look similar but behave very differently. This is asked in almost every interview.

slice() — Copies a Part, Does NOT Change Original

slice(start, end) — takes from start index UP TO (but NOT including) the end index.

```
let arr = [10, 20, 30, 40, 50];
let part = arr.slice(1, 4);
console.log(part); // Output: [20, 30, 40]
console.log(arr);  // Output: [10, 20, 30, 40, 50]  (original NOT changed!)

// Negative index — count from end:
console.log(arr.slice(-2)); // Output: [40, 50]  (last 2 elements)
```

splice() — CHANGES the Original Array (add, remove, replace)

splice(start, deleteCount, ...itemsToAdd)

```
let arr = [10, 20, 30, 40, 50];

// Remove 2 elements starting at index 1:
let removed = arr.splice(1, 2);
console.log(arr);     // Output: [10, 40, 50]  (original IS changed!)
console.log(removed); // Output: [20, 30]      (removed items returned)
```

```
// Add elements at index 1 (delete 0, just insert):
arr.splice(1, 0, 99, 100);
console.log(arr); // Output: [10, 99, 100, 40, 50]

// Replace element at index 2:
arr.splice(2, 1, 999);
console.log(arr); // Output: [10, 99, 999, 40, 50]
```

✅Remember: slice = safe copy, does NOT touch original. splice = surgery on original, CHANGES it. This is the most asked difference!

Topic 6: Searching in Arrays

indexOf() — Find Position of an Element

```
let fruits = ["Apple", "Banana", "Mango", "Banana"];
console.log(fruits.indexOf("Banana"));     // Output: 1 (first match)
console.log(fruits.lastIndexOf("Banana")); // Output: 3 (last match)
console.log(fruits.indexOf("Grapes"));     // Output: -1 (not found)
```

Note: indexOf returns -1 when the item is NOT found. Always check for -1 in your code!

includes() — Does it Exist? Returns true/false

```
console.log(fruits.includes("Mango"));   // Output: true
console.log(fruits.includes("Grapes"));  // Output: false
```

find() — Find First Element That Matches a Condition

```
let numbers = [5, 12, 8, 130, 44];
let found = numbers.find(n => n > 10);
console.log(found); // Output: 12  (first number greater than 10)
```

findIndex() — Find Index of First Match

```
let index = numbers.findIndex(n => n > 10);
console.log(index); // Output: 1  (index of first number greater than 10)
```

Topic 7: map(), filter(), reduce() — The Power Trio (Must Know!)

These three are the most used and most asked array methods in interviews. Learn them very well.

map() — Transform Every Element, Get New Array

Think of it like this: you have a list of raw fruits. map() processes each one and gives you a new list. The original list is NEVER changed.

```
let numbers = [1, 2, 3, 4, 5];
let doubled = numbers.map(n => n * 2);
console.log(doubled); // Output: [2, 4, 6, 8, 10]
console.log(numbers); // Output: [1, 2, 3, 4, 5]  (original unchanged!)

// Real-world example — extract names from objects:
let users = [{ name: "Shaaz" }, { name: "Aman" }, { name: "Riya" }];
let names = users.map(user => user.name);
console.log(names); // Output: ["Shaaz", "Aman", "Riya"]
```

✅Remember: map() always returns a NEW array of the SAME length. Use it when you want to transform each item.

filter() — Keep Only What Passes the Test

Think of filter like a sieve. Only items that PASS the condition go through. Original array is NOT changed.

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8];
let evens = numbers.filter(n => n % 2 === 0);
console.log(evens); // Output: [2, 4, 6, 8]

// Real-world example — filter active users:
let users = [
  { name: "Shaaz", active: true  },
  { name: "Aman",  active: false },
  { name: "Riya",  active: true  }
];
let activeUsers = users.filter(user => user.active);
console.log(activeUsers); // Output: [{name:"Shaaz",...}, {name:"Riya",...}]
```

✅Remember: filter() returns a NEW array that is SMALLER than or equal to the original. Use it when you want to remove some items.

reduce() — Combine Everything Into ONE Value

Think of reduce like putting all items in a blender. You put many things in, one result comes out.

reduce takes two things: a function AND a starting value (called initial value).

```
let numbers = [1, 2, 3, 4, 5];
let sum = numbers.reduce((total, current) => total + current, 0);
//                        ^^^accumulator  ^^^current item    ^^^starting value
console.log(sum); // Output: 15

// Step by step what happens:
// Start: total = 0
// Step 1: total = 0 + 1 = 1
// Step 2: total = 1 + 2 = 3
// Step 3: total = 3 + 3 = 6
// Step 4: total = 6 + 4 = 10
// Step 5: total = 10 + 5 = 15

// Find maximum value using reduce:
let max = numbers.reduce((biggest, current) => current > biggest ? current : biggest,
0);
console.log(max); // Output: 5
```

Tip: reduce() is the most powerful but also the trickiest. Always remember the starting value (second argument). Forgetting it is the most common mistake.

Topic 8: forEach() vs map() — Very Commonly Asked Difference!

forEach() — Just Loop, Returns Nothing

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.forEach((fruit, index) => {
  console.log(`${index}: ${fruit}`);
});
// Output: 0: Apple   1: Banana   2: Mango

// forEach returns undefined — you cannot chain it:
let result = fruits.forEach(f => f.toUpperCase());
```

```
console.log(result); // Output: undefined
```

The Key Difference Table

forEach — loops through array, returns nothing (undefined), cannot chain

map — loops through array, returns a NEW array, can be chained

```
// map can be chained:
let result = [1, 2, 3]
  .map(n => n * 2)        // [2, 4, 6]
  .filter(n => n > 2);    // [4, 6]
console.log(result); // Output: [4, 6]
```

✅Remember: Use forEach when you just want to DO something with each item (like print it). Use map when you want to CREATE a new transformed array.

Topic 9: Sorting Arrays — Tricky Part!

sort() — Sorts the Original Array (CHANGES it!)

```
let fruits = ["Banana", "Apple", "Mango", "Cherry"];
fruits.sort();
console.log(fruits); // Output: ["Apple", "Banana", "Cherry", "Mango"]

// WARNING — sort() on numbers gives WRONG result by default:
let nums = [10, 1, 5, 100, 25];
nums.sort();
console.log(nums); // Output: [1, 10, 100, 25, 5]  WRONG! (sorted as text)

// CORRECT way to sort numbers:
nums.sort((a, b) => a - b);  // Ascending
console.log(nums); // Output: [1, 5, 10, 25, 100]  CORRECT!

nums.sort((a, b) => b - a);  // Descending
console.log(nums); // Output: [100, 25, 10, 5, 1]
```

✅Remember: sort() without a compare function converts items to strings first, which gives wrong results for numbers. Always use (a, b) => a - b for numbers.

reverse() — Flips the Array (CHANGES it!)

```
let arr = [1, 2, 3, 4, 5];
arr.reverse();
console.log(arr); // Output: [5, 4, 3, 2, 1]
```

Topic 10: Other Useful Methods

concat() — Merge Two Arrays

```
let a = [1, 2, 3];
let b = [4, 5, 6];
let merged = a.concat(b);
console.log(merged); // Output: [1, 2, 3, 4, 5, 6]
// Modern way using spread:
let merged2 = [...a, ...b];
console.log(merged2); // Output: [1, 2, 3, 4, 5, 6]
```

join() — Array to String

```
let words = ["Hello", "World", "JS"];
```

```
console.log(words.join(" ")); // Output: Hello World JS
console.log(words.join("-")); // Output: Hello-World-JS
console.log(words.join(""));  // Output: HelloWorldJS
```

flat() — Flatten Nested Arrays

Used to flatten arrays that are inside other arrays.

```
let nested = [1, [2, 3], [4, [5, 6]]];
console.log(nested.flat());      // Output: [1, 2, 3, 4, [5, 6]]  (one level deep)
console.log(nested.flat(2));     // Output: [1, 2, 3, 4, 5, 6]    (two levels deep)
console.log(nested.flat(Infinity)); // Output: [1, 2, 3, 4, 5, 6] (all levels)
```

every() and some()

every() — returns true only if ALL elements pass the condition.

some() — returns true if AT LEAST ONE element passes the condition.

```
let numbers = [2, 4, 6, 8, 10];
console.log(numbers.every(n => n % 2 === 0)); // Output: true  (all are even)
console.log(numbers.some(n => n > 8));        // Output: true  (10 is > 8)

let mixed = [1, 2, 3, 4, 5];
console.log(mixed.every(n => n % 2 === 0));   // Output: false (not all even)
console.log(mixed.some(n => n > 4));           // Output: true  (5 is > 4)
```

fill() — Fill Array with a Value

```
let arr = [1, 2, 3, 4, 5];
arr.fill(0, 2, 4);  // fill with 0 from index 2 to 4
console.log(arr);   // Output: [1, 2, 0, 0, 5]
```

Topic 11: Spread Operator and Destructuring — Asked a Lot!

Spread Operator (...) with Arrays

Spread operator opens up an array and spreads its elements out.

```
let a = [1, 2, 3];
let b = [4, 5, 6];
let combined = [...a, ...b];
console.log(combined); // Output: [1, 2, 3, 4, 5, 6]

// Copy an array (not reference — a real separate copy):
let original = [1, 2, 3];
let copy = [...original];
copy.push(4);
console.log(original); // Output: [1, 2, 3]  (not affected!)
console.log(copy);     // Output: [1, 2, 3, 4]
```

Tip: Always use [...arr] to copy an array. If you do let copy = arr, both point to the same array — changing one changes the other!

Array Destructuring — Pick Values by Position

Destructuring lets you pull out values from an array into separate variables in one line.

```
let fruits = ['Apple', 'Banana', 'Mango', 'Grapes'];

let [first, second] = fruits;
console.log(first);  // Output: Apple
```

```
console.log(second); // Output: Banana

// Skip elements using commas:
let [, , third] = fruits;
console.log(third);  // Output: Mango

// Get rest of array using ...:
let [head, ...rest] = fruits;
console.log(head); // Output: Apple
console.log(rest); // Output: ["Banana", "Mango", "Grapes"]

// Swap two variables — very cool trick:
let x = 1, y = 2;
[x, y] = [y, x];
console.log(x, y); // Output: 2  1
```

✅Remember: Array destructuring picks values by POSITION. Object destructuring picks values by NAME.

Topic 12: Array Interview Questions with Solutions

Q1: What is the difference between slice() and splice()?

slice — copies a part of array, does NOT change original, returns new array.

splice — cuts/adds/replaces in original array, CHANGES original, returns removed items.

```
let arr = [1, 2, 3, 4, 5];
console.log(arr.slice(1, 3));  // [2, 3]  — original safe
arr.splice(1, 2);              // removes 2 and 3
console.log(arr);              // [1, 4, 5]  — original changed
```

Q2: Remove Duplicates from an Array

```
let arr = [1, 2, 2, 3, 4, 4, 5];

// Method 1 — Set (cleanest way):
let unique = [...new Set(arr)];
console.log(unique); // Output: [1, 2, 3, 4, 5]

// Method 2 — filter:
let unique2 = arr.filter((item, index) => arr.indexOf(item) === index);
console.log(unique2); // Output: [1, 2, 3, 4, 5]
```

Q3: Flatten a Deeply Nested Array

```
let arr = [1, [2, [3, [4, [5]]]]];
console.log(arr.flat(Infinity)); // Output: [1, 2, 3, 4, 5]
```

Q4: Find the Second Largest Number in an Array

```
function secondLargest(arr) {
  let sorted = [...new Set(arr)].sort((a, b) => b - a);
  return sorted[1];
}
console.log(secondLargest([3, 1, 4, 1, 5, 9, 2])); // Output: 5
```

Q5: What is the difference between map() and forEach()?

map — returns a new array. forEach — returns nothing (undefined).

```
let arr = [1, 2, 3];
let mapped   = arr.map(n => n * 2);     // [2, 4, 6]
let forEached = arr.forEach(n => n * 2); // undefined
```

Q6: Count How Many Times Each Item Appears

```
let fruits = ['apple', 'banana', 'apple', 'mango', 'banana', 'apple'];
let count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
console.log(count); // Output: { apple: 3, banana: 2, mango: 1 }
```

Q7: Rotate an Array to the Right by K Steps

```
function rotateRight(arr, k) {
  let n = arr.length;
  k = k % n;
  return [...arr.slice(n - k), ...arr.slice(0, n - k)];
}
console.log(rotateRight([1, 2, 3, 4, 5], 2)); // Output: [4, 5, 1, 2, 3]
```

Q8: Check if Two Arrays are Equal

```
function arraysEqual(a, b) {
  if (a.length !== b.length) return false;
  return a.every((val, i) => val === b[i]);
}
console.log(arraysEqual([1,2,3], [1,2,3])); // Output: true
console.log(arraysEqual([1,2,3], [1,2,4])); // Output: false
```

Q9: What happens when you copy an array like this: let b = a?

```
let a = [1, 2, 3];
let b = a;          // b does NOT copy — it points to same array!
b.push(4);
console.log(a);     // Output: [1, 2, 3, 4]  — a is also changed!
console.log(b);     // Output: [1, 2, 3, 4]

// Correct way to copy:
let c = [...a];     // or a.slice()
c.push(5);
console.log(a);     // Output: [1, 2, 3, 4]  — a is safe!
console.log(c);     // Output: [1, 2, 3, 4, 5]
```

✅Remember: Arrays are stored by reference in JavaScript. let b = a means BOTH variables point to the same array. Always use spread [...arr] or .slice() to make a real copy.

Q10: Merge and Sort Two Sorted Arrays

```
function mergeSorted(a, b) {
  return [...a, ...b].sort((x, y) => x - y);
}
console.log(mergeSorted([1, 3, 5], [2, 4, 6])); // Output: [1, 2, 3, 4, 5, 6]
```

Topic 13: Quick Reference — All Array Methods at a Glance

push(item) — add to END, changes original

pop() — remove from END, returns removed item

unshift(item) — add to BEGINNING, changes original
shift() — remove from BEGINNING, returns removed item
slice(start, end) — copy portion, does NOT change original
splice(i, n) — remove/add/replace, CHANGES original
indexOf(val) — first position, -1 if not found
includes(val) — true or false check
find(fn) — first element that passes condition
findIndex(fn) — index of first element that passes condition
map(fn) — transform each item, returns NEW array
filter(fn) — keep items that pass condition, returns NEW array
reduce(fn, start) — combine all items into one value
forEach(fn) — loop only, returns nothing
sort(fn) — sort IN PLACE, changes original
reverse() — flip IN PLACE, changes original
concat(arr) — merge arrays, does NOT change original
join(sep) — array to string
flat(depth) — flatten nested arrays
every(fn) — true if ALL pass condition
some(fn) — true if ANY passes condition
fill(val, s, e) — fill with value, changes original

✅Remember: Most important for interviews: slice vs splice difference, map vs forEach difference, removing duplicates with Set, and reference vs copy of arrays.

Chapter 5: JavaScript Objects

How to Define in an Interview

"An Object in JavaScript is a collection of key-value pairs where keys are called properties and values can be of any data type — including strings, numbers, booleans, arrays, functions, or even other objects. Objects are used to represent real-world entities like a person, a car, or a user. Unlike arrays which are ordered by index, objects are accessed by key name. Objects in JavaScript are mutable and are stored by reference, not by value."

Example you can say in interview:

```
let person = {
  name: "Shaaz",      // key: value pair — string
  age: 23,            // key: value pair — number
  isStudent: true,    // key: value pair — boolean
  greet: function() { // key: value pair — function (called a method)
    return `Hello, I am ${this.name}`;
  }
};
console.log(person.name);     // Output: Shaaz  (dot notation)
console.log(person["age"]);   // Output: 23     (bracket notation)
console.log(person.greet());  // Output: Hello, I am Shaaz
```

Topic 1: What is an Object? (Super Simple)

Think of an object like an Aadhaar card. It has different fields — name, age, address, ID number. All these details belong to ONE person and are stored together.

In JavaScript, an object stores related information together using key-value pairs.

Key = the name of the field (like 'name', 'age')

Value = the actual data stored in that field

```
let student = {
  name:    "Shaaz",   // key is name,    value is "Shaaz"
  age:     23,        // key is age,      value is 23
  city:    "Delhi",   // key is city,     value is "Delhi"
  passing: true       // key is passing,  value is true
};
```

Tip: Object keys are always strings (or Symbols). Values can be anything — string, number, boolean, array, function, or another object.

Topic 2: 4 Ways to Create an Object — Must Know!

Way 1 — Object Literal (Most Common Way)

Just write key-value pairs inside curly braces. This is how 90% of objects are created.

```
let car = {
  brand: "Toyota",
  color: "Red",
  speed: 180
};
```

Way 2 — new Object() (Old Way, Rarely Used)

```
let car = new Object();
car.brand = "Toyota";
```

```
car.color = "Red";
```

Way 3 — Constructor Function (When You Need Many Similar Objects)

When you want to create many objects of the same type — like 100 students — use a constructor function.

```
function Student(name, age, city) {
  this.name = name;
  this.age  = age;
  this.city = city;
}
let s1 = new Student("Shaaz", 23, "Delhi");
let s2 = new Student("Aman",  25, "Mumbai");
console.log(s1.name); // Output: Shaaz
console.log(s2.name); // Output: Aman
```

Note: The 'new' keyword creates a brand new object. 'this' inside the function refers to that new object.

Way 4 — Object.create() (For Prototype-Based Inheritance)

Creates a new object and sets another object as its prototype (parent).

```
let animal = {
  eat: function() { console.log("Eating..."); }
};
let dog = Object.create(animal);  // dog inherits from animal
dog.bark = function() { console.log("Woof!"); };
dog.eat();  // Output: Eating...  (inherited from animal)
dog.bark(); // Output: Woof!      (dog's own method)
```

Topic 3: Accessing and Modifying Properties

Dot Notation vs Bracket Notation

There are two ways to access a property. Both work, but they have different use cases.

```
let person = { name: "Shaaz", age: 23, "home city": "Delhi" };

// Dot Notation — clean and simple, used most of the time
console.log(person.name);  // Output: Shaaz

// Bracket Notation — used when key has spaces or key is dynamic
console.log(person["home city"]); // Output: Delhi  (key has a space!)

// Dynamic key — when the key name is stored in a variable:
let key = "age";
console.log(person[key]); // Output: 23
```

✅Remember: Use dot notation normally. Use bracket notation when the key has a space, special character, or when the key is stored in a variable.

Adding, Updating, and Deleting Properties

```
let person = { name: "Shaaz", age: 23 };

person.city = "Delhi";   // ADD a new property
person.age  = 24;        // UPDATE an existing property
delete person.city;      // DELETE a property
```

```
console.log(person); // Output: { name: "Shaaz", age: 24 }
```

Topic 4: Methods — Functions Inside Objects

When a function is stored inside an object, it is called a METHOD. Inside a method, 'this' refers to the object itself.

```
let calculator = {
  value: 0,
  add: function(n) {
    this.value += n;   // "this" means the calculator object
    return this;       // return "this" allows chaining
  },
  subtract: function(n) {
    this.value -= n;
    return this;
  },
  result: function() {
    return this.value;
  }
};

console.log(calculator.add(10).add(5).subtract(3).result()); // Output: 12
```

Tip: Returning 'this' from a method allows METHOD CHAINING — calling multiple methods in one line. This is asked in interviews!

Topic 5: Object Destructuring — Very Commonly Asked!

Destructuring lets you pull out properties from an object into separate variables in one clean line.

```
let person = { name: "Shaaz", age: 23, city: "Delhi" };

// Old way — verbose
let name = person.name;
let age  = person.age;

// New way — destructuring (clean and short)
let { name, age } = person;
console.log(name); // Output: Shaaz
console.log(age);  // Output: 23

// Rename while destructuring:
let { name: fullName, age: years } = person;
console.log(fullName); // Output: Shaaz
console.log(years);    // Output: 23

// Default value if property does not exist:
let { name, country = "India" } = person;
console.log(country); // Output: India  (person has no country, so default used)

// Destructuring in function parameters — very common in React!
function greet({ name, age }) {
  return `Hello ${name}, you are ${age} years old.`;
}
console.log(greet(person)); // Output: Hello Shaaz, you are 23 years old.
```

Topic 6: Spread Operator and Rest with Objects

Spread — Copy or Merge Objects

The spread operator (...) opens up an object and spreads all its properties into a new one.

```
let person = { name: "Shaaz", age: 23 };

// Copy an object:
let copy = { ...person };
copy.age = 99;
console.log(person.age); // Output: 23  (original NOT affected!)
console.log(copy.age);   // Output: 99

// Merge two objects:
let address  = { city: "Delhi", country: "India" };
let fullInfo = { ...person, ...address };
console.log(fullInfo);
// Output: { name:"Shaaz", age:23, city:"Delhi", country:"India" }

// Override a property while spreading:
let updated = { ...person, age: 25 };
console.log(updated); // Output: { name:"Shaaz", age:25 }
```

✅Remember: Just like arrays, objects are stored by reference. let b = a means both point to the SAME object. Use spread {...obj} to make a real copy.

Rest in Object Destructuring

Rest (...) collects the remaining properties that you did not destructure.

```
let person = { name: "Shaaz", age: 23, city: "Delhi", job: "Dev" };

let { name, age, ...rest } = person;
console.log(name); // Output: Shaaz
console.log(age);  // Output: 23
console.log(rest); // Output: { city: "Delhi", job: "Dev" }
```

Topic 7: Looping Through an Object

for...in Loop — Loops Through Keys

```
let person = { name: "Shaaz", age: 23, city: "Delhi" };

for (let key in person) {
  console.log(key + " : " + person[key]);
}
// Output:
// name : Shaaz
// age  : 23
// city : Delhi
```

Object.keys(), Object.values(), Object.entries()

```
let car = { brand: "Honda", color: "Blue", speed: 150 };

console.log(Object.keys(car));
// Output: ["brand", "color", "speed"]   — only keys
```

```
console.log(Object.values(car));
// Output: ["Honda", "Blue", 150]         — only values

console.log(Object.entries(car));
// Output: [["brand","Honda"], ["color","Blue"], ["speed",150]]

// Loop using entries — most powerful way:
Object.entries(car).forEach(([key, value]) => {
  console.log(`${key} => ${value}`);
});
```

Tip: Object.keys(), Object.values(), Object.entries() are very commonly asked in interviews. Remember that entries() returns an array of [key, value] pairs.

Topic 8: Checking Properties in an Object

'in' Operator — Check if Key Exists

```
let person = { name: "Shaaz", age: 23 };

console.log("name" in person);    // Output: true
console.log("salary" in person);  // Output: false
```

hasOwnProperty() — Check if Property Belongs to This Object

```
console.log(person.hasOwnProperty("name"));   // Output: true
console.log(person.hasOwnProperty("salary")); // Output: false
```

Note: 'in' checks the object AND its prototype chain. hasOwnProperty() checks ONLY the object itself. This is an interview gotcha!

Topic 9: Object.freeze() vs Object.seal() — Interview Question!

Object.freeze() — No Changes At All

After freezing, you CANNOT add, update, or delete any property. The object is completely locked.

```
let config = { theme: "dark", lang: "en" };
Object.freeze(config);

config.theme = "light"; // Silently ignored!
config.font  = "Arial"; // Silently ignored!
delete config.lang;     // Silently ignored!

console.log(config); // Output: { theme: "dark", lang: "en" }  — unchanged!
console.log(Object.isFrozen(config)); // Output: true
```

Object.seal() — Can Update, Cannot Add or Delete

After sealing, you CAN update existing properties, but you CANNOT add new ones or delete existing ones.

```
let user = { name: "Shaaz", age: 23 };
Object.seal(user);

user.name = "Aman"; // ALLOWED — updating existing property
user.city = "Delhi"; // Silently ignored — cannot ADD new
delete user.age;     // Silently ignored — cannot DELETE
```

```
console.log(user); // Output: { name: "Aman", age: 23 }
```

✅Remember: freeze = completely locked, nothing can change. seal = can only UPDATE existing properties, cannot add or delete. Very commonly asked difference!

Topic 10: Shallow Copy vs Deep Copy — Very Important!

This is one of the trickiest and most asked object topics in interviews.

Shallow Copy — Only First Level is Copied

When you copy an object using spread {...} or Object.assign(), only the top-level properties are truly copied. If a property is itself an object (nested), it is still shared — not truly copied.

```
let original = {
  name: "Shaaz",
  address: { city: "Delhi" }  // nested object
};

let copy = { ...original };   // shallow copy

copy.name = "Aman";           // top-level — safe, original not affected
copy.address.city = "Mumbai"; // nested — CHANGES ORIGINAL TOO!

console.log(original.name);         // Output: Shaaz  (safe)
console.log(original.address.city); // Output: Mumbai (CHANGED! Not safe!)
```

Note: Spread and Object.assign() both do SHALLOW copy only. Nested objects are still shared between original and copy.

Deep Copy — All Levels are Truly Copied

A deep copy means EVERY level is independently copied. Changes to copy will NEVER affect original.

```
// Method 1 — JSON trick (works for simple objects, no functions)
let deepCopy = JSON.parse(JSON.stringify(original));

deepCopy.address.city = "Chennai";
console.log(original.address.city); // Output: Delhi  (safe now!)

// Method 2 — structuredClone (modern, best way)
let deepCopy2 = structuredClone(original);
deepCopy2.address.city = "Pune";
console.log(original.address.city); // Output: Delhi  (still safe!)
```

✅Remember: Shallow copy = only top level is safe. Deep copy = all levels are safe. JSON trick does not work if object has functions or undefined values.

Topic 11: Optional Chaining (?.) and Nullish Coalescing (??) — Modern Must Know!

Optional Chaining (?.) — Safely Access Nested Properties

When you try to access a property of something that is null or undefined, JavaScript throws an error. Optional chaining prevents that crash.

```
let user = {
  name: "Shaaz",
  address: {
```

```
    city: "Delhi"
  }
};

// Without optional chaining — CRASHES if address is missing:
console.log(user.phone.number); // ERROR! Cannot read property of undefined

// With optional chaining — returns undefined safely, no crash:
console.log(user?.phone?.number); // Output: undefined  (no crash!)
console.log(user?.address?.city); // Output: Delhi      (works fine!)
```

Tip: Always use ?. when accessing deeply nested properties from API responses. You never know which field might be missing.

Nullish Coalescing (??) — Default Value When null or undefined

?? returns the RIGHT side only when the LEFT side is null or undefined. It is stricter than || (OR operator).

```
let username = null;
console.log(username ?? "Guest");   // Output: Guest  (null → use default)

let score = 0;
console.log(score ?? "No score");   // Output: 0      (0 is valid, not null!)
console.log(score || "No score");   // Output: No score  (0 is falsy, || is wrong
here)
```

✅Remember: ?? only treats null and undefined as missing. || treats 0, false, empty string as missing too. In most real cases, ?? is the correct choice.

Topic 12: Computed Property Names — Dynamic Keys

You can use a variable as a key name when creating an object by putting it inside square brackets.

```
let field = "username";
let value = "Shaaz";

let obj = {
  [field]: value    // computed property — key comes from variable
};
console.log(obj); // Output: { username: "Shaaz" }

// Real-world use — building a form data object dynamically:
function createField(key, val) {
  return { [key]: val };
}
console.log(createField("email", "shaaz@mail.com"));
// Output: { email: "shaaz@mail.com" }
```

Topic 13: Object Interview Questions with Solutions

Q1: What is the difference between dot notation and bracket notation?

Dot notation is simple and clean. Bracket notation is used when the key has spaces, special characters, or when the key name is stored in a variable.

```
obj.name       // dot — works when key is a simple identifier
obj["name"]    // bracket — same result
obj["my key"]  // bracket — MUST use this for keys with spaces
```

```
obj[variable]  // bracket — MUST use this when key is a variable
```

Q2: What is the difference between Object.freeze() and Object.seal()?

freeze — nothing can be added, updated, or deleted. Completely locked.

seal — existing properties can be updated but cannot add or delete.

```
let obj = { name: "Shaaz" };
Object.freeze(obj);
obj.name = "Aman";  // Ignored — cannot update when frozen

let obj2 = { name: "Shaaz" };
Object.seal(obj2);
obj2.name = "Aman"; // WORKS — can update when sealed
```

Q3: What is the difference between shallow copy and deep copy?

Shallow copy — only top-level properties are copied. Nested objects are still shared.

Deep copy — every level is fully independent. No sharing at all.

```
// Shallow:
let copy = { ...original };
// Deep:
let deepCopy = structuredClone(original);
```

Q4: How do you check if a key exists in an object?

```
let person = { name: "Shaaz", age: 23 };

// Method 1 — "in" operator
console.log("name" in person);            // Output: true

// Method 2 — hasOwnProperty
console.log(person.hasOwnProperty("age")); // Output: true

// Method 3 — check for undefined
console.log(person.salary !== undefined);  // Output: false
```

Q5: What will this output? (Reference Trap — Very Tricky!)

```
let a = { x: 10 };
let b = a;
b.x = 99;
console.log(a.x); // What is the output?
```

Answer: 99

Why? Because b = a does NOT create a copy. Both a and b point to the SAME object. Changing b.x also changes a.x.

```
// Fix — use spread to make a real copy:
let c = { ...a };
c.x = 50;
console.log(a.x); // Output: 99  (a is safe now)
```

Q6: Merge two objects and if same key exists, keep the second one's value

```
let obj1 = { a: 1, b: 2, c: 3 };
let obj2 = { b: 99, d: 4 };

let merged = { ...obj1, ...obj2 };
```

```
console.log(merged); // Output: { a:1, b:99, c:3, d:4 }
```

When both objects have the same key (b), the second spread wins and its value (99) is kept.

Q7: Convert an object into an array of key-value pairs and back

```
let person = { name: "Shaaz", age: 23, city: "Delhi" };

// Object to array of pairs:
let pairs = Object.entries(person);
// Output: [["name","Shaaz"], ["age",23], ["city","Delhi"]]

// Array of pairs back to object:
let backToObj = Object.fromEntries(pairs);
console.log(backToObj); // Output: { name:"Shaaz", age:23, city:"Delhi" }
```

Q8: Count the number of properties in an object

```
let person = { name: "Shaaz", age: 23, city: "Delhi" };
console.log(Object.keys(person).length); // Output: 3
```

Q9: What is optional chaining? Why is it useful?

Optional chaining (?.) safely accesses a nested property without throwing an error if the parent is null or undefined. It returns undefined instead of crashing.

```
let user = { profile: null };

console.log(user.profile.name);   // ERROR! Cannot read properties of null
console.log(user?.profile?.name); // Output: undefined  (no crash!)
```

Q10: Create a function that deeply clones an object

```
function deepClone(obj) {
  return JSON.parse(JSON.stringify(obj));
}

let original = { name: "Shaaz", address: { city: "Delhi" } };
let clone = deepClone(original);
clone.address.city = "Mumbai";
console.log(original.address.city); // Output: Delhi  (safe!)
```

Note: JSON trick does not work if the object has functions, undefined, or Date objects. Use structuredClone() for a more complete solution.

Topic 14: Quick Reference — All Object Concepts at a Glance

{ key: value } — object literal, most common way to create

obj.key — dot notation to access property

obj["key"] — bracket notation, use for dynamic or special keys

obj.key = value — add or update a property

delete obj.key — remove a property

"key" in obj — check if key exists (includes prototype chain)

obj.hasOwnProperty('key') — check if key is OWN property only

Object.keys(obj) — returns array of all keys

Object.values(obj) — returns array of all values

Object.entries(obj) — returns array of [key, value] pairs
Object.fromEntries(arr) — converts [key, value] array back to object
Object.assign(target, src) — copies properties from src to target
{ ...obj } — spread, shallow copy
Object.freeze(obj) — completely lock, nothing can change
Object.seal(obj) — only updates allowed, no add or delete
JSON.parse(JSON.stringify()) — deep copy (no functions or undefined)
structuredClone(obj) — modern deep copy, handles more types
obj?.key?.nested — optional chaining, safe access
value ?? 'default' — nullish coalescing, fallback for null/undefined

✅Remember: Most asked in interviews: dot vs bracket notation, shallow vs deep copy, freeze vs seal, reference trap, optional chaining, and Object.keys/values/entries.

Chapter 6: Scope in JavaScript

How to Define in an Interview

"Scope in JavaScript refers to the accessibility or visibility of variables — it defines where a variable can be read and used in the code. JavaScript has three types of scope: Global Scope (variable is accessible everywhere), Function Scope (variable is accessible only inside the function it was declared in), and Block Scope (variable is accessible only inside the { } block it was declared in, applicable to let and const). JavaScript also has a Scope Chain — when a variable is not found in the current scope, JavaScript automatically looks in the outer (parent) scope, then the outer of that, all the way up to the global scope."

Example you can say in interview:

```
let globalVar = "I am global";        // global scope

function myFunction() {
  let funcVar = "I am function scoped"; // function scope

  if (true) {
    let blockVar = "I am block scoped"; // block scope
    console.log(blockVar);  // Works — inside block
  }
  // console.log(blockVar); // ERROR — outside block
}
```

Topic 1: What is Scope? (Super Simple Explanation)

Think of scope like rooms in a house. If something is in the living room (global), everyone in the house can see it. If something is in your bedroom (function), only you can see it from inside your room — nobody outside can access it.

Scope = the area of your code where a variable is alive and can be used.

If you try to use a variable outside its scope, JavaScript throws a ReferenceError — it simply cannot find it.

```
function myRoom() {
  let secret = "hidden";  // only exists inside this function
}

console.log(secret); // ReferenceError: secret is not defined
```

Tip: Scope protects your variables. Variables inside a function are private to that function — outside code cannot accidentally read or overwrite them.

Topic 2: The 3 Types of Scope — Must Know All Three

Type 1 — Global Scope

A variable declared OUTSIDE of any function or block is in the global scope. It can be accessed from ANYWHERE in the entire code — inside functions, inside blocks, everywhere.

```
let username = "Shaaz";   // global variable

function greet() {
  console.log(username);  // accessible inside function
}
```

```
if (true) {
  console.log(username);  // accessible inside block
}

greet();
console.log(username);    // accessible at global level
// All print: Shaaz
```

Note: Too many global variables are BAD practice. Any part of your code can accidentally change a global variable. This creates bugs that are very hard to find. Keep variables as local as possible.

Type 2 — Function Scope

A variable declared INSIDE a function exists only inside that function. It is created when the function runs and destroyed when the function ends. Outside code has NO access to it.

```
function calculateTax() {
  let taxRate = 0.18;      // only lives inside this function
  let amount  = 1000;
  let tax     = amount * taxRate;
  console.log("Tax:", tax); // Output: Tax: 180
}

calculateTax();
console.log(taxRate); // ReferenceError! taxRate does not exist here
```

Each time you call the function, fresh copies of the variables are created. When the function ends, they are all gone.

```
function counter() {
  let count = 0;    // fresh copy each time
  count++;
  console.log(count);
}

counter(); // Output: 1
counter(); // Output: 1  (fresh copy — does NOT remember previous call)
counter(); // Output: 1
```

Type 3 — Block Scope (let and const only)

A block is anything inside curly braces { }. This includes if blocks, for loops, while loops, and even just standalone { } braces.

Variables declared with let or const inside a block ONLY exist inside that block.

```
if (true) {
  let blockMessage = "Inside if block";
  const PI = 3.14;
  console.log(blockMessage); // Works — inside block
}
console.log(blockMessage); // ReferenceError! Cannot access outside

// Very common in for loops:
for (let i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}
console.log(i); // ReferenceError! i does not exist here
```

✅Remember: let and const are block-scoped. var is NOT block-scoped — it leaks out of blocks. This is the most important difference between var and let/const.

Topic 3: var vs let vs const — The Scope Difference (Very Important!)

This is one of the most asked topics in JavaScript interviews. You must know the exact difference.

var — Function Scoped, NOT Block Scoped (The Problematic One)

```
// var leaks out of blocks:
if (true) {
  var leaky = "I escaped!";
}
console.log(leaky); // Output: I escaped!  (var ignores block scope!)

// var in a for loop leaks too:
for (var i = 0; i < 3; i++) {
  // i exists here
}
console.log(i); // Output: 3  (i leaked out of the loop!)
```

let — Block Scoped (The Modern Way)

```
if (true) {
  let safe = "I stay inside!";
}
console.log(safe); // ReferenceError! Cannot access outside block

for (let j = 0; j < 3; j++) {
  // j exists here
}
console.log(j); // ReferenceError! j is gone after the loop
```

const — Block Scoped + Cannot be Reassigned

```
if (true) {
  const MAX = 100;
  // MAX = 200; // TypeError! Cannot reassign const
}
console.log(MAX); // ReferenceError! Cannot access outside block
```

Classic Bug with var in Loops — Very Tricky Interview Question!

```
// With var — BUGGY:
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 3, 3, 3  (NOT 0, 1, 2 — because var is shared!)

// With let — CORRECT:
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 0, 1, 2  (each iteration has its OWN i)
```

Why does var give 3, 3, 3? Because var is function-scoped, there is only ONE i shared across all iterations. By the time setTimeout runs after 1 second, the loop is already finished and i is 3.

Why does let give 0, 1, 2? Because let is block-scoped, each iteration creates its OWN separate i. Each setTimeout captures its own copy.

> **✅Remember: This var vs let in setTimeout loop question is one of the most commonly asked tricky interview questions. Always use let in loops!**

Topic 4: Scope Chain — How JavaScript Finds Variables

When you use a variable, JavaScript does not just look in the current scope. If it cannot find it there, it goes UP to the parent scope. Then to the parent's parent. All the way up to global. If still not found, it throws a ReferenceError.

This chain of looking upward is called the Scope Chain.

```
let globalName = "Global";

function outer() {
  let outerName = "Outer";

  function inner() {
    let innerName = "Inner";

    // inner can access ALL of these:
    console.log(innerName);  // found in own scope
    console.log(outerName);  // not in own scope → goes up → found in outer
    console.log(globalName); // not in outer → goes up again → found in global
  }

  inner();
}

outer();
```

Scope Chain goes: inner → outer → global. It NEVER goes downward — a parent cannot access a child's variables.

```
function parent() {
  function child() {
    let childSecret = "private";
  }
  console.log(childSecret); // ReferenceError! Parent cannot see child
}
```

✅Remember: Scope chain goes UPWARD only — inner functions can see outer variables, but outer functions can NEVER see inner variables.

Topic 5: Lexical Scope — Where You WRITE the Code Matters

Lexical scope means the scope of a variable is determined by WHERE you WRITE the code — not where you call it from.

In simple words: a function remembers the scope where it was BORN (written), not where it is CALLED.

```
let city = "Delhi";

function showCity() {
  console.log(city); // looks at where showCity was WRITTEN (global level)
}

function changeCity() {
  let city = "Mumbai"; // local variable
  showCity();          // calling showCity from here
}
```

```
changeCity(); // Output: Delhi  (NOT Mumbai!)
```

Even though showCity() is CALLED from inside changeCity() where city = 'Mumbai', it still prints 'Delhi'. Why? Because showCity was WRITTEN at the global level, so it uses the global city variable.

> *Note: This is lexical scoping — also called static scoping. JavaScript decides scope at write time, not at run time. This concept is the foundation of closures.*

Topic 6: Hoisting — Variables and Functions Move to the Top

Hoisting is JavaScript's behavior of moving variable and function DECLARATIONS to the top of their scope before the code runs. Only declarations are hoisted — not the values.

Function Declarations are Fully Hoisted

```
greet(); // Output: Hello!  — works even before the function is written!

function greet() {
  console.log("Hello!");
}
```

The entire function body is hoisted — you can call it anywhere in its scope.

var is Hoisted but Value is NOT

```
console.log(name); // Output: undefined  (NOT an error, but NOT the value either)

var name = "Shaaz";

console.log(name); // Output: Shaaz
```

What happens: JavaScript hoists 'var name' to the top of its scope and sets it to undefined. The assignment (= 'Shaaz') stays where it is. So accessing it before assignment gives undefined, not an error.

let and const are Hoisted but in Temporal Dead Zone (TDZ)

```
console.log(age); // ReferenceError: Cannot access before initialization

let age = 23;
```

let and const are also hoisted, but they go into a Temporal Dead Zone (TDZ) — a period from the start of the block until the line where they are initialized. Any access during TDZ throws a ReferenceError.

> **✅Remember: var hoisting = hoisted as undefined (no error but wrong value). let/const hoisting = hoisted into TDZ (ReferenceError if accessed before declaration). Function declaration = fully hoisted (works everywhere in scope).**

The Temporal Dead Zone (TDZ) in Detail

```
// TDZ starts here for "score"
{
  // TDZ zone — accessing score here throws ReferenceError
  console.log(score); // ReferenceError!

  let score = 100;    // TDZ ends here — score is now initialized

  console.log(score); // Output: 100  — works fine now
}
```

Tip: TDZ is why let and const are safer than var. They tell you immediately if you try to use a variable before declaring it, instead of silently giving undefined like var does.

Topic 7: Variable Shadowing — Inner Hides Outer

When an inner scope creates a variable with the SAME NAME as an outer scope variable, the inner one 'shadows' the outer one. Inside the inner scope, the outer variable is hidden — but it is NOT changed or deleted.

```
let color = "Red";      // global

function paintWall() {
  let color = "Blue";   // shadows the global color inside this function
  console.log(color);   // Output: Blue  (sees the inner one)
}

paintWall();
console.log(color);     // Output: Red  (global is unchanged!)

// Shadowing inside a block:
let score = 100;
{
  let score = 999;      // shadows outer score
  console.log(score);   // Output: 999
}
console.log(score);     // Output: 100  (outer unchanged)
```

Note: Shadowing is allowed in JavaScript. The inner variable does not change the outer one — they are completely separate. Be careful with shadowing — it can be confusing and cause bugs.

Topic 8: Scope Interview Questions with Solutions

Q1: What are the 3 types of scope in JavaScript?

Global Scope — variable accessible everywhere.

Function Scope — variable accessible only inside the function it was declared in.

Block Scope — variable accessible only inside the { } block (let and const only).

Q2: What is the difference between var, let, and const regarding scope?

var — function scoped. Ignores block scope. Leaked out of if blocks and for loops.

let — block scoped. Stays inside { }. Cannot be accessed outside the block.

const — block scoped. Same as let but value cannot be reassigned.

```
if (true) {
  var a = 1;   // leaks out
  let b = 2;   // stays inside
  const c = 3; // stays inside
}
console.log(a); // 1  — var leaked
console.log(b); // ReferenceError
console.log(c); // ReferenceError
```

Q3: What is the Scope Chain?

When JavaScript cannot find a variable in the current scope, it looks in the parent scope, then the grandparent scope, all the way up to the global scope. This chain of lookups is called the Scope Chain. It only goes UPWARD — never downward.

Q4: What is Lexical Scope?

Lexical scope means a function's scope is determined by WHERE it is written in the code — not where it is called from. JavaScript uses lexical scoping, which means scope is fixed at write time.

```
let x = "global";
function foo() { console.log(x); } // x refers to global x (where foo is written)
function bar() { let x = "local"; foo(); } // calling foo from here
bar(); // Output: global  (NOT local — because foo was WRITTEN at global level)
```

Q5: What is Hoisting? What is the difference between var and let hoisting?

Hoisting is JavaScript moving declarations to the top of their scope before execution.

var — hoisted as undefined. You can access it before declaration but get undefined.

let/const — hoisted into Temporal Dead Zone. Accessing before declaration throws ReferenceError.

Function declarations — fully hoisted. Can be called before they are written.

Q6: What is the Temporal Dead Zone (TDZ)?

TDZ is the period between the start of a block and the line where a let/const variable is initialized. During TDZ, the variable exists in memory but cannot be accessed — any attempt throws a ReferenceError. This makes let/const safer than var.

Q7: What will this output? (Classic var in loop bug)

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

Answer: 3, 3, 3

Because var is function-scoped, there is only ONE i. By the time the setTimeout callbacks run after 100ms, the loop has already finished and i = 3. All three callbacks share the same i.

```
// Fix with let:
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

Answer: 0, 1, 2 — because let creates a new i for each iteration.

Q8: What is Variable Shadowing?

When an inner scope declares a variable with the same name as an outer scope variable, the inner one shadows (hides) the outer one inside that scope. The outer variable is not changed — it is just hidden temporarily.

```
let name = "Global Shaaz";
function test() {
  let name = "Local Shaaz";  // shadows global name
  console.log(name);         // Output: Local Shaaz
}
test();
console.log(name); // Output: Global Shaaz  (global unchanged)
```

Q9: Can a function access variables from another function at the same level?

No. Functions at the same level cannot access each other's variables. Scope only goes UPWARD to the parent — never sideways to a sibling function.

```
function funcA() {
  let secret = "A secret";
}

function funcB() {
  console.log(secret); // ReferenceError! funcB cannot see funcA variables
}

funcA();
funcB();
```

Q10: What happens when you use a variable without declaring it?

```
function test() {
  undeclared = "I have no var/let/const!";
}
test();
console.log(undeclared); // Output: I have no var/let/const!
```

In non-strict mode, assigning to an undeclared variable creates it as a GLOBAL variable — even if the assignment is inside a function! This is a very dangerous and silent bug.

```
"use strict";
function test() {
  undeclared = "oops"; // ReferenceError in strict mode — caught immediately!
}
```

Tip: Always use 'use strict' or modern ES Modules (which are strict by default) to prevent accidentally creating global variables.

Topic 9: Quick Reference — Scope at a Glance

Global Scope — declared outside all functions/blocks, accessible everywhere

Function Scope — declared inside a function, only accessible inside that function

Block Scope — declared with let/const inside {}, only accessible inside {}

var — function scoped, hoisted as undefined, leaks from blocks

let — block scoped, hoisted into TDZ, can be reassigned

const — block scoped, hoisted into TDZ, cannot be reassigned

Scope Chain — JavaScript looks upward from inner to outer to global

Lexical Scope — scope determined by WHERE code is written, not where called

Hoisting — declarations moved to top of scope before execution

TDZ — let/const exist but cannot be accessed before initialization

Variable Shadowing — inner variable with same name hides the outer one

✅Remember: Most asked scope topics in interviews: var vs let vs const scope difference, the var in setTimeout loop bug, hoisting and TDZ, lexical scope, scope chain, and what happens with undeclared variables in strict vs non-strict mode.

Chapter 7: The 'this' Keyword

How to Define in an Interview

"'this' is a special keyword in JavaScript that refers to the object that is currently executing the function. Its value is NOT fixed — it changes depending on HOW and WHERE the function is called, not where it is defined. There are 5 main contexts where 'this' behaves differently: Global context, Object method, Regular function, Arrow function, and Event listener. You can also manually control 'this' using call(), apply(), and bind()."

The Golden Rule — always remember this one line:

"this" always refers to the object that is LEFT of the dot when the function is called.

Topic 1: The Story That Makes 'this' Crystal Clear Forever

Imagine a chef named Ramesh. When he is cooking in Hotel Taj, he uses Hotel Taj's kitchen, Hotel Taj's ingredients, Hotel Taj's equipment. 'this' for him is Hotel Taj.

The same Ramesh goes to cook at Hotel Oberoi the next day. Now 'this' for him is Hotel Oberoi — Hotel Oberoi's kitchen, ingredients, equipment.

Ramesh (the function) is the same. But 'this' changes based on WHERE he is working (which object called him).

In JavaScript, 'this' is like asking: 'Who hired this function right now? Who is the boss at this moment?' That boss is 'this'.

```
const hotelTaj = {
  name: "Hotel Taj",
  cook: function() {
    console.log(this.name + " is cooking!"); // this = hotelTaj
  }
};
const hotelOberoi = { name: "Hotel Oberoi", cook: hotelTaj.cook };
hotelTaj.cook();    // Output: Hotel Taj is cooking!    (this = hotelTaj)
hotelOberoi.cook(); // Output: Hotel Oberoi is cooking! (this = hotelOberoi)
```

✅Remember: 'this' is not about WHERE the function is written. It is about WHO calls the function. The caller is 'this'.

Topic 2: Context 1 — 'this' in Global Scope

When you use 'this' outside of any function — at the top level — it refers to the global object. In a browser that is 'window'. In Node.js it is 'global'.

```
console.log(this);            // Output: Window object
console.log(this === window); // Output: true

var city = "Delhi";
console.log(this.city);       // Output: Delhi  (var attaches to window)

let country = "India";
console.log(this.country);    // Output: undefined  (let does NOT attach to window)
```

Topic 3: Context 2 — 'this' Inside an Object Method

When a function is called as a METHOD of an object using dot notation, 'this' refers to that object. Golden Rule: 'this' = the object LEFT of the dot.

```
const student = {
  name: "Shaaz",
  marks: 85,
  introduce: function() {
    console.log(`I am ${this.name} with ${this.marks} marks`);
    //              ^^^ this = student (student.introduce() was called)
  }
};
student.introduce(); // Output: I am Shaaz with 85 marks

// Same function, different owner — this changes:
const person2 = { name: "Aman", marks: 90 };
person2.introduce = student.introduce;
person2.introduce(); // Output: I am Aman with 90 marks  (this = person2)
```

Tip: Same function. Two different 'this' values. Because different objects are calling it. This is the heart of 'this'.

Topic 4: Context 3 — 'this' in a Regular Standalone Function

When a regular function is called WITHOUT any object (no dot notation), 'this' depends on mode:

Non-strict mode → 'this' = window (global object)

Strict mode → 'this' = undefined

```
function showThis() {
  console.log(this);           // window (non-strict)
}
showThis();

"use strict";
function showStrict() {
  console.log(this);           // undefined (strict)
}
showStrict();
```

The 'this' Lost Bug — Most Common Mistake

```
const user = {
  name: "Shaaz",
  greet: function() { console.log("Hello " + this.name); }
};
user.greet();          // Output: Hello Shaaz  ✓

let fn = user.greet;   // extracted — standalone function now
fn();                  // Output: Hello undefined ✗  (this = window)

// Fix — bind this permanently:
let fn2 = user.greet.bind(user);
fn2(); // Output: Hello Shaaz ✓
```

✅Remember: When you extract a method from an object and call it alone, 'this' is lost. The dot notation is gone, so JavaScript has no idea who the owner is. Fix it with bind().

Topic 5: Context 4 — 'this' in Arrow Functions (Most Important!)

Arrow functions do NOT have their own 'this' at all. They inherit 'this' from the outer scope where they were WRITTEN. This is called Lexical 'this'.

Think of it: arrow functions are born without a name badge. They wear their parent's name badge instead.

Arrow as Object Method — WRONG Use

```
const person = {
  name: "Shaaz",
  greetArrow:   () => { console.log(this.name); }, // this = window ✗
  greetRegular: function() { console.log(this.name); } // this = person ✓
};
person.greetArrow();   // Output: undefined ✗
person.greetRegular(); // Output: Shaaz ✓
```

✅Remember: NEVER use arrow function as an object method if you need 'this' to be the object. Always use regular function for object methods.

Arrow Inside Regular Method — CORRECT Use (This is Where it Shines!)

```
const team = {
  name:    "Dev Team",
  members: ["Shaaz", "Aman", "Riya"],
  showMembers: function() {
    // "this" = team here ✓
    this.members.forEach(member => {   // arrow — inherits this from showMembers
      console.log(this.name + ": " + member); // this = team ✓
    });
  }
};
team.showMembers();
// Output: Dev Team: Shaaz
//         Dev Team: Aman
//         Dev Team: Riya
```

💡 Tip: Arrow functions were invented to solve 'this is lost inside callbacks'. Use regular function for methods. Use arrow function for callbacks inside methods.

setTimeout with Arrow vs Regular

```
const timer = {
  name: "My Timer",
  start: function() {
    setTimeout(function() {
      console.log(this.name); // undefined ✗ — regular fn, this = window
    }, 1000);
    setTimeout(() => {
      console.log(this.name); // "My Timer" ✓ — arrow, inherits from start()
    }, 1000);
  }
};
timer.start();
```

Topic 6: Context 5 — 'this' with new Keyword (Constructor)

When you call a function with 'new', JavaScript creates a brand new empty object and 'this' inside the constructor refers to that brand new object.

Step 1 — Creates a new empty object {}

Step 2 — Sets 'this' to that new object

Step 3 — Runs the function (attaches properties to 'this')

Step 4 — Returns 'this' automatically

```
function Person(name, age) {
  this.name = name;   // attaches to new object
  this.age  = age;
  this.greet = function() {
    console.log(`Hi, I am ${this.name}`);
  };
}
let p1 = new Person("Shaaz", 23);
let p2 = new Person("Aman",  25);
p1.greet(); // Output: Hi, I am Shaaz  (this = p1)
p2.greet(); // Output: Hi, I am Aman   (this = p2)
```

Topic 7: Context 6 — 'this' in ES6 Classes

Classes are just cleaner syntax for constructor functions. 'this' inside a class method refers to the specific instance that called the method.

```
class Car {
  constructor(brand, speed) {
    this.brand = brand;
    this.speed = speed;
  }
  describe() {
    console.log(`${this.brand} goes ${this.speed} km/h`);
  }
}
let car1 = new Car("Toyota", 100);
let car2 = new Car("BMW",    150);
car1.describe(); // Output: Toyota goes 100 km/h  (this = car1)
car2.describe(); // Output: BMW goes 150 km/h     (this = car2)
```

Topic 8: Context 7 — 'this' in Event Listeners

Regular function in event listener — 'this' = the HTML element the event is attached to.

Arrow function in event listener — 'this' = window (inherits from outer scope, NOT the element).

```
let btn = document.getElementById("myBtn");

btn.addEventListener("click", function() {
  console.log(this);        // Output: <button>  ✓
  console.log(this.id);     // Output: myBtn     ✓
  this.style.background = "red"; // works ✓
});

btn.addEventListener("click", () => {
  console.log(this);        // Output: Window    ✗
  console.log(this.id);     // Output: undefined ✗
});
```

✅Remember: Regular function in event listener = this is the element. Arrow function = this is window. Use regular function when you need to work with the element using 'this'.

Topic 9: The Complete 'this' Summary — All Contexts

```
Global scope                  →  window / global
Object method (dot notation)  →  the object left of the dot
```

```
Regular function alone        →  window (non-strict) / undefined (strict)
Arrow function                →  inherited from outer scope — PERMANENT
new keyword / Constructor     →  brand new object being created
Class method                  →  the instance that called the method
Event listener (regular fn)   →  the HTML element
Event listener (arrow fn)     →  window (outer scope)
call() / apply()              →  explicitly set first argument
bind()                        →  permanently set object — cannot change
```

Topic 10: Tricky 'this' Scenarios — Interview Traps!

Trap 1 — Nested Regular Function Loses 'this'

```
const obj = {
  name: "Shaaz",
  outer: function() {
    console.log(this.name); // Shaaz ✓
    function inner() {
      console.log(this.name); // undefined ✗  (inner called alone)
    }
    inner();
  }
};
obj.outer();
// Fix: use arrow function for inner
const obj2 = {
  name: "Shaaz",
  outer: function() {
    const inner = () => console.log(this.name); // inherits ✓
    inner();
  }
};
obj2.outer(); // Output: Shaaz ✓
```

Trap 2 — Method as Callback Loses 'this'

```
const user = {
  name: "Shaaz",
  greet() { console.log("Hello " + this.name); }
};
setTimeout(user.greet, 1000);             // Hello undefined ✗
setTimeout(user.greet.bind(user), 1000);  // Hello Shaaz ✓
setTimeout(() => user.greet(), 1000);     // Hello Shaaz ✓
```

Trap 3 — Arrow Method Cannot Be Fixed with bind()

```
const obj = {
  name: "Shaaz",
  greet: () => console.log(this.name) // this = window permanently
};
obj.greet();           // undefined ✗
obj.greet.call(obj);   // undefined ✗ — call cannot override arrow this
obj.greet.bind(obj)(); // undefined ✗ — bind cannot override either
```

✅Remember: Arrow function 'this' is PERMANENT — set at write time, cannot be changed by call/apply/bind. Regular function 'this' is DYNAMIC — set at call time, can be changed.

Trap 4 — setInterval 'this' Bug and Fix

```
function Counter() {
  this.count = 0;
  setInterval(function() {
    this.count++;              // this = window — NaN, NaN, NaN ✗
    console.log(this.count);
  }, 1000);
}
new Counter();

// Fix:
function Counter() {
  this.count = 0;
  setInterval(() => {          // arrow — inherits this from Counter
    this.count++;
    console.log(this.count);   // 1, 2, 3, 4... ✓
  }, 1000);
}
new Counter();
```

Topic 11: 'this' Keyword Interview Questions with Solutions

Q1: What is 'this' in JavaScript?

'this' is a keyword that refers to the object currently executing the function. Its value depends on HOW the function is called — not where it is written. Object method = the object. Alone = window or undefined. Arrow = outer scope. new = new object.

Q2: What is the difference between 'this' in regular vs arrow function?

Regular function — 'this' is dynamic, determined at call time. Can be changed with call/apply/bind.

Arrow function — 'this' is lexical (static), determined at write time, inherited from outer scope. Cannot be changed by call/apply/bind.

Q3: What will this output?

```
const obj = { x: 10, getX: function() { return this.x; } };
let fn = obj.getX;
console.log(fn()); // Output: undefined
```

Method extracted → called alone → this = window → window.x = undefined.

Q4: How do you fix 'this' inside a setTimeout?

```
const obj = {
  name: "Shaaz",
  start: function() {
    setTimeout(() => console.log(this.name), 1000);   // arrow ✓
    setTimeout(function() {                            // bind ✓
      console.log(this.name);
    }.bind(this), 1000);
  }
};
obj.start(); // Both output: Shaaz
```

Q5: Can you change 'this' in an arrow function using call()?

No. Arrow functions permanently inherit 'this' from their outer scope when they are written. call(), apply(), and bind() are ignored when used on arrow functions.

Q6: What does 'this' refer to in a class method?

'this' refers to the specific instance that called the method. car1.describe() — this = car1. car2.describe() — this = car2. Each instance gets its own 'this'.

Q7: Why is 'this' undefined inside a regular function in strict mode?

In non-strict mode, JavaScript automatically assigns the global object as 'this' when a function is called without an owner. In strict mode, this default behaviour is disabled to prevent accidental global pollution — 'this' stays undefined.

Q8: What is the output of this code?

```
const obj = {
  name: "Outer",
  inner: {
    name: "Inner",
    show: function() { console.log(this.name); }
  }
};
obj.inner.show(); // Output: ?
```

Answer: Inner

Why? The Golden Rule — 'this' = object left of the dot. obj.inner.show() → the object immediately left of show() is inner, not obj. So this.name = inner.name = 'Inner'.

Topic 12: Quick Reference — 'this' in Every Context

Global scope — window (browser) / global (Node.js)

Object method — the object that called the method (left of dot)

Regular function alone — window (non-strict) / undefined (strict)

Arrow function — inherited from outer scope — CANNOT be changed

new keyword — the brand new object being created

Class method — the instance that called the method

Event listener regular fn — the HTML element the event is attached to

Event listener arrow fn — window (inherited from outer scope)

call() / apply() — the first argument you explicitly pass

bind() — permanently set — cannot be overridden

> **✅Remember: The one sentence: 'this' is the object LEFT of the dot when the function is called. No dot = window or undefined. Arrow = outer scope always. new = brand new object.**

Chapter 8: Closures

How to Define in an Interview

"A Closure is a function that remembers the variables from its outer (parent) scope even after that outer function has finished executing and returned. In JavaScript, every time a function is created, it forms a closure — it carries a reference to its surrounding scope (the lexical environment where it was written). Closures are used to create private variables, maintain state between function calls, create function factories, and implement the module pattern."

Example you can say in interview:

```
function outer() {
  let count = 0;              // outer variable
  return function inner() {
    count++;                  // inner remembers count even after outer finishes
    console.log(count);
  };
}

let increment = outer();     // outer() has finished and returned
increment(); // Output: 1    // but count is still remembered!
increment(); // Output: 2
increment(); // Output: 3
```

Topic 1: The Story That Will Make You Never Forget Closures

Imagine a mother prepares a lunchbox for her son every morning. She puts food inside and CLOSES the box. The son takes the lunchbox to school. The mother is no longer there — she went to work. But the son can still OPEN the box and eat the food inside.

In JavaScript:

Mother = the outer function (she prepared the box and is now gone)

Lunchbox = the closed-over variables (count, name, etc.)

Son = the inner function (closure — still has access to what was inside)

Going to school = the outer function has already returned and finished

Opening and eating = calling the inner function and using those variables

The box stays accessible even though the mother (outer function) is gone. That is exactly what a closure is.

> **✅Remember: Closure = inner function + the variables it remembers from its outer function, even after that outer function is done.**

Topic 2: What Actually Happens in Memory

Normally, when a function finishes running, JavaScript cleans up its variables from memory — they are garbage collected.

BUT — if an inner function still has a reference to those variables (through a closure), JavaScript says 'I cannot delete these yet, someone still needs them.' The variables stay alive in memory as long as the closure exists.

```
function makeGreeting() {
  let message = "Hello, Shaaz!";   // normally deleted when makeGreeting ends

  return function() {              // this inner function holds a reference
```

```
    console.log(message);          // to message — so message stays alive
  };
}

let greet = makeGreeting();        // makeGreeting is done and returned
// message would normally be gone — BUT the closure keeps it alive

greet(); // Output: Hello, Shaaz!  // message is still there!
```

Note: The closure does not copy the value. It holds a LIVE REFERENCE to the variable. If the variable changes, the closure sees the updated value.

Topic 3: How to Spot a Closure in Code — 3 Signs

You have a closure when you see ALL THREE of these:

Sign 1 — A function is defined INSIDE another function

Sign 2 — The inner function uses variables from the outer function

Sign 3 — The inner function is returned or used somewhere outside

```
function outer() {                 // Sign 1: outer function
  let name = "Shaaz";              // Sign 2: variable used by inner

  function inner() {
    console.log("Hello " + name); // Sign 2: inner uses outer variable
  }

  return inner;                   // Sign 3: inner is returned outside
}

let sayHello = outer();
sayHello(); // Output: Hello Shaaz  — closure in action!
```

Topic 4: Private Variables — The Most Important Use of Closures

JavaScript does not have a built-in 'private' keyword like Java or C++. But closures let you create TRULY PRIVATE variables — variables that cannot be touched or seen from outside.

Real world use: a bank account. The balance should be private — nobody should be able to directly change it. You should only be able to deposit or withdraw through proper methods.

```
function createBankAccount(initialBalance) {
  let balance = initialBalance;  // PRIVATE — nobody outside can touch this

  return {
    deposit(amount) {
      if (amount > 0) {
        balance += amount;
        console.log(`Deposited ${amount}. Balance: ${balance}`);
      }
    },
    withdraw(amount) {
      if (amount > balance) {
        console.log("Not enough balance!");
      } else {
        balance -= amount;
        console.log(`Withdrew ${amount}. Balance: ${balance}`);
```

```
      }
    },
    getBalance() {
      return balance;             // can only READ balance — not change directly
    }
  };
}

let myAccount = createBankAccount(1000);
myAccount.deposit(500);         // Output: Deposited 500. Balance: 1500
myAccount.withdraw(200);        // Output: Withdrew 200. Balance: 1300
console.log(myAccount.getBalance()); // Output: 1300

// Cannot access balance directly — it is PRIVATE:
console.log(myAccount.balance); // Output: undefined  (protected!)
myAccount.balance = 999999;     // Does NOTHING — balance is not changed
console.log(myAccount.getBalance()); // Output: 1300  (still safe!)
```

✅Remember: Closures are the ONLY way to create truly private variables in JavaScript (without using ES2022 # syntax). This is one of the most powerful features of closures.

Topic 5: Maintaining State Between Calls

Normally, every time you call a function, its local variables start fresh. But with closures, the inner function remembers the previous state — like a memory that persists between calls.

Counter — The Classic Closure Example

```
function createCounter() {
  let count = 0;   // this persists between calls — not reset each time

  return {
    increment() { count++; console.log(count); },
    decrement() { count--; console.log(count); },
    reset()     { count = 0; console.log("Reset!"); },
    getCount()  { return count; }
  };
}

let counter = createCounter();
counter.increment(); // Output: 1
counter.increment(); // Output: 2
counter.increment(); // Output: 3
counter.decrement(); // Output: 2
counter.reset();     // Output: Reset!
counter.increment(); // Output: 1
```

Each call to counter.increment() remembers and updates the SAME count variable. That count is private — nothing outside can reset or corrupt it.

Each Closure Gets Its Own Private Copy

If you create two counters, they do NOT share state. Each one has its own separate closure with its own count variable.

```
let counterA = createCounter();
let counterB = createCounter();
```

```
counterA.increment(); // Output: 1
counterA.increment(); // Output: 2
counterA.increment(); // Output: 3

counterB.increment(); // Output: 1  (starts from 0 — completely separate!)

console.log(counterA.getCount()); // Output: 3
console.log(counterB.getCount()); // Output: 1
```

Tip: This is very important — each call to createCounter() creates a brand new closure with its own separate count. They are completely independent.

Topic 6: Function Factories — Create Customised Functions

A function factory is a function that creates and returns other customised functions. Each returned function remembers the arguments it was created with — through closure.

Multiplier Factory

```
function multiplier(factor) {
  return function(number) {
    return number * factor;   // remembers factor through closure
  };
}

let double  = multiplier(2);   // factor = 2, remembered forever
let triple  = multiplier(3);   // factor = 3, remembered forever
let tenTimes = multiplier(10); // factor = 10

console.log(double(5));    // Output: 10
console.log(triple(5));    // Output: 15
console.log(tenTimes(5));  // Output: 50
```

Greeting Factory

```
function createGreeting(greeting) {
  return function(name) {
    return `${greeting}, ${name}!`;
  };
}

let sayHello  = createGreeting("Hello");
let sayNamaste = createGreeting("Namaste");
let sayHey    = createGreeting("Hey");

console.log(sayHello("Shaaz"));   // Output: Hello, Shaaz!
console.log(sayNamaste("Aman"));  // Output: Namaste, Aman!
console.log(sayHey("Riya"));      // Output: Hey, Riya!
```

Discount Factory — Real World Example

```
function discountMaker(discountPercent) {
  return function(originalPrice) {
    let discount = (originalPrice * discountPercent) / 100;
    return originalPrice - discount;
  };
}
```

```
let apply10 = discountMaker(10);
let apply25 = discountMaker(25);
let apply50 = discountMaker(50);

console.log(apply10(1000));  // Output: 900
console.log(apply25(1000));  // Output: 750
console.log(apply50(1000));  // Output: 500
```

Topic 7: Closure in Loops — The Most Famous Interview Trap!

This is one of the most asked closure questions in interviews. Pay very careful attention.

The Bug — var in a Loop with Closure

```
for (var i = 1; i <= 3; i++) {
  setTimeout(function() {
    console.log("Value of i:", i);
  }, i * 1000);
}

// You expect: 1, 2, 3 (after 1s, 2s, 3s)
// Actual output: 4, 4, 4
```

Why? Because var is function-scoped — there is only ONE i shared by all three closures. By the time the setTimeout callbacks run, the loop has finished and i is already 4. All three closures see the same final value: 4.

Fix 1 — Use let (Creates New i Each Iteration)

```
for (let i = 1; i <= 3; i++) {
  setTimeout(function() {
    console.log("Value of i:", i);
  }, i * 1000);
}
// Output: 1 (after 1s), 2 (after 2s), 3 (after 3s)  CORRECT!
```

let is block-scoped, so each iteration creates its own separate i. Each closure captures its own separate i.

Fix 2 — Use IIFE (The Old Way Before let Existed)

```
for (var i = 1; i <= 3; i++) {
  (function(j) {           // IIFE captures i as a NEW variable j
    setTimeout(function() {
      console.log("Value:", j);
    }, j * 1000);
  })(i);                   // pass current i into IIFE immediately
}
// Output: 1, 2, 3  CORRECT!
```

The IIFE creates a new scope each iteration with its own copy of the value.

> **✅Remember: This is THE most asked closure interview question. var loop gives wrong values because all closures share one variable. let fixes it by creating a new variable per iteration. Always use let in loops.**

Topic 8: Memoization — Caching Results with Closures

Memoization means remembering the result of a function call so you do not have to calculate it again if the same input is given. Closures are perfect for this — the cache object stays alive in memory between calls.

```
function memoize(fn) {
  let cache = {};   // this cache persists between calls — closure!

  return function(n) {
    if (cache[n] !== undefined) {
      console.log("From cache:", n);
      return cache[n];          // return stored result — no recalculation
    }
    console.log("Calculating:", n);
    cache[n] = fn(n);           // calculate and store
    return cache[n];
  };
}

function square(n) { return n * n; }

let fastSquare = memoize(square);

fastSquare(5);   // Output: Calculating: 5  → returns 25
fastSquare(5);   // Output: From cache: 5   → returns 25  (instant!)
fastSquare(6);   // Output: Calculating: 6  → returns 36
fastSquare(6);   // Output: From cache: 6   → returns 36  (instant!)
```

Tip: Memoization using closures is a popular interview question. The cache object is private and persists — it's a perfect real-world use of closures.

Topic 9: The Module Pattern — Closures in Real Projects

Before ES6 modules existed, developers used the Module Pattern to create private state and expose only what they wanted. It is built entirely on closures.

```
const ShoppingCart = (function() {
  let items = [];         // private — nobody outside can see this
  let total = 0;          // private

  return {
    addItem(name, price) {
      items.push({ name, price });
      total += price;
      console.log(`${name} added. Total: ${total}`);
    },
    removeItem(name) {
      let item = items.find(i => i.name === name);
      if (item) {
        items = items.filter(i => i.name !== name);
        total -= item.price;
        console.log(`${name} removed. Total: ${total}`);
      }
    },
    getTotal()  { return total; },
    getItems()  { return [...items]; }
  };
```

```
})();   // IIFE — runs immediately and returns the object

ShoppingCart.addItem("Laptop", 50000);   // Laptop added. Total: 50000
ShoppingCart.addItem("Mouse", 1500);     // Mouse added. Total: 51500
ShoppingCart.removeItem("Mouse");        // Mouse removed. Total: 50000
console.log(ShoppingCart.getTotal());    // Output: 50000

// Private variables are truly protected:
console.log(ShoppingCart.items); // Output: undefined  (private!)
console.log(ShoppingCart.total); // Output: undefined  (private!)
```

Topic 10: Closure Interview Questions with Solutions

Q1: What is a closure? Explain with an example.

A closure is an inner function that remembers the variables of its outer function even after the outer function has finished running. Every function in JavaScript forms a closure over the variables in its surrounding scope.

```
function outer() {
  let secret = 42;
  return function inner() {
    console.log(secret); // remembers secret after outer is done
  };
}
let fn = outer();
fn(); // Output: 42
```

Q2: What will this print?

```
function makeAdder(x) {
  return function(y) {
    return x + y;
  };
}

let add5  = makeAdder(5);
let add10 = makeAdder(10);

console.log(add5(3));   // ?
console.log(add10(3));  // ?
```

Answer: add5(3) = 8, add10(3) = 13

add5 remembers x=5 through closure. add10 remembers x=10. They are separate closures with separate x values.

Q3: What is the output of this code?

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Answer: 3, 3, 3

var is function-scoped — one shared i. By the time callbacks run, i is already 3. Fix: use let.

Q4: How do closures help create private variables?

Variables declared inside a function are not accessible outside. If you return an inner function that accesses those variables, the outer variable becomes private — readable and modifiable only through the returned function.

```
function createPassword() {
  let password = "abc123";    // private — nobody outside can see
  return {
    check: (p) => p === password ? "Correct!" : "Wrong!",
    change: (newP) => { password = newP; }
  };
}
let auth = createPassword();
console.log(auth.check("abc123")); // Output: Correct!
console.log(auth.password);        // Output: undefined  (private!)
```

Q5: What is the difference between closure and scope?

Scope is the rule that defines where a variable is accessible. Closure is a feature where an inner function retains access to its outer function's variables even after that outer function has returned. Scope is the concept — closure is the practical result of lexical scoping combined with functions being first-class citizens.

Q6: Do all functions in JavaScript form closures?

Yes — technically every function in JavaScript forms a closure over its surrounding scope. But in practice, we talk about closures specifically when an inner function accesses and retains outer function variables after the outer function has returned.

Q7: What is a memory leak with closures and how to avoid it?

If a closure holds a reference to a large object or DOM element that is no longer needed, that memory cannot be garbage collected — causing a memory leak. To avoid it, set the reference to null when done.

```
function setup() {
  let bigData = new Array(1000000).fill("data"); // large array
  return function() {
    console.log(bigData.length); // closure keeps bigData alive
  };
}
let fn = setup();
fn();          // uses bigData
fn = null;     // release the closure — bigData can now be garbage collected
```

Q8: Write a function that can only be called once

```
function once(fn) {
  let called = false;   // private state — closure!
  let result;
  return function(...args) {
    if (!called) {
      called = true;
      result = fn(...args);
    }
    return result;
  };
}

let initialize = once(() => console.log("Initialized!"));
```

```
initialize(); // Output: Initialized!
initialize(); // Output: nothing — called only once
initialize(); // Output: nothing
```

Q9: What is the output?

```
function outer() {
  let x = 10;
  function inner() {
    console.log(x);
  }
  x = 20;             // x is changed AFTER inner is defined
  return inner;
}
outer()(); // What prints?
```

Answer: 20

Because the closure holds a LIVE REFERENCE to x — not a copy of its value at the time inner was defined. When x changed to 20, the closure sees the updated value.

> **✅Remember: Closures capture REFERENCES — not values. If the outer variable changes after the inner function is defined, the closure sees the new value. This is a common source of bugs AND interview questions.**

Q10: Build a memoized function using closure

```
function memoize(fn) {
  let cache = {};
  return function(...args) {
    let key = JSON.stringify(args);
    if (key in cache) return cache[key];
    cache[key] = fn(...args);
    return cache[key];
  };
}

let expensiveCalc = memoize((n) => {
  console.log("Computing...");
  return n * n;
});

expensiveCalc(5);  // Output: Computing... → 25
expensiveCalc(5);  // No computing → 25  (from cache)
expensiveCalc(6);  // Output: Computing... → 36
```

Topic 11: Quick Reference — Closures at a Glance

Closure — inner function that remembers outer function's variables after outer returns

Private variable — outer variable only accessible through returned inner function

State — closure keeps state alive between calls (like count in a counter)

Function factory — outer function returns customised inner functions

Module pattern — IIFE + closure = private state + public API

Memoization — cache results using closure to remember previous outputs

Loop + var bug — var closures share one variable, use let to fix

Live reference — closure captures reference not value, sees latest changes

Memory leak — closure keeping large unused data alive, set to null when done

once() — function that runs only once using closure flag

> ✅**Remember: The ONE sentence to remember: A closure is a function that remembers its birth environment — the variables around it when it was created — and carries them wherever it goes, even after its parent is gone.**

Chapter 9: Call, Apply, and Bind

How to Define in an Interview

"call(), apply(), and bind() are three methods available on every JavaScript function that allow you to explicitly control what 'this' refers to inside that function. call() invokes the function immediately and passes arguments one by one. apply() also invokes immediately but passes arguments as an array. bind() does NOT invoke immediately — it returns a brand new function with 'this' permanently set, which you can call later. All three are used to borrow methods from one object and use them for another object."

One line to remember:

call = call now (comma separated) | apply = call now (array) | bind = call later (returns new function)

Topic 1: The Problem — Why Do call, apply, bind Exist?

First understand the problem. Then the solution will make perfect sense.

Imagine you have a DOCTOR object with a method called introduce(). Now you also have a PATIENT object that needs the same introduction logic — but with its own data. You have two choices:

Option A — Copy the same function into the patient object. (Bad — duplicate code)

Option B — BORROW the doctor's function and run it as if it belongs to the patient. (Good — this is what call/apply/bind do)

```
const doctor = {
  name:     "Dr. Sharma",
  speciality: "Cardiology",
  introduce: function(hospital, city) {
    console.log(`I am ${this.name}, ${this.speciality} at ${hospital}, ${city}`);
  }
};

const patient = {
  name:     "Shaaz",
  speciality: "Patient"
  // introduce method does NOT exist here
};

// We want to run doctor.introduce() but with patient.name and patient.speciality
// This is exactly what call, apply, bind solve!
```

Tip: The key idea: all three methods let you BORROW a function and change what 'this' means inside it. Instead of this = doctor, you make this = patient.

Topic 2: Quick Understanding of 'this' (Very Important Before Moving On)

Inside any function, 'this' means the object that is calling the function. By default:

```
const doctor = {
  name: "Dr. Sharma",
  introduce: function() {
    console.log(this.name); // "this" = doctor — because doctor is calling it
  }
};
doctor.introduce(); // Output: Dr. Sharma
```

But what if we want 'this' to mean a DIFFERENT object? That is where call, apply, bind come in. They let you manually say: 'Hey function, when you run, treat THIS object as your this.'

Topic 3: call() — Borrow and Run Immediately (Arguments One by One)

Think of call() like this: you are CALLING someone and telling them — 'Hey, do this job but pretend you are this other person.'

Syntax: functionName.call(thisObject, arg1, arg2, arg3...)

thisObject = the object you want 'this' to refer to

arg1, arg2... = arguments you pass to the function, one by one

Basic Example

```
const person1 = {
  name: "Shaaz",
  age:  23,
  introduce: function(city, country) {
    console.log(`I am ${this.name}, ${this.age}, from ${city}, ${country}`);
  }
};

const person2 = { name: "Aman", age: 25 };
// person2 has NO introduce method — we borrow it from person1

person1.introduce.call(person2, "Mumbai", "India");
// Output: I am Aman, 25, from Mumbai, India
//                 ^^^^ this.name = person2.name
//                          ^^^ this.age = person2.age
```

What happened? call() ran introduce() immediately, but forced 'this' to be person2. So this.name became 'Aman' and this.age became 25.

call() with a Standalone Function

```
function greet(greeting, punctuation) {
  console.log(`${greeting}, ${this.name}${punctuation}`);
}

const user1 = { name: "Shaaz" };
const user2 = { name: "Riya" };

greet.call(user1, "Hello", "!");   // Output: Hello, Shaaz!
greet.call(user2, "Namaste", ".");  // Output: Namaste, Riya.
greet.call(user1, "Hi", "...");    // Output: Hi, Shaaz...
```

✅Remember: call() = runs IMMEDIATELY. Arguments come after the this object, separated by commas. The first argument is ALWAYS the object you want as 'this'.

call() with null — When You Don't Need 'this'

```
function add(a, b) {
  return a + b;   // no "this" used inside
}

console.log(add.call(null, 5, 3)); // Output: 8
```

When your function does not use 'this' at all, pass null as the first argument. It just means 'no object needed for this'.

Topic 4: apply() — Same as call() but Arguments in an Array

apply() works EXACTLY like call(). The ONLY difference is how you pass arguments:

call() — arguments one by one: fn.call(obj, arg1, arg2, arg3)

apply() — arguments as an array: fn.apply(obj, [arg1, arg2, arg3])

Think of the 'a' in apply as 'array' — easy to remember!

Same Example with apply()

```
const person1 = {
  name: "Shaaz",
  introduce: function(city, country) {
    console.log(`I am ${this.name} from ${city}, ${country}`);
  }
};

const person2 = { name: "Aman" };

// call() — args one by one:
person1.introduce.call(person2, "Mumbai", "India");

// apply() — args as array:
person1.introduce.apply(person2, ["Mumbai", "India"]);

// Both give same Output: I am Aman from Mumbai, India
```

Real World Use of apply() — Math.max with an Array

apply() was very useful before the spread operator existed. A classic example is finding the max number in an array:

```
let numbers = [3, 67, 12, 99, 45, 8];

// Math.max does NOT accept an array directly:
console.log(Math.max(numbers));          // Output: NaN  (WRONG!)

// Old way — use apply to spread array as separate arguments:
console.log(Math.max.apply(null, numbers)); // Output: 99  (CORRECT!)

// Modern way — use spread operator:
console.log(Math.max(...numbers));          // Output: 99  (same result)
```

Note: In modern JavaScript, apply() for this purpose has been mostly replaced by the spread operator (...). But interviewers still ask about it because it shows you understand the concept.

✅Remember: apply() = runs IMMEDIATELY. Arguments come as an ARRAY. 'a' in apply = array. Everything else is identical to call().

Topic 5: bind() — Prepare Now, Run Later

bind() is different from call() and apply(). It does NOT run the function immediately. Instead, it creates and returns a BRAND NEW FUNCTION with 'this' permanently locked to the object you specified. You can call this new function whenever you want.

Think of bind() like preparing a pre-filled form. You fill in the object (who 'this' is) and maybe some arguments in advance. You get a ready-to-go function. You call it whenever needed.

Syntax: let newFn = functionName.bind(thisObject, arg1, arg2...)

Basic Example

```
const person1 = {
  name: "Shaaz",
  introduce: function(city, country) {
    console.log(`I am ${this.name} from ${city}, ${country}`);
  }
};

const person2 = { name: "Aman" };

// bind returns a NEW function — does NOT run yet:
let amanIntroduce = person1.introduce.bind(person2, "Mumbai", "India");

// Call it later whenever you want:
amanIntroduce(); // Output: I am Aman from Mumbai, India
amanIntroduce(); // Output: I am Aman from Mumbai, India  (can call multiple times)
```

bind() in Event Listeners — Most Common Real-World Use

The most common real use of bind() is fixing 'this' inside event listeners. When you click a button, 'this' inside the callback becomes the button element — not your object. bind() fixes this.

```
const timer = {
  message: "Time is up!",
  start: function() {
    // Without bind — "this" inside setTimeout is window/undefined
    // With bind — "this" is locked to the timer object
    setTimeout(function() {
      console.log(this.message); // undefined without bind!
    }.bind(this), 2000);         // bind(this) locks "this" = timer
  }
};

timer.start(); // Output after 2 sec: Time is up!
```

Partial Application with bind() — Pre-fill Some Arguments

You can use bind() to pre-fill some arguments of a function. The remaining arguments are passed when you actually call the bound function. This is called Partial Application.

```
function multiply(a, b) {
  return a * b;
}

// Pre-fill "a" = 2, leave "b" open:
let double  = multiply.bind(null, 2);  // a is permanently 2
let triple  = multiply.bind(null, 3);  // a is permanently 3
let tenTimes = multiply.bind(null, 10);

console.log(double(5));    // Output: 10  (2 × 5)
console.log(double(8));    // Output: 16  (2 × 8)
console.log(triple(5));    // Output: 15  (3 × 5)
```

```
console.log(tenTimes(7));  // Output: 70  (10 × 7)
```

Tip: Partial application with bind() lets you create specialised versions of a general function. This is a very elegant pattern.

✅Remember: bind() = does NOT run immediately. Returns a NEW function with 'this' permanently locked. Call it whenever you want, as many times as you want.

Topic 6: call vs apply vs bind — Side by Side

Runs immediately?

call() — YES, runs immediately
apply() — YES, runs immediately
bind() — NO, returns a new function to call later

How to pass arguments?

call() — one by one: fn.call(obj, arg1, arg2)
apply() — as array: fn.apply(obj, [arg1, arg2])
bind() — one by one: fn.bind(obj, arg1, arg2) then call later

What does it return?

call() — returns whatever the function returns
apply() — returns whatever the function returns
bind() — returns a brand new function

When to use?

call() — borrow a method and run it right now
apply() — same, but your arguments are already in an array
bind() — fix 'this' for event listeners, callbacks, setTimeout

```
const obj = { name: "Shaaz" };

function greet(greeting, mark) {
  console.log(`${greeting} ${this.name}${mark}`);
}

greet.call(obj,  "Hello",   "!");  // runs now — Output: Hello Shaaz!
greet.apply(obj, ["Hello",  "!"]);  // runs now — Output: Hello Shaaz!

let fn = greet.bind(obj, "Hello", "!"); // NOT running yet
fn();  // runs now — Output: Hello Shaaz!
fn();  // can run again — Output: Hello Shaaz!
```

Topic 7: Real World Examples — When You Actually Use These

Method Borrowing — Using Array methods on Array-Like Objects

Some objects look like arrays (they have length and numbered keys) but are not real arrays — so array methods like slice, forEach etc. do not work on them directly. We can borrow these methods using call().

```
function showArgs() {
  // "arguments" is array-like but NOT a real array
  // forEach does not work directly on it

  // Borrow forEach from Array:
  Array.prototype.forEach.call(arguments, function(arg) {
    console.log(arg);
  });
}

showArgs("apple", "banana", "mango");
// Output: apple  banana  mango
```

bind() for Fixing 'this' in Class Methods

```
class Button {
  constructor(label) {
    this.label = label;
    this.handleClick = this.handleClick.bind(this); // fix this permanently
  }

  handleClick() {
    console.log(`${this.label} was clicked!`);
  }
}

let btn = new Button("Submit");
document.querySelector("button").addEventListener("click", btn.handleClick);
// Without bind: this.label would be undefined
// With bind: Output: Submit was clicked!
```

apply() for Combining Arrays

```
let fruits = ["apple", "banana"];
let veggies = ["carrot", "potato"];

// Push all veggies into fruits using apply:
Array.prototype.push.apply(fruits, veggies);
console.log(fruits); // Output: ["apple", "banana", "carrot", "potato"]

// Modern equivalent using spread:
fruits.push(...veggies);
```

Topic 8: Call, Apply, Bind Interview Questions with Solutions

Q1: What is the difference between call, apply, and bind?

call — invokes function immediately, arguments passed one by one.

apply — invokes function immediately, arguments passed as an array.

bind — does NOT invoke immediately, returns a new function with 'this' permanently set.

```
fn.call(obj, a, b)       // runs now, args separate
fn.apply(obj, [a, b])    // runs now, args as array
let newFn = fn.bind(obj, a, b); newFn(); // runs later
```

Q2: What will this output?

```
const obj1 = { name: "Shaaz" };
const obj2 = { name: "Aman"  };

function sayName() {
  console.log(this.name);
}

sayName.call(obj1);        // Output: ?
sayName.apply(obj2);       // Output: ?
let fn = sayName.bind(obj1);
fn();                      // Output: ?
```

Answers: Shaaz, Aman, Shaaz

Q3: Can you use bind() to create a partial function?

Yes — bind() can pre-fill some arguments. The remaining arguments are provided when calling the bound function.

```
function power(base, exponent) {
  return Math.pow(base, exponent);
}

let square = power.bind(null, 2);  // base is pre-filled as 2
let cube   = power.bind(null, 3);  // base is pre-filled as 3

console.log(square(4)); // Output: 16  (2^4)
console.log(cube(3));   // Output: 27  (3^3)
```

Q4: What is the output?

```
const person = {
  name: "Shaaz",
  greet: function() {
    console.log("Hello " + this.name);
  }
};

const greetFn = person.greet;
greetFn();                    // Output: ?
greetFn.call(person);         // Output: ?
```

Answer:

greetFn() — Output: Hello undefined (or Hello, depending on strict mode). 'this' is lost because greetFn is a regular function call, not a method call.

greetFn.call(person) — Output: Hello Shaaz. call() explicitly sets 'this' to person.

> **✅Remember: When you extract a method from an object and call it as a standalone function, 'this' is lost. call(), apply(), or bind() are how you restore it.**

Q5: What is the difference between call() and a normal function call?

A normal function call: fn() — 'this' is determined by how the function is called (could be window, undefined, or the calling object).

call(): fn.call(obj) — 'this' is EXPLICITLY set to obj. You have full control.

Q6: Why would you use bind() instead of call()?

Use bind() when you need to pass the function somewhere else to be called LATER — like an event listener, a setTimeout callback, or a function argument. call() runs it immediately which is not always what you want.

```
// Need function for later — use bind:
button.addEventListener("click", obj.method.bind(obj));

// Need to run now — use call:
obj.method.call(anotherObj);
```

Q7: Can bind() be overridden by another call()?

```
function greet() {
  console.log(this.name);
}

const obj1 = { name: "Shaaz" };
const obj2 = { name: "Aman" };

let boundFn = greet.bind(obj1);
boundFn.call(obj2); // Output: Shaaz  (NOT Aman!)
```

No — once a function is bound with bind(), the 'this' is permanently locked. Even if you call call() or apply() on a bound function, the 'this' cannot be changed.

> **✅Remember: A bound function's 'this' is permanent — it cannot be overridden by call() or apply(). This is asked in many senior interviews.**

Q8: How would you implement your own bind() from scratch?

This is a classic interview question. Here is a simple custom bind implementation:

```
Function.prototype.myBind = function(context, ...args) {
  let fn = this;   // "this" here is the function bind is called on
  return function(...moreArgs) {
    return fn.call(context, ...args, ...moreArgs);
  };
};

function greet(greeting, mark) {
  console.log(`${greeting} ${this.name}${mark}`);
}

const obj = { name: "Shaaz" };
let fn = greet.myBind(obj, "Hello");
fn("!");    // Output: Hello Shaaz!
fn("...");  // Output: Hello Shaaz...
```

> **□ Tip: Writing your own bind() is one of the most impressive things you can do in a JavaScript interview. Practice this!**

Topic 9: Quick Reference — call, apply, bind at a Glance

fn.call(obj, a, b) — run now, this=obj, args one by one

fn.apply(obj, [a, b]) — run now, this=obj, args as array

fn.bind(obj, a, b) — NOT now, returns new function, this=obj permanently

fn.call(null, a, b) — run now, no this needed, just use for arguments

bound function — this cannot be changed by call/apply after binding

Partial application — bind with pre-filled args, remaining passed later

Method borrowing — use call/apply to use one object's method for another

✅Remember: The ONE line to always remember: call and apply run NOW. bind runs LATER. call uses commas, apply uses an Array. bind returns a New function. C-A-N: Call-Apply-Now, Bind-New-Later.

Chapter 10: Understanding the DOM

How to Define in an Interview

"The DOM (Document Object Model) is a programming interface provided by the browser that represents an HTML document as a tree of nodes. Every HTML element becomes a node in this tree — the document is the root, html is its child, and so on. JavaScript can use the DOM to select, create, modify, or delete HTML elements and their attributes dynamically without reloading the page. The DOM is not part of JavaScript itself — it is a Web API provided by the browser."

Example you can say in interview:

```
// Select an element and change its text:
let heading = document.getElementById("title");
heading.innerText = "Hello DOM!";

// Create a new element and add it to the page:
let para = document.createElement("p");
para.innerText = "This was added by JavaScript";
document.body.appendChild(para);
```

Topic 1: What is the DOM? (Super Simple)

Imagine your HTML file is a family tree. The whole document is the grandfather at the top. Under it comes the html tag. Under html come head and body. Under body come all your divs, headings, paragraphs, buttons, and so on.

The DOM is exactly this family tree — but in a form that JavaScript can read and change. When the browser loads your HTML, it converts it into this tree automatically.

Before DOM: You write HTML → browser shows it → nothing can change without reload.

With DOM: JavaScript can find any element, change its text, change its style, add new elements, or delete elements — ALL without reloading the page.

Tip: The DOM is what makes websites interactive. Without DOM manipulation, every change would require a full page reload.

Topic 2: Selecting Elements — 5 Ways

Before you can change anything on the page, you first need to SELECT it. There are 5 ways to select elements.

1. getElementById() — Select by ID (Returns ONE element)

```
// HTML: <h1 id="title">Hello</h1>
let heading = document.getElementById("title");
console.log(heading.innerText); // Output: Hello
```

Note: IDs must be unique on a page. getElementById returns only ONE element — never a list.

2. getElementsByClassName() — Select by Class (Returns HTMLCollection)

```
// HTML: <p class="text">P1</p>  <p class="text">P2</p>
let paras = document.getElementsByClassName("text");
console.log(paras.length); // Output: 2
console.log(paras[0].innerText); // Output: P1
```

3. getElementsByTagName() — Select by Tag (Returns HTMLCollection)

```
let allDivs = document.getElementsByTagName("div");
console.log(allDivs.length); // number of divs on the page
```

4. querySelector() — CSS Selector, Returns FIRST match only

This is the most flexible selector. You can use any CSS selector — id, class, tag, attribute, or a combination.

```
let heading   = document.querySelector("#title");     // by ID
let firstPara = document.querySelector(".text");      // by class — FIRST match
let firstBtn  = document.querySelector("button");     // by tag
let input     = document.querySelector("input[type='email']"); // by attribute
let nested    = document.querySelector(".card h2");   // nested selector
```

5. querySelectorAll() — CSS Selector, Returns ALL matches (NodeList)

```
let allParas = document.querySelectorAll(".text");    // all elements with class
"text"
let allBtns  = document.querySelectorAll("button");   // all buttons

// Loop through NodeList:
allParas.forEach(para => {
  console.log(para.innerText);
});
```

✅Remember: querySelector and querySelectorAll are the most modern and powerful selectors. They accept any CSS selector. querySelector returns the FIRST match, querySelectorAll returns ALL matches as a NodeList.

HTMLCollection vs NodeList — Interview Question!

getElementsByClassName and getElementsByTagName return HTMLCollection — it is LIVE (updates automatically when DOM changes).

querySelectorAll returns NodeList — it is STATIC (does not update when DOM changes).

Both can be looped with forEach only on NodeList. For HTMLCollection, use a for loop or convert with Array.from().

```
let collection = document.getElementsByClassName("item"); // HTMLCollection — live
let nodeList   = document.querySelectorAll(".item");       // NodeList — static

// Convert HTMLCollection to Array:
let arr = Array.from(collection);
arr.forEach(el => console.log(el.innerText));
```

Topic 3: Reading and Changing Content

innerText vs innerHTML vs textContent — Very Commonly Asked!

```
// HTML: <p id="para">Hello <strong>World</strong></p>
let el = document.getElementById("para");

console.log(el.innerText);   // Output: Hello World   (visible text only)
console.log(el.innerHTML);   // Output: Hello <strong>World</strong>  (with HTML tags)
console.log(el.textContent); // Output: Hello World   (ALL text, even hidden)

// Changing content:
el.innerText  = "New plain text";            // sets as plain text — safe
el.innerHTML  = "<em>New HTML content</em>"; // sets as HTML — can insert tags
```

✅Remember: Never use innerHTML with user-provided data — it can cause XSS (Cross-Site Scripting) attacks. A hacker can inject malicious scripts. Use innerText for plain text, innerHTML only for trusted HTML.

Getting and Setting Attributes

```
// HTML: <img id="photo" src="old.jpg" alt="Photo">
let img = document.getElementById("photo");

console.log(img.getAttribute("src"));     // Output: old.jpg
img.setAttribute("src", "new.jpg");       // Change src
img.setAttribute("alt", "New Photo");     // Change alt
img.removeAttribute("alt");               // Remove alt
console.log(img.hasAttribute("alt"));     // Output: false
```

Getting and Setting Styles

```
let box = document.getElementById("box");

// Set individual styles:
box.style.backgroundColor = "blue";
box.style.width           = "200px";
box.style.fontSize        = "20px";

// Read a computed style (actual applied style):
let computedStyle = window.getComputedStyle(box);
console.log(computedStyle.width); // Output: 200px
```

💡 Tip: In JavaScript, CSS property names use camelCase, not kebab-case. background-color becomes backgroundColor. font-size becomes fontSize.

Topic 4: classList — The Cleanest Way to Manage CSS Classes

Instead of directly changing style, the best practice is to add/remove CSS classes. This keeps your styles in CSS and logic in JavaScript — clean separation.

```
let btn = document.getElementById("myBtn");

btn.classList.add("active");         // Adds class "active"
btn.classList.remove("active");      // Removes class "active"
btn.classList.toggle("active");      // Adds if absent, removes if present
btn.classList.contains("active");    // Returns true or false
btn.classList.replace("old","new");  // Replace one class with another

// Add multiple classes at once:
btn.classList.add("btn-primary", "large", "rounded");

// Real example — dark mode toggle:
let toggleBtn = document.getElementById("darkToggle");
toggleBtn.addEventListener("click", () => {
  document.body.classList.toggle("dark-mode");
});
```

Topic 5: Events and Event Listeners — Very Important!

An event is any action the user does — clicking, typing, hovering, scrolling, submitting a form. An event listener waits for that action and runs code when it happens.

addEventListener — The Right Way

```
let btn = document.getElementById("myBtn");

btn.addEventListener("click", function(event) {
  console.log("Button clicked!");
  console.log(event.target); // the element that was clicked
});
```

Common Events — Must Know

Mouse Events: click, dblclick, mouseover, mouseout, mousedown, mouseup

Keyboard Events: keydown, keyup, keypress

Form Events: submit, change, input, focus, blur

Window Events: load, resize, scroll, DOMContentLoaded

```
// Keyboard event — detect what key was pressed:
document.addEventListener("keydown", function(event) {
  console.log("Key pressed: " + event.key);
  if (event.key === "Enter") console.log("Enter pressed!");
});

// Input event — fires on every character typed:
let input = document.getElementById("searchBox");
input.addEventListener("input", function(event) {
  console.log("Current value: " + event.target.value);
});
```

The Event Object — event.target vs event.currentTarget

event.target — the EXACT element that was clicked/interacted with.

event.currentTarget — the element the listener is attached to.

```
let parent = document.getElementById("parent");
parent.addEventListener("click", function(event) {
  console.log(event.target);        // element user actually clicked
  console.log(event.currentTarget); // always the parent (where listener is)
});
```

Removing an Event Listener

To remove a listener, you must pass the same named function — anonymous functions cannot be removed.

```
function handleClick() {
  console.log("Clicked!");
}

btn.addEventListener("click", handleClick);    // add
btn.removeEventListener("click", handleClick); // remove — works!

// This does NOT work for removal — anonymous function:
btn.addEventListener("click", () => console.log("hi"));
btn.removeEventListener("click", () => console.log("hi")); // FAILS
```

Topic 6: Event Bubbling, Capturing, and Delegation — Hot Interview Topics!

Event Bubbling — Events Bubble UP the Tree

When you click on a child element, the event first fires on the child, then moves UP to the parent, then grandparent, all the way to the document. This is called bubbling — like a bubble rising up.

```
// HTML structure:
// <div id="grandparent">
//   <div id="parent">
//     <button id="child">Click Me</button>
//   </div>
// </div>

document.getElementById("child").addEventListener("click", () => {
  console.log("1. Child clicked");
});
document.getElementById("parent").addEventListener("click", () => {
  console.log("2. Parent fired");
});
document.getElementById("grandparent").addEventListener("click", () => {
  console.log("3. Grandparent fired");
});

// When button is clicked, output is:
// 1. Child clicked
// 2. Parent fired
// 3. Grandparent fired
```

Stopping Bubbling — event.stopPropagation()

```
document.getElementById("child").addEventListener("click", (event) => {
  event.stopPropagation(); // stops event from bubbling up
  console.log("Only child fires, parent does not!");
});
```

Event Capturing — Events Come DOWN the Tree

Capturing is the opposite of bubbling — event goes from document DOWN to the target element. To use capturing, pass true as the third argument to addEventListener.

```
// Capturing phase — fires from TOP to BOTTOM:
parent.addEventListener("click", () => {
  console.log("Parent fires FIRST in capturing!");
}, true); // true = capturing mode
```

✅Remember: Default is BUBBLING (bottom to top). Capturing is top to bottom. In interviews, bubbling is asked much more than capturing.

Event Delegation — Most Efficient Pattern!

Instead of adding a listener to each child element separately, you add ONE listener to the parent and use event.target to figure out which child was clicked. This is called Event Delegation.

Why use it? If you have 100 list items, instead of 100 listeners, you use just 1 listener on the parent. Also works for elements added dynamically later!

```
// HTML: <ul id="list"> <li>Item 1</li> <li>Item 2</li> </ul>

// BAD way — separate listener for each item:
document.querySelectorAll("li").forEach(li => {
  li.addEventListener("click", () => console.log(li.innerText));
});
```

```
// GOOD way — one listener on parent (Event Delegation):
document.getElementById("list").addEventListener("click", (event) => {
  if (event.target.tagName === "LI") {
    console.log("Clicked: " + event.target.innerText);
  }
});
```

 Tip: Event Delegation is the preferred pattern in real projects. It is memory efficient and works even for dynamically added elements.

Preventing Default Behavior — event.preventDefault()

Some elements have default behavior — clicking a link navigates to a URL, submitting a form reloads the page. preventDefault() stops this default action.

```
// Stop link from navigating:
let link = document.querySelector("a");
link.addEventListener("click", (event) => {
  event.preventDefault();
  console.log("Link click prevented!");
});

// Stop form from reloading page:
let form = document.querySelector("form");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  console.log("Form submitted without page reload!");
});
```

Topic 7: Creating, Inserting, and Removing Elements

createElement and appendChild — Create and Add

```
// Create a new paragraph element:
let para = document.createElement("p");
para.innerText = "I was created by JavaScript!";
para.classList.add("highlight");
para.setAttribute("id", "newPara");

// Add to the end of body:
document.body.appendChild(para);

// Add inside a specific container:
let container = document.getElementById("container");
container.appendChild(para);
```

insertBefore, prepend, append, insertAdjacentHTML

```
let list = document.getElementById("myList");
let newItem = document.createElement("li");
newItem.innerText = "New Item";

list.prepend(newItem);        // Add at the BEGINNING of list
list.append(newItem);         // Add at the END of list (same as appendChild)

// insertAdjacentHTML — most flexible, insert HTML string anywhere:
```

```
list.insertAdjacentHTML("beforeend", "<li>Last Item</li>");
list.insertAdjacentHTML("afterbegin", "<li>First Item</li>");
```

Removing Elements

```
let el = document.getElementById("toDelete");

el.remove();                        // Modern way — remove the element itself
el.parentNode.removeChild(el);      // Old way — remove via parent

// Remove all children of a container:
let container = document.getElementById("container");
container.innerHTML = "";           // Fastest way to clear all children
```

Cloning Elements

```
let original = document.getElementById("card");

let shallowCopy = original.cloneNode(false); // clone only the element, not children
let deepCopy    = original.cloneNode(true);  // clone element AND all its children

document.body.appendChild(deepCopy);
```

Topic 8: DOM Traversal — Moving Through the Tree

Once you have an element, you can move to its relatives — parent, children, siblings — without selecting them by ID or class.

```
let el = document.getElementById("middle");

// Going UP:
console.log(el.parentNode);           // immediate parent
console.log(el.parentElement);        // same as parentNode for elements
console.log(el.closest(".wrapper")); // nearest ancestor matching selector

// Going DOWN:
console.log(el.children);             // all direct child elements
console.log(el.firstElementChild);    // first child element
console.log(el.lastElementChild);     // last child element
console.log(el.childNodes);           // all nodes including text nodes

// Going SIDEWAYS:
console.log(el.nextElementSibling);   // next sibling element
console.log(el.previousElementSibling); // previous sibling element
```

Topic 9: DOM Performance — Reflow, Repaint, DocumentFragment

Changing the DOM is expensive. Every time you add/remove/change an element, the browser may need to recalculate layout (reflow) and redraw pixels (repaint). Doing this many times in a loop is slow.

Bad Practice — Multiple DOM Changes in a Loop

```
let list = document.getElementById("list");

// BAD — hits the DOM 100 times, causes 100 reflows:
for (let i = 0; i < 100; i++) {
  let li = document.createElement("li");
```

```
  li.innerText = "Item " + i;
  list.appendChild(li); // DOM touched 100 times!
}
```

Good Practice — DocumentFragment (Batch DOM Changes)

DocumentFragment is an invisible container that exists in memory, not in the actual DOM. Build everything in it first, then insert it once. Only ONE reflow happens.

```
let list = document.getElementById("list");
let fragment = document.createDocumentFragment();

// GOOD — build in memory first:
for (let i = 0; i < 100; i++) {
  let li = document.createElement("li");
  li.innerText = "Item " + i;
  fragment.appendChild(li); // touches memory, NOT the real DOM
}

list.appendChild(fragment); // ONE DOM update — much faster!
```

✅Remember: Always batch DOM changes. If you need to add 100 elements, use DocumentFragment and insert once. This is a very impressive answer in performance interviews.

Topic 10: DOM Interview Questions with Solutions

Q1: What is the DOM? Is it part of JavaScript?

DOM is a tree-like representation of the HTML page that the browser creates. JavaScript uses it to read and change the page dynamically. The DOM is NOT part of JavaScript — it is a Web API provided by the browser. JavaScript is just the language used to interact with it.

Q2: What is the difference between innerHTML, innerText, and textContent?

innerHTML — gets/sets content including HTML tags. Can inject HTML. Risk of XSS.

innerText — gets/sets only visible text. Respects CSS (hidden text is excluded).

textContent — gets/sets all text including hidden text. Fastest and safest for plain text.

```
let el = document.getElementById("box");
// el.innerHTML gives HTML with tags
// el.innerText gives only visible text
// el.textContent gives all text including hidden
```

Q3: What is Event Bubbling? How do you stop it?

Event Bubbling means when an event fires on a child element, it then propagates upward to parent, grandparent, and eventually the document. Stop it with event.stopPropagation().

```
child.addEventListener("click", (e) => {
  e.stopPropagation(); // event stops here, does not bubble up
});
```

Q4: What is Event Delegation and why is it useful?

Event Delegation means attaching one event listener to a parent instead of many listeners on each child. It works because of event bubbling — clicks on children bubble up to the parent.

Benefits: fewer listeners (better memory), works for dynamically added elements.

```
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") console.log(e.target.innerText);
```

```
});
```

Q5: What is the difference between querySelector and getElementById?

getElementById is faster and older — only works with IDs.

querySelector is modern and flexible — accepts any CSS selector (ID, class, tag, attribute, nested).

```
document.getElementById("title")       // only by id
document.querySelector("#title")       // same result, but more flexible
document.querySelector(".card h2")     // can do nested selectors
```

Q6: What is the difference between HTMLCollection and NodeList?

HTMLCollection is returned by getElementsByClassName and getElementsByTagName. It is LIVE — updates automatically if the DOM changes.

NodeList is returned by querySelectorAll. It is STATIC — does not update when DOM changes.

forEach works directly on NodeList. For HTMLCollection, you need Array.from() first.

Q7: What happens when you set innerHTML with user input?

It is a security risk called XSS (Cross-Site Scripting). If a hacker enters a script tag as input and you set it as innerHTML, that script runs in the browser.

```
// DANGEROUS:
el.innerHTML = userInput; // if userInput = "<script>stealCookies()</script>"

// SAFE:
el.innerText = userInput; // treats input as plain text, no script runs
```

Q8: What is the difference between event.preventDefault() and event.stopPropagation()?

preventDefault() — stops the default browser action (like link navigation or form submit).

stopPropagation() — stops the event from bubbling up to parent elements.

They are independent — you can use one, both, or neither.

Q9: How would you improve performance when adding 1000 elements to the DOM?

Use DocumentFragment — build all elements in memory first, then insert into the DOM in ONE operation. This causes only one reflow instead of 1000.

```
let fragment = document.createDocumentFragment();
for (let i = 0; i < 1000; i++) {
  let div = document.createElement("div");
  div.innerText = "Item " + i;
  fragment.appendChild(div);
}
document.getElementById("container").appendChild(fragment);
```

Q10: What is the difference between window.onload and DOMContentLoaded?

DOMContentLoaded fires when HTML is fully parsed and DOM is ready — images and stylesheets may still be loading.

window.onload fires when EVERYTHING is loaded — HTML, images, stylesheets, scripts. It is slower.

```
document.addEventListener("DOMContentLoaded", () => {
  console.log("DOM ready — can start selecting elements");
});

window.addEventListener("load", () => {
```

```
  console.log("Everything loaded including images");
});
```

✅Remember: Always use DOMContentLoaded for your JavaScript that manipulates the DOM — it fires faster and you do not need to wait for images to load.

Topic 11: Quick Reference — All DOM Methods at a Glance

document.getElementById(id) — select by ID
document.getElementsByClassName(class) — select by class, returns live HTMLCollection
document.getElementsByTagName(tag) — select by tag, returns live HTMLCollection
document.querySelector(css) — first match, static
document.querySelectorAll(css) — all matches, returns static NodeList
el.innerText — get/set visible text
el.innerHTML — get/set HTML content
el.textContent — get/set all text (fastest)
el.getAttribute / setAttribute — read/write attributes
el.style.property — set inline style (camelCase)
el.classList.add/remove/toggle/contains — manage CSS classes
el.addEventListener(event, fn) — attach event listener
el.removeEventListener(event, fn) — detach event listener
event.preventDefault() — stop default browser action
event.stopPropagation() — stop event from bubbling
event.target — element that triggered event
document.createElement(tag) — create new element
parent.appendChild(child) — add child at end
parent.prepend / append / insertAdjacentHTML — flexible insertion
el.remove() — delete element from DOM
el.cloneNode(true/false) — clone element
el.parentElement / children / nextElementSibling — traverse tree
document.createDocumentFragment() — batch DOM updates for performance

✅Remember: Most asked in DOM interviews: event bubbling vs capturing, event delegation, innerHTML XSS risk, querySelector vs getElementById, HTMLCollection vs NodeList, and DocumentFragment for performance.

Chapter 11: Asynchronous JavaScript

How to Define in an Interview

"Asynchronous JavaScript means JavaScript can start a task, move on to other tasks while waiting, and come back to finish the first task when it is ready — without blocking the rest of the code. JavaScript is single-threaded, meaning it can only do one thing at a time. But with the help of the browser's Web APIs, the Event Loop, and the Callback Queue, it can handle time-consuming operations like API calls, timers, and file reading without freezing the page. The three main ways to handle async code in JavaScript are: Callbacks (old way), Promises (better way), and Async/Await (modern and cleanest way)."

Example you can say in interview:

```
console.log("1: Start");

setTimeout(() => {
  console.log("2: This runs after 2 seconds");
}, 2000);

console.log("3: End");

// Output: 1: Start → 3: End → 2: This runs after 2 seconds
// JavaScript did NOT wait at line 2 — it moved on and came back later
```

Topic 1: Synchronous vs Asynchronous — The Big Difference

Synchronous — Do One Thing at a Time, Wait for Each

Imagine you are at a coffee shop. You order a coffee. The cashier STOPS everything, goes and makes the coffee himself, gives it to you, and THEN takes the next customer's order. Every customer has to wait.

This is synchronous — everything happens one by one. Nobody moves until the current task is done.

```
console.log("Step 1: Wake up");
console.log("Step 2: Brush teeth");
console.log("Step 3: Have breakfast");
// Output always in this exact order: Step 1 → Step 2 → Step 3
```

Asynchronous — Start a Task, Move On, Come Back When Ready

Now imagine a better coffee shop. You order coffee. The cashier gives your order to the barista and IMMEDIATELY takes the next customer's order. When your coffee is ready, they call your name. You get your coffee WITHOUT making everyone else wait.

This is asynchronous — you start a task, move on to other things, and handle the result when it is ready.

```
console.log("Step 1: Order placed");

setTimeout(() => {                // This is like the barista making coffee
  console.log("Step 3: Coffee ready!");
}, 2000);                         // 2 seconds later

console.log("Step 2: Doing other things while waiting...");

// Output: Step 1 → Step 2 → Step 3 (after 2 seconds)
```

Notice: Step 2 printed BEFORE Step 3 even though Step 3 is written above it in the code. JavaScript did NOT wait at setTimeout.

> ✅**Remember: Synchronous = blocking (wait). Asynchronous = non-blocking (do not wait, come back later). This is the most fundamental concept in this chapter.**

Topic 2: Why Does Asynchronous Code Exist? (Real Reason)

JavaScript runs in the browser. The browser has ONE thread for JavaScript — meaning it can only do ONE thing at a time. If JavaScript stops and waits for a slow task, the entire page FREEZES. The user cannot click anything, scroll, or type.

Example: Imagine fetching data from a server. The server might take 3 seconds to respond. If JavaScript WAITED for 3 seconds, the page would be completely frozen and unresponsive. Nobody would use your website.

Async code solves this: JavaScript says 'start fetching data, browser you handle that, I will continue running other code. Tell me when data arrives.' When data arrives, JavaScript handles it.

> **Tip: Things that are always asynchronous in JavaScript: API/network calls (fetch), timers (setTimeout, setInterval), reading files, database queries, geolocation requests.**

Topic 3: How JavaScript Actually Works — Event Loop (Very Important!)

This is a concept that is asked in almost every senior JavaScript interview. Understand it step by step.

JavaScript has 4 parts working together:

1. Call Stack — this is where JavaScript actually runs code, one line at a time. Like a stack of plates — last in, first out.

2. Web APIs — these are browser tools (not JavaScript itself) that handle async tasks. Examples: setTimeout, fetch, DOM events. When you call setTimeout, the browser handles the countdown, not JavaScript.

3. Callback Queue — when a Web API task is done (like timer finished), its callback function is placed here, waiting to be picked up.

4. Event Loop — it constantly watches: 'Is the Call Stack empty? If yes, pick the next function from the Callback Queue and push it to the Call Stack.'

Step-by-Step Example

```
console.log("A");           // Line 1

setTimeout(() => {          // Line 2
  console.log("B");
}, 0);                      // 0 milliseconds delay

console.log("C");           // Line 3

// Output: A → C → B
```

Why does B print last even though delay is 0ms?

Step 1: 'A' goes to Call Stack → runs → prints A → removed.

Step 2: setTimeout goes to Call Stack → browser takes the callback to Web APIs → removed from Call Stack.

Step 3: 'C' goes to Call Stack → runs → prints C → removed.

Step 4: Timer (0ms) finishes in Web APIs → callback moves to Callback Queue.

Step 5: Event Loop sees Call Stack is empty → moves callback from Queue to Call Stack → runs → prints B.

> ✅**Remember: Even setTimeout with 0ms delay does NOT run immediately. It always runs AFTER the current code finishes. This is because it goes through Web APIs and the Callback Queue first.**

Topic 4: Callbacks — The Old Way to Handle Async

A callback is just a function that you pass to another function. That other function will CALL it BACK when the task is done.

Real life example: You give your phone number to a pizza shop. When the pizza is ready, they call you back. Your phone number is the callback — it gets called when something is ready.

Simple Callback Example

```
function makePizza(topping, callback) {
  console.log("Making " + topping + " pizza...");
  setTimeout(() => {
    let pizza = topping + " pizza is ready!";
    callback(pizza);            // call the function that was passed in
  }, 2000);
}

makePizza("Cheese", function(result) {
  console.log(result);         // Output: Cheese pizza is ready!
});
```

Callback Hell — The Problem with Callbacks

What if you need to do multiple async tasks one after another? Like: order pizza, then get drinks, then get dessert, then pay the bill — each one waiting for the previous. With callbacks, this looks like a pyramid of doom:

```
orderPizza(function(pizza) {
  console.log("Got pizza");
  getDrinks(function(drinks) {
    console.log("Got drinks");
    getDessert(function(dessert) {
      console.log("Got dessert");
      payBill(function(receipt) {
        console.log("Paid!"); // deeply nested — very hard to read!
      });
    });
  });
});
```

> *Note: This deeply nested structure is called Callback Hell or Pyramid of Doom. It is very hard to read, debug, and maintain. This is exactly why Promises were invented.*

Topic 5: Promises — The Better Way

A Promise is like a receipt you get from a restaurant. When you order food, they give you a receipt (the promise). The receipt says: 'Your food WILL come.' It does not have the food yet, but it promises it will.

Later, one of two things happen:

- The food arrives — this is called RESOLVED (success)
- The kitchen is out of that item — this is called REJECTED (failure)

3 States of a Promise

Pending — the task is still going on. You are waiting.

Resolved — the task finished successfully. You got what you wanted.

Rejected — the task failed. Something went wrong.

```
// Creating a Promise:
let myPromise = new Promise((resolve, reject) => {
  let success = true;

  setTimeout(() => {
    if (success) {
      resolve("Task done!");    // call resolve when success
    } else {
      reject("Something failed!"); // call reject when failure
    }
  }, 2000);
});

// Handling the Promise:
myPromise
  .then(result => console.log(result))   // runs when resolved
  .catch(error => console.log(error))    // runs when rejected
  .finally(() => console.log("Done!"));  // runs ALWAYS, success or fail
```

Promise Chaining — Fix for Callback Hell

Promises can be chained using .then() — each .then() returns a new promise, allowing you to write async steps one after another in a flat, readable way.

```
orderPizza()
  .then(pizza  => { console.log("Got:", pizza);  return getDrinks(); })
  .then(drinks => { console.log("Got:", drinks); return getDessert(); })
  .then(dessert => { console.log("Got:", dessert); return payBill(); })
  .then(receipt => console.log("Paid!", receipt))
  .catch(error  => console.log("Something went wrong:", error));
```

Compare this to the callback hell version — much flatter and easier to read!

> **✅Remember: A Promise can only be resolved or rejected ONCE. After that, its state never changes. This is called being 'settled'.**

Topic 6: async / await — The Cleanest and Modern Way

async/await is just a cleaner way to write Promise code. It makes async code LOOK like normal step-by-step synchronous code. Much easier to read and understand.

Think of it like this: instead of saying 'when pizza is ready, THEN do this', you say 'WAIT for the pizza, and when it is ready, continue.'

async keyword — Makes a Function Return a Promise

Adding 'async' before a function makes it automatically return a Promise, even if you just return a normal value.

```
async function greet() {
  return "Hello!";   // automatically wrapped in a Promise
}

greet().then(msg => console.log(msg)); // Output: Hello!
```

await keyword — Wait for a Promise to Finish

'await' pauses the function at that line until the Promise is resolved. But it does NOT block the rest of the program — only this async function pauses.

```
function waitTwoSeconds() {
  return new Promise(resolve => setTimeout(() => resolve("Done!"), 2000));
}

async function main() {
  console.log("Start");
  let result = await waitTwoSeconds(); // wait here until promise resolves
  console.log(result);                 // Output: Done!  (after 2 seconds)
  console.log("End");
}

main();
// Output: Start → Done! (after 2 sec) → End
```

Note: await can ONLY be used INSIDE an async function. If you try to use it outside, you get an error.

Error Handling with try/catch

With async/await, you handle errors using try/catch — just like normal synchronous code. Much cleaner than .catch() chaining.

```
async function fetchData() {
  try {
    let response = await fetch("https://api.example.com/users");
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Error fetching data:", error.message);
  } finally {
    console.log("This always runs — success or error");
  }
}

fetchData();
```

Callbacks vs Promises vs Async/Await — Same Thing, 3 Ways

```
// ─── Callback way ───
getData(function(data) {
  processData(data, function(result) {
    console.log(result);
  });
});

// ─── Promise way ───
getData()
  .then(data => processData(data))
  .then(result => console.log(result))
  .catch(err => console.log(err));

// ─── Async/Await way (cleanest) ───
async function run() {
  try {
```

```
    let data   = await getData();
    let result = await processData(data);
    console.log(result);
  } catch (err) {
    console.log(err);
  }
}
```

✅Remember: All three do the same thing. Callbacks are old and hard to read. Promises are better. Async/Await is the most modern, cleanest, and recommended way.

Topic 7: Promise.all, Promise.race, Promise.allSettled, Promise.any

These four methods let you handle MULTIPLE promises at the same time.

Promise.all() — Wait for ALL, Fail if ANY Fails

Runs all promises at the same time. Waits for ALL to finish. If even ONE fails, everything fails.

Use case: You need user data AND order data AND payment info — all three needed before showing the page.

```
let getUser    = fetch("/api/user");
let getOrders  = fetch("/api/orders");
let getPayment = fetch("/api/payment");

Promise.all([getUser, getOrders, getPayment])
  .then(([user, orders, payment]) => {
    console.log("All data received!");
  })
  .catch(error => {
    console.log("One of them failed:", error); // ALL fail if any one fails
  });
```

Promise.race() — First One to Finish Wins

Runs all promises at the same time. Returns result of whichever finishes first — success OR failure.

Use case: Timeout — if API takes more than 5 seconds, show an error.

```
let apiCall = fetch("/api/data");
let timeout = new Promise((_, reject) =>
  setTimeout(() => reject("Request timed out!"), 5000)
);

Promise.race([apiCall, timeout])
  .then(data => console.log("Got data:", data))
  .catch(err => console.log(err)); // "Request timed out!" if API is slow
```

Promise.allSettled() — Wait for ALL, Never Fails

Runs all promises at the same time. Waits for ALL to finish — but does NOT fail even if some reject. You get the result of each promise individually.

Use case: Send 5 emails — some may fail, but you want to know the result of each one.

```
let p1 = Promise.resolve("Success 1");
let p2 = Promise.reject("Failed 2");
let p3 = Promise.resolve("Success 3");

Promise.allSettled([p1, p2, p3]).then(results => {
  results.forEach(result => {
```

```
    if (result.status === "fulfilled") {
      console.log("OK:", result.value);
    } else {
      console.log("FAIL:", result.reason);
    }
  });
});
// Output:
// OK: Success 1
// FAIL: Failed 2
// OK: Success 3
```

Promise.any() — First SUCCESS Wins (Ignores Failures)

Returns the first promise that RESOLVES (succeeds). Ignores rejections. Only fails if ALL promises fail.

Use case: Try 3 different servers — use whichever responds first successfully.

```
let server1 = Promise.reject("Server 1 down");
let server2 = Promise.resolve("Server 2 responded!");
let server3 = Promise.resolve("Server 3 responded!");

Promise.any([server1, server2, server3])
  .then(result => console.log(result)) // Output: Server 2 responded!
  .catch(err => console.log("All failed!", err));
```

✅Remember: all = ALL must succeed. race = first to finish (success or fail). allSettled = wait for all, never throws. any = first SUCCESS wins.

Topic 8: fetch() — Getting Data from an API

fetch() is the modern, built-in way to make HTTP requests in JavaScript. It returns a Promise.

Think of fetch like a delivery order. You make the request (place the order), wait for a response (delivery), and then unpack the response (open the package).

Basic fetch with async/await

```
async function getUsers() {
  try {
    // Step 1: Make the request
    let response = await fetch("https://jsonplaceholder.typicode.com/users");

    // Step 2: Check if request was successful
    if (!response.ok) {
      throw new Error("HTTP error! Status: " + response.status);
    }

    // Step 3: Convert response to JSON
    let users = await response.json();

    // Step 4: Use the data
    console.log(users[0].name); // Output: Leanne Graham

  } catch (error) {
    console.log("Fetch failed:", error.message);
  }
}
```

```
getUsers();
```

 Note: response.json() also returns a Promise — that is why we need await before it too. This is a very common mistake beginners make.

POST Request — Sending Data to Server

```
async function createUser(userData) {
  try {
    let response = await fetch("https://jsonplaceholder.typicode.com/users", {
      method: "POST",                          // HTTP method
      headers: {
        "Content-Type": "application/json"     // tell server we are sending JSON
      },
      body: JSON.stringify(userData)           // convert object to JSON string
    });

    let result = await response.json();
    console.log("Created user:", result);

  } catch (error) {
    console.log("Error:", error.message);
  }
}

createUser({ name: "Shaaz", email: "shaaz@mail.com" });
```

 Tip: Always set Content-Type: application/json header when sending JSON data. Always use JSON.stringify() on the body. Always use JSON.parse() or response.json() when receiving.

Topic 9: setTimeout and setInterval

setTimeout — Run Once After a Delay

```
// Run once after 2 seconds:
let timer = setTimeout(() => {
  console.log("Runs once after 2 seconds");
}, 2000);

// Cancel before it runs:
clearTimeout(timer);
```

setInterval — Run Repeatedly at Every N milliseconds

```
let count = 0;

let interval = setInterval(() => {
  count++;
  console.log("Count:", count);
  if (count === 5) {
    clearInterval(interval); // stop after 5 times
  }
}, 1000); // runs every 1 second
// Output: Count:1, Count:2, Count:3, Count:4, Count:5  (then stops)
```

✅Remember: Always save the timer ID returned by setTimeout/setInterval so you can cancel it with clearTimeout/clearInterval when needed. Forgetting to clear intervals is a very common memory leak bug.

Topic 10: Async JavaScript Interview Questions with Solutions

Q1: What is the difference between synchronous and asynchronous code?

Synchronous — code runs line by line. Each line waits for the previous one to finish. If one task is slow, everything blocks.

Asynchronous — code can start a task and move on. When the task is done, it handles the result. Does not block other code from running.

Q2: What will this print? (Most Tricky Interview Question!)

```
console.log("1");

setTimeout(() => console.log("2"), 0);

console.log("3");
```

Answer: 1 → 3 → 2

Why? setTimeout goes to Web APIs even with 0ms delay. The Call Stack runs '1' and '3' first. Only after Call Stack is empty, the Event Loop pushes the setTimeout callback. So '2' always prints last.

Q3: What is the difference between Promise.all and Promise.allSettled?

Promise.all — waits for all promises. If ANY one fails, the whole thing fails immediately. You don't get results of the successful ones.

Promise.allSettled — waits for all promises. Even if some fail, it gives you the result of every single promise individually — success or failure.

```
// Use Promise.all when: ALL must succeed (e.g., loading page data)
// Use Promise.allSettled when: you want results of all, even if some fail
```

Q4: What is the difference between .then().catch() and try/catch with async/await?

Both handle promise errors. .then().catch() is chaining style. try/catch with async/await looks like synchronous error handling — cleaner and easier to read. async/await is preferred in modern code.

```
// Promise style:
fetch(url).then(r => r.json()).catch(e => console.log(e));

// Async/await style (preferred):
async function load() {
  try { let data = await fetch(url); }
  catch (e) { console.log(e); }
}
```

Q5: Can you use await outside of an async function?

No — in regular code you cannot. await must always be inside an async function. However, in modern JavaScript (ES2022+), top-level await is supported in ES Modules.

```
// This causes an ERROR in regular scripts:
let data = await fetch(url); // SyntaxError!

// Must be inside async function:
async function load() {
```

```
  let data = await fetch(url); // Works fine!
}
```

Q6: What is a Promise? What are its 3 states?

A Promise is an object representing the eventual completion or failure of an async operation. It has 3 states: Pending (still running), Fulfilled/Resolved (success), and Rejected (failure). Once a promise is resolved or rejected, it cannot change state again.

Q7: What is the Event Loop? Explain simply.

JavaScript has a Call Stack where code runs. When async tasks (like setTimeout) are done, their callbacks go to the Callback Queue. The Event Loop watches the Call Stack — when it is empty, it takes the next function from the Callback Queue and pushes it to the Call Stack. This is how JavaScript handles async without multiple threads.

Q8: What is the difference between Promise.race and Promise.any?

Promise.race — returns the first promise to SETTLE — whether success or failure.

Promise.any — returns the first promise to SUCCEED. Ignores failures. Only fails if ALL fail.

```
// race: first to finish wins, even if it fails
// any:  first to succeed wins, failures are ignored
```

Q9: What is async/await? Is it different from Promises?

async/await is NOT a different mechanism — it is just cleaner SYNTAX built on top of Promises. Under the hood, it uses Promises. An async function always returns a Promise. await pauses the async function until the Promise resolves, but does not block the rest of the program.

Q10: What happens if you forget await before a Promise?

```
async function getData() {
  let response = fetch("https://api.example.com/data"); // forgot await!
  console.log(response); // Output: Promise { <pending> }  NOT the actual data!
}
```

Without await, you get a pending Promise object — not the actual data. Always use await before any function that returns a Promise.

Topic 11: Quick Reference — All Async Concepts at a Glance

setTimeout(fn, ms) — run fn once after ms milliseconds

setInterval(fn, ms) — run fn every ms milliseconds

clearTimeout(id) — cancel a setTimeout

clearInterval(id) — cancel a setInterval

new Promise((res, rej)) — create a promise manually

.then(fn) — runs when promise resolves (success)

.catch(fn) — runs when promise rejects (failure)

.finally(fn) — always runs, success or failure

async function — function that always returns a Promise

await expression — wait for a Promise to resolve (inside async only)

Promise.all([]) — wait for ALL, fail if any fails

Promise.race([]) — first to settle (success or fail) wins

Promise.allSettled([]) — wait for ALL, never fails, gives each result

Promise.any([]) — first to SUCCEED wins, ignores failures

fetch(url) — make HTTP request, returns Promise

response.json() — parse response as JSON, returns Promise

response.ok — true if status is 200-299

✅Remember: Most asked in async interviews: synchronous vs asynchronous, Event Loop explanation, setTimeout output trick, Promise states, Promise.all vs allSettled, async/await vs Promises, and forgetting await bug.

Chapter 12: Error Handling

How to Define in an Interview

"Error handling in JavaScript is the process of gracefully managing errors so your program does not crash. JavaScript provides try-catch-finally blocks to catch runtime errors, the throw statement to create custom errors, and different Error types (TypeError, ReferenceError, SyntaxError, RangeError). In async code, errors are caught using .catch() or try-catch with async/await."

Topic 1: Why Error Handling Exists

Imagine you are using an app and the server is down. Without error handling — the app freezes or shows a blank white screen. With error handling — the app shows a friendly message: 'Something went wrong, please try again.'

Errors happen everywhere — wrong user input, network failure, file not found, invalid data from API. You cannot stop errors from happening. But you CAN handle them gracefully so users have a good experience.

```
// WITHOUT error handling — program crashes completely:
let data = JSON.parse("invalid json text"); // throws SyntaxError
console.log("This line never runs"); // app is dead

// WITH error handling — program continues:
try {
  let data = JSON.parse("invalid json text");
} catch (error) {
  console.log("Bad data received:", error.message);
  console.log("Using default data instead");
}
console.log("App keeps running!"); // this runs fine
```

Topic 2: try / catch / finally — The Main Tool

try — put risky code here. If it throws, execution stops and jumps to catch.

catch — runs ONLY if try block threw an error. Gets the error object.

finally — ALWAYS runs whether there was an error or not. Used for cleanup.

```
function readUserData(userId) {
  try {
    // Risky code:
    if (!userId) throw new Error("userId is required");
    let data = JSON.parse('{"name":"Shaaz"}');
    console.log("Data loaded:", data.name);
    return data;
  } catch (error) {
    // Runs only if error:
    console.log("Error:", error.message);
    return null;
  } finally {
    // ALWAYS runs — good for cleanup:
    console.log("Finished reading — cleanup done");
  }
}
```

```
readUserData(1);    // Data loaded: Shaaz → Finished reading
readUserData(null); // Error: userId is required → Finished reading
```

✅Remember: finally is perfect for cleanup tasks — close database connections, hide loading spinners, release resources — things that must happen whether the operation succeeded or failed.

Topic 3: The Error Object — What You Get in catch

When an error is caught, you get an Error object with useful properties.

```
try {
  undeclaredVariable; // ReferenceError
} catch (error) {
  console.log(error.name);    // ReferenceError
  console.log(error.message); // undeclaredVariable is not defined
  console.log(error.stack);   // full stack trace (where error happened)
}
```

Topic 4: Types of Errors in JavaScript

SyntaxError — Wrong Grammar

Code is written incorrectly. JavaScript cannot even parse it. Happens before the code runs.

```
// let x = ;   // SyntaxError — missing value
// function(   // SyntaxError — incomplete
```

ReferenceError — Variable Does Not Exist

You tried to use a variable that was never declared.

```
try {
  console.log(ghostVariable); // never declared
} catch (e) {
  console.log(e.name);    // ReferenceError
  console.log(e.message); // ghostVariable is not defined
}
```

TypeError — Wrong Data Type Operation

You tried to do something on a value that does not support it.

```
try {
  let num = 42;
  num.toUpperCase(); // numbers have no toUpperCase
} catch (e) {
  console.log(e.name);    // TypeError
  console.log(e.message); // num.toUpperCase is not a function
}

try {
  null.name; // cannot read property of null
} catch (e) {
  console.log(e.name); // TypeError
}
```

RangeError — Value Out of Allowed Range

```
try {
  let arr = new Array(-5); // negative length
} catch (e) {
```

```
  console.log(e.name);    // RangeError
  console.log(e.message); // Invalid array length
}
```

Topic 5: throw — Create Your Own Errors

You can throw your own errors when something goes wrong in your business logic — not just JavaScript runtime errors. Use throw new Error() to create a descriptive error.

```
function validateAge(age) {
  if (typeof age !== "number") throw new TypeError("Age must be a number");
  if (age < 0)                 throw new RangeError("Age cannot be negative");
  if (age > 150)               throw new RangeError("Age cannot exceed 150");
  return true;
}

try { validateAge("twenty"); }
catch (e) { console.log(e.name + ":", e.message); }
// TypeError: Age must be a number

try { validateAge(-5); }
catch (e) { console.log(e.name + ":", e.message); }
// RangeError: Age cannot be negative

try { validateAge(25); console.log("Valid age!"); }
catch (e) { console.log(e.message); }
// Valid age!
```

Custom Error Classes

You can create your own Error types by extending the Error class. This is very useful in large applications to distinguish between different kinds of errors.

```
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name  = "ValidationError";
    this.field = field;
  }
}

class NetworkError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.name       = "NetworkError";
    this.statusCode = statusCode;
  }
}

function processForm(data) {
  if (!data.email) throw new ValidationError("Email is required", "email");
  if (!data.name)  throw new ValidationError("Name is required",  "name");
}

try {
  processForm({ name: "Shaaz" });
```

```
} catch (e) {
  if (e instanceof ValidationError) {
    console.log(`Field "${e.field}" error: ${e.message}`);
    // Field "email" error: Email is required
  } else if (e instanceof NetworkError) {
    console.log(`Network error ${e.statusCode}: ${e.message}`);
  } else {
    console.log("Unknown error:", e.message);
  }
}
```

✅Remember: Custom error classes let you handle different types of errors differently. Use instanceof to check what kind of error was thrown.

Topic 6: Error Handling in Async Code

Promises — use .catch()

```
fetch("https://api.example.com/data")
  .then(response => {
    if (!response.ok) throw new Error(`HTTP error: ${response.status}`);
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.log("Failed:", error.message))
  .finally(() => console.log("Request complete"));
```

Async/Await — use try-catch (cleaner)

```
async function loadData() {
  try {
    let response = await fetch("https://api.example.com/data");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    let data = await response.json();
    console.log("Loaded:", data);
  } catch (error) {
    console.log("Error loading data:", error.message);
  } finally {
    console.log("Loading finished");
  }
}
loadData();
```

Topic 7: Error Handling Interview Questions

Q1: What is the difference between throw and try-catch?

throw is used to CREATE and send an error. try-catch is used to CATCH and handle errors that were thrown. They work together — throw sends the error, catch receives it.

Q2: When does finally run?

finally ALWAYS runs — whether the try block succeeded or the catch block ran. Even if you return inside try or catch, finally still runs. It is perfect for cleanup like closing connections or hiding loading spinners.

Q3: What are the 4 common error types?

SyntaxError — invalid code syntax. ReferenceError — variable not declared. TypeError — wrong operation on a data type. RangeError — value outside allowed range.

Q4: How do you handle errors in async/await?

Wrap the await calls in a try-catch block. The catch block receives any error thrown by the awaited promise — whether it is a network error, HTTP error, or parsing error.

Q5: What is a custom Error class?

By extending the built-in Error class, you create your own error types with custom properties. This lets you use instanceof to identify which type of error was thrown and handle each one differently.

Chapter 13: Storing Data in the Browser

How to Define in an Interview

"Browsers provide three main ways to store data on the client side — localStorage, sessionStorage, and Cookies. localStorage stores data permanently with no expiry until manually cleared. sessionStorage stores data only for the current browser tab session — it is deleted when the tab is closed. Cookies are small pieces of data that are sent to the server with every HTTP request and are mainly used for authentication and tracking. All three only store data as strings, so objects must be converted using JSON.stringify() before storing and JSON.parse() when reading back."

Example you can say in interview:

```
// localStorage — permanent
localStorage.setItem("name", "Shaaz");
console.log(localStorage.getItem("name")); // Output: Shaaz

// sessionStorage — gone when tab closes
sessionStorage.setItem("page", "2");

// Cookie — sent to server with every request
document.cookie = "token=abc123; expires=Fri, 31 Dec 2025 23:59:59 GMT";
```

Topic 1: Why Do We Store Data in the Browser?

Think about when you log into a website like Amazon. After logging in, you close the tab and open it again — you are still logged in! The website did not ask for your password again. How?

Because your login information was saved in the browser. The browser remembered who you are.

Other real examples:

- Dark mode preference — the website remembers you like dark mode
- Shopping cart — items stay even if you refresh the page
- Form data — some sites save what you typed so you do not lose it
- Language setting — website remembers you selected Hindi or English

Tip: Storing data in the browser reduces the need to go back to the server again and again. This makes the website faster and better for the user.

Topic 2: localStorage — Permanent Storage

localStorage is like a permanent locker in your browser. You put something inside and it stays there FOREVER — even if you close the tab, close the browser, or restart your computer. It only goes away when you manually clear it.

Basic Operations — setItem, getItem, removeItem, clear

```
// SAVE data:
localStorage.setItem("username", "Shaaz");
localStorage.setItem("theme", "dark");

// READ data:
let name = localStorage.getItem("username");
console.log(name); // Output: Shaaz

// READ non-existing key — returns null:
```

```
console.log(localStorage.getItem("salary")); // Output: null

// DELETE one item:
localStorage.removeItem("theme");

// DELETE everything:
localStorage.clear();
```

Storing Objects and Arrays — JSON.stringify and JSON.parse

localStorage can only store STRINGS. If you try to store an object directly, it will be converted to '[object Object]' which is useless. You must convert it to a string first using JSON.stringify().

```
let user = { name: "Shaaz", age: 23, city: "Delhi" };

// WRONG way — stores "[object Object]"
localStorage.setItem("user", user); // WRONG!

// CORRECT way — convert to JSON string first
localStorage.setItem("user", JSON.stringify(user));

// Reading back — convert from string to object
let stored = localStorage.getItem("user");
let parsed = JSON.parse(stored);
console.log(parsed.name); // Output: Shaaz
console.log(parsed.age);  // Output: 23
```

✅Remember: Always use JSON.stringify() when saving objects/arrays to localStorage. Always use JSON.parse() when reading them back. Forgetting this is the most common mistake.

Checking if a Key Exists Safely

```
let data = localStorage.getItem("username");

if (data !== null) {
  console.log("Found: " + data);
} else {
  console.log("Key does not exist");
}
```

Looping Through All localStorage Items

```
// Save some items first:
localStorage.setItem("name", "Shaaz");
localStorage.setItem("theme", "dark");
localStorage.setItem("lang", "en");

// Loop through all:
for (let i = 0; i < localStorage.length; i++) {
  let key   = localStorage.key(i);
  let value = localStorage.getItem(key);
  console.log(`${key} : ${value}`);
}
// Output:
// name  : Shaaz
// theme : dark
// lang  : en
```

Topic 3: sessionStorage — Temporary Storage

sessionStorage is like a whiteboard in a meeting room. Whatever you write on it stays as long as the meeting (tab) is running. The moment the meeting ends (tab is closed), everything is wiped clean.

Real example: An online exam website saves your current question number and timer in sessionStorage. If you close the tab by mistake, that data is lost — which is intentional so you cannot cheat by opening the exam in another tab.

Basic Operations — exactly same API as localStorage

```
// SAVE:
sessionStorage.setItem("currentPage", "3");
sessionStorage.setItem("score", "45");

// READ:
console.log(sessionStorage.getItem("currentPage")); // Output: 3

// DELETE one:
sessionStorage.removeItem("score");

// DELETE all:
sessionStorage.clear();
```

Note: sessionStorage is tab-specific. If you open the same website in two different tabs, each tab has its own completely separate sessionStorage. They do NOT share data.

Storing Objects in sessionStorage

```
let cart = { item: "Laptop", qty: 1, price: 50000 };
sessionStorage.setItem("cart", JSON.stringify(cart));

let savedCart = JSON.parse(sessionStorage.getItem("cart"));
console.log(savedCart.item);  // Output: Laptop
console.log(savedCart.price); // Output: 50000
```

Topic 4: Cookies — The Bridge Between Browser and Server

Cookies are the oldest way to store data. The key difference from localStorage and sessionStorage is that cookies are automatically SENT to the server with EVERY request. The server can also CREATE cookies and send them to the browser.

Real example: When you log in on a website, the server creates a cookie with your session token and sends it to your browser. Every time your browser makes a request to that website, it sends the cookie back — so the server knows who you are without asking for password again.

Creating a Cookie

```
// Basic cookie:
document.cookie = "username=Shaaz";

// With expiry date:
document.cookie = "username=Shaaz; expires=Fri, 31 Dec 2025 23:59:59 GMT";

// With path — accessible on all pages of the site:
document.cookie = "username=Shaaz; expires=Fri, 31 Dec 2025 23:59:59 GMT; path=/";

// Secure — only sent over HTTPS:
```

```
document.cookie = "token=abc123; Secure; SameSite=Strict";
```

Reading Cookies

Reading cookies is a bit tricky because document.cookie returns ALL cookies as one long string.

```
// All cookies come as one string:
console.log(document.cookie);
// Output: "username=Shaaz; theme=dark; lang=en"

// Function to get a specific cookie by name:
function getCookie(name) {
  let cookies = document.cookie.split("; ");
  for (let cookie of cookies) {
    let [key, value] = cookie.split("=");
    if (key === name) return value;
  }
  return null;
}
console.log(getCookie("username")); // Output: Shaaz
```

Deleting a Cookie

To delete a cookie, set its expiry date to a date in the past.

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 GMT; path=/";
```

Tip: Setting the expiry to January 1, 1970 (the past) immediately expires the cookie and the browser deletes it.

Cookie Properties — Must Know for Interviews

expires / max-age — when the cookie should expire. Without this, cookie is deleted when browser closes (session cookie).

path — which pages can access this cookie. path=/ means all pages.

domain — which domain can access this cookie.

Secure — cookie is only sent over HTTPS connections.

HttpOnly — cookie cannot be accessed by JavaScript (only server). Protects against XSS attacks.

SameSite — controls when cookie is sent with cross-site requests. Protects against CSRF attacks.

✅Remember: HttpOnly cookies are very important for security. If a cookie has HttpOnly flag, JavaScript cannot read it — only the server can. This prevents hackers from stealing cookies using XSS attacks.

Topic 5: localStorage vs sessionStorage vs Cookies — Full Comparison

Storage Limit:

localStorage — about 5MB per origin
sessionStorage — about 5MB per origin
Cookies — only 4KB per cookie

Expiry:

localStorage — no expiry, stays forever until manually cleared
sessionStorage — deleted when the tab/browser is closed
Cookies — you set the expiry date yourself

Sent to Server:

localStorage — NO, stays only in browser
sessionStorage — NO, stays only in browser
Cookies — YES, sent with every HTTP request automatically

Accessible from JavaScript:

localStorage — YES
sessionStorage — YES
Cookies — YES (unless HttpOnly flag is set)

Tab Specific:

localStorage — NO, shared across all tabs of same origin
sessionStorage — YES, each tab has its own separate storage
Cookies — NO, shared across all tabs

Best Used For:

localStorage — user preferences, theme, language, saved items
sessionStorage — temporary data like current page, exam timer
Cookies — authentication tokens, session IDs, tracking

Topic 6: JSON.stringify() and JSON.parse() — Very Important!

These two methods are used everywhere — not just with storage but also when sending data to an API or receiving it. You must understand them clearly.

JSON.stringify() — Object to String

Converts a JavaScript object or array into a JSON string so it can be stored or sent.

```
let user = {
  name: "Shaaz",
  age: 23,
  hobbies: ["coding", "reading"],
  address: { city: "Delhi" }
};

let jsonString = JSON.stringify(user);
console.log(jsonString);
// Output:
{"name":"Shaaz","age":23,"hobbies":["coding","reading"],"address":{"city":"Delhi"}}
console.log(typeof jsonString); // Output: string
```

Note: Functions, undefined, and Symbol values are REMOVED by JSON.stringify(). They cannot be represented in JSON.

JSON.parse() — String Back to Object

Converts a JSON string back into a real JavaScript object that you can use.

```
let obj = JSON.parse(jsonString);
console.log(obj.name);           // Output: Shaaz
```

```
console.log(obj.hobbies[0]);     // Output: coding
console.log(obj.address.city);   // Output: Delhi
console.log(typeof obj);         // Output: object
```

Pretty Printing — Make JSON Readable

```
// Add indentation for readability:
console.log(JSON.stringify(user, null, 2));
// Output:
// {
//   "name": "Shaaz",
//   "age": 23,
//   "hobbies": ["coding", "reading"],
//   "address": { "city": "Delhi" }
// }
```

Tip: JSON.stringify(obj, null, 2) with the third argument (2) adds 2-space indentation. Use this when you want to display JSON in a readable format.

Topic 7: Browser Storage Interview Questions with Solutions

Q1: What is the difference between localStorage and sessionStorage?

localStorage is permanent — data stays even after browser is closed, and is shared across all tabs of the same origin.

sessionStorage is temporary — data is deleted when the tab is closed, and is unique to each tab. Two tabs cannot share sessionStorage data.

```
// localStorage — survives browser close:
localStorage.setItem("theme", "dark");

// sessionStorage — deleted on tab close, tab-specific:
sessionStorage.setItem("page", "3");
```

Q2: What is the difference between localStorage and Cookies?

localStorage — 5MB storage, never sent to server, no expiry, only accessible in browser.

Cookies — only 4KB, automatically sent to server with every request, has expiry date, can be set by server.

✅Remember: If the interviewer asks 'where do you store authentication tokens?', the best answer is HttpOnly cookies — because JavaScript cannot access them, making them safer than localStorage against XSS attacks.

Q3: Why do we need JSON.stringify() when using localStorage?

Because localStorage can only store strings. If you try to store an object directly, JavaScript calls .toString() on it which gives '[object Object]' — which is useless.

```
let user = { name: "Shaaz" };

localStorage.setItem("user", user);
console.log(localStorage.getItem("user")); // Output: [object Object] WRONG!

localStorage.setItem("user", JSON.stringify(user));
let back = JSON.parse(localStorage.getItem("user"));
console.log(back.name); // Output: Shaaz  CORRECT!
```

Q4: How do you safely read from localStorage when value might not exist?

```
function getSafe(key, defaultValue = null) {
  let item = localStorage.getItem(key);
  if (item === null) return defaultValue;
  try {
    return JSON.parse(item);
  } catch {
    return item; // return as plain string if not JSON
  }
}

console.log(getSafe("theme", "light")); // Returns "light" if not set
```

Q5: What is an HttpOnly cookie and why is it important?

An HttpOnly cookie is a cookie with the HttpOnly flag set by the server. This flag prevents JavaScript from reading the cookie using document.cookie. Only the browser and server can use it.

Why important? Protects against XSS (Cross-Site Scripting) attacks. If a hacker injects malicious JavaScript into your page, that script CANNOT steal HttpOnly cookies. This is why auth tokens should be stored in HttpOnly cookies, not localStorage.

Q6: Can sessionStorage be shared between two tabs?

No. Each browser tab has its own completely separate sessionStorage. If you open the same website in two different tabs and save something in sessionStorage in Tab 1, Tab 2 cannot access it.

But localStorage IS shared — all tabs of the same origin can read and write to the same localStorage.

Q7: Write a helper function to save and load objects from localStorage

```
const storage = {
  save(key, value) {
    localStorage.setItem(key, JSON.stringify(value));
  },
  load(key) {
    let item = localStorage.getItem(key);
    return item ? JSON.parse(item) : null;
  },
  remove(key) {
    localStorage.removeItem(key);
  }
};

storage.save("user", { name: "Shaaz", age: 23 });
console.log(storage.load("user")); // Output: { name: "Shaaz", age: 23 }
storage.remove("user");
```

Q8: What happens if localStorage is full?

When localStorage reaches its limit (about 5MB), the browser throws a QuotaExceededError. You should always wrap localStorage operations in try-catch to handle this gracefully.

```
try {
  localStorage.setItem("bigData", someHugeString);
} catch (e) {
  if (e.name === "QuotaExceededError") {
    console.log("Storage is full! Cannot save data.");
  }
}
```

Topic 8: Quick Reference — All Storage Methods at a Glance

localStorage.setItem(key, value) — save a string value

localStorage.getItem(key) — read a value (returns null if not found)

localStorage.removeItem(key) — delete one item

localStorage.clear() — delete everything

localStorage.key(index) — get key name by index

localStorage.length — total number of items stored

sessionStorage — exact same API as localStorage

document.cookie — read all cookies as one string

document.cookie = "key=value" — set a cookie

JSON.stringify(obj) — convert object to JSON string for storing

JSON.parse(string) — convert JSON string back to object

> **✅Remember: The most asked interview topics here: localStorage vs sessionStorage vs cookies differences, why JSON.stringify is needed, what HttpOnly cookie is, and the security comparison of storing tokens in localStorage vs HttpOnly cookies.**

Chapter 14: Object-Oriented Programming (OOP)

How to Define in an Interview

"Object-Oriented Programming (OOP) is a programming style where we organise code around objects rather than functions. An object bundles together related data (properties) and behaviour (methods). OOP has 4 main pillars: Encapsulation (hiding private data), Inheritance (child class gets parent's features), Polymorphism (same method, different behaviour), and Abstraction (hide complex details, show only what is needed). In JavaScript, OOP is implemented using Classes, Constructors, Prototypes, and the extends keyword."

Topic 1: What is OOP? The Story That Makes It Click Forever

Imagine you are the manager of a car manufacturing company like Maruti.

Every car you make has:

Properties — brand, color, speed, fuel, seats (DATA about the car)

Actions — start(), stop(), accelerate(), brake() (what the car can DO)

Now you need to make 10,000 cars. Do you describe each car from scratch? NO. You create ONE BLUEPRINT (a class). Then you use that blueprint to stamp out as many cars (objects) as you want. Each car has its own color, speed, fuel — but they all share the same structure from the blueprint.

Class = the blueprint / design / mould

Object = the actual car made from that blueprint (also called an instance)

new = the machine that stamps out a car from the blueprint

```
// Blueprint (Class):
class Car {
  constructor(brand, color, speed) {
    this.brand = brand;
    this.color = color;
    this.speed = speed;
  }
  start()      { console.log(`${this.brand} started!`); }
  accelerate() { this.speed += 10; console.log(`Speed: ${this.speed}`); }
}

// Stamping out actual cars (objects / instances):
let car1 = new Car("Maruti",  "Red",  80);
let car2 = new Car("Honda",   "Blue", 100);
let car3 = new Car("Toyota",  "White", 120);

car1.start();      // Output: Maruti started!
car2.accelerate(); // Output: Speed: 110
car3.start();      // Output: Toyota started!
```

✅Remember: Class = blueprint. Object = thing made from blueprint. new = creates the thing. this = the specific thing being created or used.

Topic 2: Class and Constructor — The Foundation

A class is written using the 'class' keyword. Inside it, the constructor() is a special function that runs AUTOMATICALLY every time you create a new object using 'new'.

Think of constructor like the assembly line worker who sets up each new car when it comes off the line — puts in the engine, paints it, adds the seats.

```
class Student {
  constructor(name, age, marks) {  // runs automatically with new
    this.name  = name;             // set up this student
    this.age   = age;
    this.marks = marks;
  }

  // Methods — actions the student can do:
  introduce() {
    console.log(`Hi, I am ${this.name}, age ${this.age}`);
  }

  getGrade() {
    if (this.marks >= 90) return "A";
    if (this.marks >= 75) return "B";
    if (this.marks >= 60) return "C";
    return "F";
  }

  study(hours) {
    this.marks += hours * 2;
    console.log(`${this.name} studied ${hours} hrs. Marks now: ${this.marks}`);
  }
}

let s1 = new Student("Shaaz", 23, 80);
let s2 = new Student("Aman",  25, 92);

s1.introduce();   // Output: Hi, I am Shaaz, age 23
s1.study(5);      // Output: Shaaz studied 5 hrs. Marks now: 90
console.log(s1.getGrade()); // Output: A

s2.introduce();   // Output: Hi, I am Aman, age 25
console.log(s2.getGrade()); // Output: A
```

Tip: Each object (s1, s2) has its own separate data (name, marks). But the methods (introduce, getGrade, study) are shared — they live on the prototype, not duplicated for each object.

Topic 3: The 4 Pillars of OOP — Must Know All Four

These 4 are the most asked OOP concepts in interviews. Understand each one deeply with a real-world analogy.

Pillar 1 — Encapsulation: Hide What Should Be Private

Encapsulation means bundling data and methods together AND protecting private data so outside code cannot directly read or change it.

Real life: ATM machine. You can deposit, withdraw, check balance. But you CANNOT directly open the ATM and take money — it is protected. You can only interact through the allowed methods.

Without Encapsulation — DANGEROUS:

```
let bankBalance = 5000;
bankBalance = -99999;  // Anyone can directly change it — DANGEROUS!
```

With Encapsulation — SAFE:

```
class BankAccount {
  #balance;            // # makes it PRIVATE — cannot access outside

  constructor(owner, initialBalance) {
    this.owner = owner;
    this.#balance = initialBalance;
  }

  deposit(amount) {
    if (amount <= 0) { console.log("Invalid amount!"); return; }
    this.#balance += amount;
    console.log(`Deposited ${amount}. Balance: ${this.#balance}`);
  }

  withdraw(amount) {
    if (amount > this.#balance) { console.log("Not enough balance!"); return; }
    this.#balance -= amount;
    console.log(`Withdrew ${amount}. Balance: ${this.#balance}`);
  }

  getBalance() { return this.#balance; }
}

let acc = new BankAccount("Shaaz", 5000);
acc.deposit(2000);      // Output: Deposited 2000. Balance: 7000
acc.withdraw(1000);     // Output: Withdrew 1000. Balance: 6000
console.log(acc.getBalance()); // Output: 6000

// Try to access directly — BLOCKED:
console.log(acc.#balance); // SyntaxError! Private field
acc.#balance = 999999;     // SyntaxError! Cannot set private field
```

✅Remember: Encapsulation = bundle data + methods together, make private data inaccessible from outside. In modern JS, use # for private fields. In older code, closures were used.

Pillar 2 — Inheritance: Child Gets Parent's Features

Inheritance means a child class automatically gets all the properties and methods of its parent class — WITHOUT rewriting them. The child can also add its own new features on top.

Real life: You inherit your parents' features — their eyes, height, surname. But you also have your own unique personality, skills, and choices that are only yours.

```
// PARENT CLASS — general features
class Animal {
  constructor(name, sound) {
    this.name  = name;
    this.sound = sound;
    this.alive = true;
  }

  breathe() { console.log(`${this.name} is breathing...`); }
  eat(food) { console.log(`${this.name} is eating ${food}`); }
  speak()   { console.log(`${this.name} says: ${this.sound}`); }
}
```

```
// CHILD CLASS — gets everything from Animal + adds its own stuff
class Dog extends Animal {
  constructor(name, breed) {
    super(name, "Woof!");  // call parent constructor — MUST be first!
    this.breed = breed;    // Dog-specific property
  }

  fetch(item) { console.log(`${this.name} fetched the ${item}!`); }
  wagTail()   { console.log(`${this.name} is wagging its tail!`); }
}

class Cat extends Animal {
  constructor(name, indoor) {
    super(name, "Meow!");
    this.indoor = indoor;
  }
  purr() { console.log(`${this.name} is purring...`); }
}

let dog = new Dog("Bruno", "Labrador");
let cat = new Cat("Whiskers", true);

dog.breathe();       // From Animal: Bruno is breathing...
dog.eat("bone");     // From Animal: Bruno is eating bone
dog.speak();         // From Animal: Bruno says: Woof!
dog.fetch("ball");   // Dog-only: Bruno fetched the ball!
dog.wagTail();       // Dog-only: Bruno is wagging its tail!

cat.breathe();       // From Animal: Whiskers is breathing...
cat.speak();         // From Animal: Whiskers says: Meow!
cat.purr();          // Cat-only: Whiskers is purring...
```

✅Remember: extends = inherit from parent. super() = call parent's constructor — always first line in child constructor. Child gets ALL parent methods automatically. Child adds its own methods on top.

Pillar 3 — Polymorphism: Same Method, Different Behaviour

Polymorphism means 'many forms'. The same method name behaves DIFFERENTLY depending on which class is using it.

Real life: 'speak' means something different for a dog (woof), a cat (meow), a cow (moo), and a human (hello). Same action name, totally different result based on who is doing it.

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name} makes a sound`); } // default
}

class Dog extends Animal {
  speak() { console.log(`${this.name} says: Woof woof!`); } // override
}

class Cat extends Animal {
  speak() { console.log(`${this.name} says: Meow meow!`); } // override
}
```

```
class Cow extends Animal {
  speak() { console.log(`${this.name} says: Moooo!`); }    // override
}

class Fish extends Animal {
  // No speak() — uses parent default
}

// The POWER of polymorphism — same loop, different behaviour:
let animals = [
  new Dog("Bruno"),
  new Cat("Whiskers"),
  new Cow("Goru"),
  new Fish("Nemo")
];

animals.forEach(animal => animal.speak());
// Output:
// Bruno says: Woof woof!
// Whiskers says: Meow meow!
// Goru says: Moooo!
// Nemo makes a sound  (no override — uses parent)
```

This is the real power of polymorphism — you write one loop, call one method, but each object does it in its own way automatically.

> **✅Remember: Polymorphism = same method name, different behaviour per class. Child class overrides parent's method. This is also called Method Overriding.**

Pillar 4 — Abstraction: Hide Complexity, Show Only What Matters

Abstraction means hiding the internal complex details and only exposing what the user needs to know.

Real life: When you drive a car, you use the steering wheel, accelerator, and brakes. You do NOT need to know how the engine combustion works, how the fuel injection system operates, or how the transmission shifts gears. All that complexity is HIDDEN (abstracted). You see only what you need.

```
class CoffeeMachine {
  // Private methods — complex internal steps (hidden from user)
  #heatWater()    { console.log("Heating water to 95°C..."); }
  #grindBeans()   { console.log("Grinding coffee beans..."); }
  #brew()         { console.log("Brewing coffee..."); }
  #fillCup()      { console.log("Filling cup..."); }

  // Public method — simple interface shown to user
  makeCoffee(type) {
    console.log(`Making ${type} coffee...`);
    this.#heatWater();  // user does not care about these steps
    this.#grindBeans();
    this.#brew();
    this.#fillCup();
    console.log(`Your ${type} coffee is ready! Enjoy!`);
  }
}

let machine = new CoffeeMachine();
```

```
machine.makeCoffee("Latte");  // user just calls this one method
// Output:
// Making Latte coffee...
// Heating water to 95 degrees...
// Grinding coffee beans...
// Brewing coffee...
// Filling cup...
// Your Latte coffee is ready! Enjoy!

// User cannot call internal steps directly:
machine.#heatWater(); // SyntaxError! Private method
```

✅Remember: Abstraction = hide HOW it works, show WHAT it does. User sees simple interface. Complexity is hidden inside private methods.

Topic 4: Prototypes — How JavaScript OOP Actually Works Under the Hood

This is what makes JavaScript unique. Unlike Java or C++, JavaScript OOP is built on PROTOTYPES — not traditional classes. The class syntax is just cleaner writing on top of prototypes.

Every object in JavaScript has a hidden link to another object called its PROTOTYPE. When you call a method on an object and that method is not found on the object itself, JavaScript automatically looks up the prototype chain.

Think of it like this: You ask your friend 'Do you have a pen?' He says no, so you ask his parent. Parent says no, so you ask the grandparent. This chain of asking upwards is the prototype chain.

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(this.name + " speaks"); }
}

let dog = new Animal("Bruno");

// speak() is NOT stored on the dog object itself
// It is stored on Animal.prototype
// When dog.speak() is called, JS looks: dog → Animal.prototype → found!

console.log(dog.hasOwnProperty("name"));   // true — name is ON the object
console.log(dog.hasOwnProperty("speak"));  // false — speak is on prototype

// Checking the prototype chain:
console.log(Object.getPrototypeOf(dog) === Animal.prototype); // true
```

Note: Every class method is stored ONCE on the prototype — not copied into every object. This saves memory. 1000 dogs share one copy of speak() on the prototype.

Prototype Chain in Action

```
class LivingThing {
  breathe() { console.log("Breathing..."); }
}

class Animal extends LivingThing {
  eat() { console.log("Eating..."); }
}
```

```
class Dog extends Animal {
  bark() { console.log("Woof!"); }
}

let d = new Dog();
d.bark();    // Found on Dog.prototype
d.eat();     // Not on Dog → go up → Found on Animal.prototype
d.breathe(); // Not on Animal → go up → Found on LivingThing.prototype
d.toString(); // Not found anywhere above → go up → Found on Object.prototype
```

The chain: d → Dog.prototype → Animal.prototype → LivingThing.prototype → Object.prototype → null

Topic 5: static Methods — Class Level, Not Object Level

Normal methods are called on objects (instances). Static methods are called on the CLASS itself — not on any specific object. They are utility functions that belong to the class as a whole, not to any individual instance.

Real life: A factory (class) has machines that make cars (instances). The machines belong to individual cars. But the factory's address, working hours, total production count — these belong to the FACTORY itself, not to any specific car.

```
class MathHelper {
  static add(a, b)      { return a + b; }
  static multiply(a, b) { return a * b; }
  static square(n)      { return n * n; }
  static isEven(n)      { return n % 2 === 0; }
}

// Call on the CLASS — not on an object:
console.log(MathHelper.add(5, 3));       // Output: 8
console.log(MathHelper.square(4));       // Output: 16
console.log(MathHelper.isEven(7));       // Output: false

// Cannot call on instance:
let m = new MathHelper();
m.add(1, 2); // TypeError! add is not a function on instances

// Real world example — tracking total instances created:
class User {
  static count = 0;           // static property — shared by all

  constructor(name) {
    this.name = name;
    User.count++;             // increment class-level counter
  }

  static getTotalUsers() {    // static method — called on class
    return User.count;
  }
}

let u1 = new User("Shaaz");
let u2 = new User("Aman");
let u3 = new User("Riya");
```

```
console.log(User.getTotalUsers()); // Output: 3
```

✅Remember: static = belongs to the CLASS, not the object. Call it as ClassName.method(), not object.method(). Use for utilities and shared data.

Topic 6: Getters and Setters — Controlled Property Access

Getters and Setters let you access and set properties like normal variables, but with hidden logic running behind the scenes. You can validate, compute, or protect data.

Real life: A weighing scale (getter) shows your weight, but it calculates it internally. You see a simple number, but complex electronics are running behind.

```
class Circle {
  constructor(radius) {
    this._radius = radius;  // _ is a convention for "intended private"
  }

  // Getter — access like a property, but runs code
  get radius() {
    return this._radius;
  }

  // Setter — validate before allowing change
  set radius(value) {
    if (value <= 0) {
      console.log("Radius must be positive!");
      return;
    }
    this._radius = value;
  }

  // Computed getter — calculates on the fly
  get area() {
    return Math.PI * this._radius ** 2;
  }

  get circumference() {
    return 2 * Math.PI * this._radius;
  }
}

let c = new Circle(5);

// Access like a property — no () needed:
console.log(c.radius);        // Output: 5
console.log(c.area.toFixed(2));    // Output: 78.54
console.log(c.circumference.toFixed(2)); // Output: 31.42

// Set like a property — triggers the setter:
c.radius = 10;
console.log(c.area.toFixed(2)); // Output: 314.16

c.radius = -5; // Output: Radius must be positive!
console.log(c.radius); // Output: 10  (unchanged — setter blocked it)
```

Tip: Getters and Setters look like properties when you use them, but they run code behind the scenes. Perfect for validation and computed values.

Topic 7: Complete Real World Example — School Management System

Let us put everything together in one real example that uses classes, inheritance, encapsulation, and polymorphism together.

```
// Base class
class Person {
  #id;
  constructor(id, name, age) {
    this.#id  = id;
    this.name = name;
    this.age  = age;
  }
  getId()      { return this.#id; }
  introduce()  { console.log(`I am ${this.name}, age ${this.age}`); }
}

// Child class — Teacher
class Teacher extends Person {
  constructor(id, name, age, subject) {
    super(id, name, age);
    this.subject = subject;
  }
  introduce() {  // polymorphism — override
    console.log(`I am ${this.name}, I teach ${this.subject}`);
  }
  teach() { console.log(`${this.name} is teaching ${this.subject}...`); }
}

// Child class — Student
class Student extends Person {
  #marks = 0;
  constructor(id, name, age, grade) {
    super(id, name, age);
    this.grade = grade;
  }
  addMarks(m)  { this.#marks += m; }
  getMarks()   { return this.#marks; }
  introduce()  { // polymorphism — override
    console.log(`I am ${this.name}, grade ${this.grade} student`);
  }
  study(subject) { console.log(`${this.name} is studying ${subject}`); }
}

let teacher = new Teacher(1, "Mr. Sharma", 40, "Math");
let student = new Student(2, "Shaaz",      23, "12th");

// Polymorphism — same method name, different output:
teacher.introduce(); // I am Mr. Sharma, I teach Math
student.introduce(); // I am Shaaz, grade 12th student

teacher.teach();     // Mr. Sharma is teaching Math...
```

```
student.study("Physics"); // Shaaz is studying Physics

student.addMarks(85);
console.log(student.getMarks()); // 85
// console.log(student.#marks);  // SyntaxError — private!
```

Topic 8: OOP Interview Questions with Solutions

Q1: What are the 4 pillars of OOP? Explain each in one line.

Encapsulation — bundle data + methods, hide private data from outside.

Inheritance — child class gets all features of parent class using extends.

Polymorphism — same method name, different behaviour in different classes.

Abstraction — hide complex internal details, expose only simple interface.

Q2: What is the difference between a class and an object?

Class = blueprint / template / design. It does not exist in real memory as a usable thing.

Object = actual instance created from the class using 'new'. It exists in memory with real values.

```
class Car { ... }        // blueprint — no real car exists yet
let c = new Car("BMW"); // object — a real car now exists in memory
```

Q3: What does 'super' do?

super() calls the parent class's constructor. It must be the first line inside a child class constructor. Without calling super(), 'this' is not available in the child constructor and you get an error.

super.methodName() can also be used to call a parent method from a child method.

```
class Dog extends Animal {
  constructor(name, breed) {
    super(name);   // MUST be first — calls Animal constructor
    this.breed = breed; // then add child-specific stuff
  }
  speak() {
    super.speak(); // call parent speak() first
    console.log("...and also barks!");
  }
}
```

Q4: What is the difference between static and instance methods?

Instance method — belongs to a specific object. Called as object.method(). Has access to 'this' (the object).

Static method — belongs to the class itself. Called as ClassName.method(). Does NOT have access to instance 'this'.

```
class Counter {
  static total = 0;
  constructor() { Counter.total++; this.id = Counter.total; }
  getId()                { return this.id; }       // instance method
  static getTotal()      { return Counter.total; } // static method
}
let c1 = new Counter(); let c2 = new Counter();
console.log(c1.getId());        // 1 — instance method
console.log(Counter.getTotal()); // 2 — static method
```

Q5: What is prototype in JavaScript?

Every JavaScript object has a hidden link to another object called its prototype. When you access a property or method that does not exist on the object, JavaScript automatically looks up the prototype chain until it finds it or reaches null. Class methods are stored on the prototype — not copied into every object — saving memory.

Q6: What is method overriding?

Method overriding is when a child class defines a method with the same name as a parent class method. The child's version replaces (overrides) the parent's version for that child. This is how polymorphism works in JavaScript.

```
class Animal { speak() { console.log("Some sound"); } }
class Dog    extends Animal { speak() { console.log("Woof!"); } }
class Cat    extends Animal { speak() { console.log("Meow!"); } }
new Dog().speak(); // Woof! — overridden
new Cat().speak(); // Meow! — overridden
new Animal().speak(); // Some sound — original
```

Q7: What is the difference between encapsulation and abstraction?

Encapsulation — about DATA protection. Bundling data + methods and making private data inaccessible. Focus: security of data.

Abstraction — about COMPLEXITY hiding. Showing simple interface, hiding complex implementation. Focus: simplicity for the user.

Example: A coffee machine — the ON button is abstraction (simple interface). The locked internal wiring is encapsulation (private, protected).

Q8: Can JavaScript classes have private fields?

Yes — using the # prefix (ES2022). Private fields with # can only be accessed inside the class. They throw a SyntaxError if accessed from outside.

```
class Secret {
  #pin = 1234;                  // private field
  checkPin(input) { return input === this.#pin; }
}
let s = new Secret();
console.log(s.checkPin(1234)); // true
console.log(s.#pin);           // SyntaxError!
```

Q9: What is the instanceof operator?

instanceof checks if an object was created from a specific class. It also returns true for parent classes in the inheritance chain.

```
class Animal {}
class Dog extends Animal {}
let d = new Dog();
console.log(d instanceof Dog);    // true
console.log(d instanceof Animal); // true  (Dog extends Animal)
console.log(d instanceof Object); // true  (everything is an Object)
```

Q10: What is the difference between Object.create() and new?

new ClassName() — calls the constructor function, sets up the prototype chain, returns the new object.

Object.create(proto) — creates a new empty object with the specified object as its prototype. Does NOT call any constructor.

```
const animalProto = {
  speak() { console.log(this.name + " speaks"); }
};

let dog = Object.create(animalProto); // dog inherits from animalProto
dog.name = "Bruno";
dog.speak(); // Output: Bruno speaks

console.log(Object.getPrototypeOf(dog) === animalProto); // true
```

Topic 9: Quick Reference — OOP at a Glance

class — blueprint / template
constructor() — auto-runs when new is called, sets up the object
new ClassName() — creates a new object (instance) from the class
this — refers to the current object inside the class
extends — child class inherits from parent class
super() — calls parent constructor — MUST be first in child constructor
super.method() — calls parent method from child
static — belongs to class, not instance — call as ClassName.method()
#field — private field — only accessible inside the class
get property() — getter — access like property, runs code
set property(v) — setter — validate/process before setting
instanceof — checks if object is an instance of a class
Encapsulation — data + method bundled, private fields protected
Inheritance — child gets parent features via extends
Polymorphism — same method name, different behaviour per class
Abstraction — hide complexity, show simple interface

> **✅Remember: The story to never forget: Class = car blueprint. Object = actual car. new = factory machine. constructor = assembly line worker. extends = child model inherits parent. super = calling the parent factory. static = belongs to the factory, not the car. # = locked compartment nobody can open from outside.**

Chapter 15: Advanced JavaScript Concepts

This chapter covers powerful modern JavaScript features that are used every day in real projects and come up constantly in interviews.

Topic 1: Destructuring — Unpack Values Easily

Destructuring means pulling out values from arrays or objects and putting them into separate variables — in ONE clean line instead of many lines.

Think of it like unpacking a suitcase: instead of taking items one by one with suitcase.shirt, suitcase.pants, suitcase.shoes, you unpack everything at once.

Array Destructuring

```
// Old way — painful:
let colors = ["red", "green", "blue"];
let c1 = colors[0];
let c2 = colors[1];
let c3 = colors[2];

// New way — destructuring:
let [c1, c2, c3] = ["red", "green", "blue"];
console.log(c1); // Output: red
console.log(c2); // Output: green

// Skip values with commas:
let [first, , third] = [10, 20, 30];
console.log(first); // 10
console.log(third); // 30  (20 was skipped)

// Default values — if element does not exist:
let [a = 5, b = 7] = [1];
console.log(a); // 1  (from array)
console.log(b); // 7  (default — nothing at index 1)

// Swap two variables — cleanest trick!
let x = 10, y = 20;
[x, y] = [y, x];
console.log(x, y); // Output: 20 10

// Rest in destructuring:
let [head, ...tail] = [1, 2, 3, 4, 5];
console.log(head); // 1
console.log(tail); // [2, 3, 4, 5]
```

Object Destructuring

```
// Old way:
let person = { name: "Shaaz", age: 23, city: "Delhi" };
let name = person.name;
let age  = person.age;

// New way — destructuring:
let { name, age, city } = person;
```

```
console.log(name); // Shaaz
console.log(age);  // 23

// Rename while destructuring:
let { name: personName, age: personAge } = person;
console.log(personName); // Shaaz
console.log(personAge);  // 23

// Default values:
let { name, country = "India" } = { name: "Shaaz" };
console.log(country); // India  (default — not in object)

// In function parameters — very common in React!
function greet({ name, age }) {
  console.log(`Hello ${name}, you are ${age} years old`);
}
greet({ name: "Shaaz", age: 23, city: "Delhi" });
// Output: Hello Shaaz, you are 23 years old

// Nested destructuring:
let { address: { city, pincode } } = {
  name: "Shaaz",
  address: { city: "Delhi", pincode: "110001" }
};
console.log(city);    // Delhi
console.log(pincode); // 110001
```

✅Remember: Destructuring is used everywhere in modern JavaScript — especially in React props, API responses, and function parameters. Master it!

Topic 2: Spread (...) and Rest (...) — Same Symbol, Different Jobs

Both use ... (three dots) but they do OPPOSITE things depending on where you use them.

Spread — EXPANDS a collection into individual items (array/object → separate items)

Rest — COLLECTS individual items into a collection (separate items → array)

Easy trick: Spread = spill out. Rest = collect the rest.

Spread with Arrays

```
let fruits  = ["apple", "banana"];
let veggies = ["carrot", "potato"];

// Combine arrays:
let food = [...fruits, ...veggies];
console.log(food); // ["apple", "banana", "carrot", "potato"]

// Copy an array (real copy — NOT reference):
let original = [1, 2, 3];
let copy = [...original];
copy.push(4);
console.log(original); // [1, 2, 3]  — unchanged!
console.log(copy);     // [1, 2, 3, 4]

// Pass array as separate arguments:
```

```
let nums = [3, 1, 4, 1, 5, 9];
console.log(Math.max(...nums)); // Output: 9
// Math.max needs: Math.max(3,1,4,1,5,9) — spread does this!
```

Spread with Objects

```
let person = { name: "Shaaz", age: 23 };
let extra  = { city: "Delhi", job: "Dev" };

// Merge objects:
let full = { ...person, ...extra };
console.log(full);
// { name:"Shaaz", age:23, city:"Delhi", job:"Dev" }

// Copy object (shallow):
let copy = { ...person };
copy.name = "Aman";
console.log(person.name); // Shaaz — original unchanged

// Override specific property:
let updated = { ...person, age: 25 };
console.log(updated); // { name:"Shaaz", age:25 }
```

Rest in Functions — Collect Extra Arguments

```
// Collect all arguments into an array:
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3));        // 6
console.log(sum(10, 20, 30, 40)); // 100
console.log(sum(5));              // 5

// Mix normal and rest:
function introduce(firstName, lastName, ...hobbies) {
  console.log(`${firstName} ${lastName}`);
  console.log("Hobbies:", hobbies);
}
introduce("Shaaz", "Akhtar", "coding", "gaming", "reading");
// Output: Shaaz Akhtar
// Output: Hobbies: ["coding", "gaming", "reading"]
```

Note: Rest parameter must ALWAYS be the LAST parameter in a function. You cannot have anything after it.

Topic 3: Short-Circuit Evaluation — Smart Boolean Tricks

JavaScript is lazy with boolean expressions. It stops evaluating as soon as it knows the answer. This is called short-circuit evaluation.

&& (AND) — Stops at first FALSY value

If left side is falsy — return it immediately (don't even look at right side).

If left side is truthy — return the right side.

```
console.log(false && "Hello");  // false  (stopped at false)
console.log(0     && "Hello");  // 0      (stopped at 0 — falsy)
console.log(true  && "Hello");  // Hello  (left is truthy, return right)
```

```
console.log("Hi"  && "Hello");  // Hello  (left is truthy, return right)

// Real use — only run code if condition is true:
let isLoggedIn = true;
isLoggedIn && console.log("Welcome back!"); // runs

let user = null;
user && console.log(user.name); // does NOT run — user is falsy — no crash!
```

|| (OR) — Stops at first TRUTHY value

If left side is truthy — return it immediately.

If left side is falsy — return the right side.

```
console.log(false || "Hello");  // Hello  (left is falsy, return right)
console.log(0     || "Hello");  // Hello  (0 is falsy)
console.log(true  || "Hello");  // true   (left is truthy, return it)
console.log("Hi"  || "Hello");  // Hi     (left is truthy, return it)

// Real use — default values:
let username = "";
let display  = username || "Guest";
console.log(display); // Guest  (username is empty string = falsy)
```

?? (Nullish Coalescing) — Only for null and undefined

?? is like || BUT it only checks for null and undefined — NOT other falsy values like 0 or empty string. This is very important!

```
console.log(null      ?? "Default"); // Default  (null → use right)
console.log(undefined ?? "Default"); // Default  (undefined → use right)
console.log(0         ?? "Default"); // 0        (0 is NOT null/undefined)
console.log("")       ?? "Default"); // ""       ("" is NOT null/undefined)
console.log(false     ?? "Default"); // false    (false is NOT null/undefined)

// The KEY difference — || vs ??:
let score = 0;
console.log(score || 100);  // 100  (WRONG! 0 is falsy — overwritten)
console.log(score ?? 100);  // 0    (CORRECT! 0 is valid — kept)
```

✅Remember: Use || when any falsy value (0, '', false) should trigger default. Use ?? when ONLY null or undefined should trigger default. Score=0 is valid — use ??.

Optional Chaining (?.) — Safe Deep Access

?. lets you safely access nested properties without crashing if something in the chain is null or undefined. It returns undefined instead of throwing an error.

```
let user = {
  name: "Shaaz",
  address: {
    city: "Delhi"
  }
};

// Without ?. — CRASH if address is missing:
console.log(user.address.city);    // Delhi ✓
console.log(user.phone.number);    // TypeError: Cannot read property of undefined ✗
```

```
// With ?. — SAFE:
console.log(user.address?.city);   // Delhi ✓
console.log(user.phone?.number);   // undefined ✓ (no crash!)
console.log(user.address?.pincode?.code); // undefined ✓ (deep chain)

// Combine with ?? for default:
let city = user.address?.city ?? "Unknown";
console.log(city); // Delhi
```

Tip: Optional chaining (?.) is extremely useful when working with API responses where some fields might be missing. Always use it for nested access on potentially null data.

Topic 4: Template Literals — String Power

Template literals use backticks (`) instead of quotes. They let you embed variables and expressions directly inside strings — no more messy + signs.

```
let name = "Shaaz";
let age  = 23;

// Old way — ugly string concatenation:
let msg1 = "Hello, my name is " + name + " and I am " + age + " years old.";

// New way — template literal:
let msg2 = `Hello, my name is ${name} and I am ${age} years old.`;
console.log(msg2); // Hello, my name is Shaaz and I am 23 years old.

// Expressions inside ${}:
let a = 5, b = 3;
console.log(`Sum is ${a + b}`);          // Sum is 8
console.log(`Is adult: ${age >= 18}`);   // Is adult: true
console.log(`Upper: ${name.toUpperCase()}`); // Upper: SHAAZ

// Multi-line strings — no \n needed:
let poem = `
  Roses are red,
  Violets are blue,
  JavaScript is awesome,
  And so are you!
`;
console.log(poem);
```

Topic 5: Symbol — Unique Keys That Never Clash

Symbol is a primitive data type that creates a completely unique value every single time. Even if two Symbols have the same description, they are never equal to each other.

Real life: Like fingerprints — two people can have the same name but different fingerprints. Symbols are like fingerprints for property keys.

```
let id1 = Symbol("id");
let id2 = Symbol("id");

console.log(id1 === id2); // false — always unique!
console.log(typeof id1);  // symbol

// Use as object key — never conflicts with other keys:
```

```
let ID = Symbol("userID");
let user = {
  name: "Shaaz",
  [ID]: 12345       // Symbol as key using []
};

console.log(user[ID]);   // 12345
console.log(user.ID);    // undefined — must use [] not dot

// Symbol keys are hidden in normal loops:
for (let key in user) console.log(key); // name  (ID is hidden)
Object.keys(user); // ["name"]           (ID is hidden)
```

Tip: Symbols are used in libraries to add internal properties to objects without risking name conflicts with user code. Also used for well-known symbols like Symbol.iterator.

Topic 6: Generators — Pause and Resume Functions

A generator is a special function that can PAUSE its execution in the middle and RESUME later from exactly where it stopped. Normal functions run from start to finish without pausing. Generators can pause.

Think of a generator like a vending machine: you press the button (next()), it gives you one item and waits. Press again — another item. It does not dump everything at once.

```
// Generator function — uses function* and yield
function* countUp() {
  console.log("Start");
  yield 1;            // pause here, give 1
  console.log("After 1");
  yield 2;            // pause here, give 2
  console.log("After 2");
  yield 3;            // pause here, give 3
  console.log("Done");
}

let gen = countUp();  // does NOT run yet — just creates generator

console.log(gen.next()); // Start  → { value: 1, done: false }
console.log(gen.next()); // After 1 → { value: 2, done: false }
console.log(gen.next()); // After 2 → { value: 3, done: false }
console.log(gen.next()); // Done   → { value: undefined, done: true }

// Infinite generator — generates unique IDs:
function* idMaker() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

let makeId = idMaker();
console.log(makeId.next().value); // 1
console.log(makeId.next().value); // 2
console.log(makeId.next().value); // 3  — goes on forever
```

Topic 7: WeakMap and WeakSet — Memory-Friendly Collections

Regular Map and Set hold strong references to their keys — if the object still exists in Map/Set, it cannot be garbage collected even if nothing else uses it. WeakMap and WeakSet hold WEAK references — if nothing else references the object, it can be garbage collected even if it is in the WeakMap.

WeakMap

Keys must be objects (not strings or numbers). Values can be anything. No iteration (no forEach, no size). Garbage-collection-friendly.

```
let weakMap = new WeakMap();

let user = { name: "Shaaz" };
weakMap.set(user, { visits: 5, lastSeen: "today" });

console.log(weakMap.get(user)); // { visits: 5, lastSeen: "today" }
console.log(weakMap.has(user)); // true

user = null; // user object is no longer referenced
// Now the WeakMap entry is automatically cleaned up — no memory leak!
```

Tip: WeakMap is perfect for storing private data associated with DOM elements or objects without worrying about memory leaks when those objects are removed.

Topic 8: Advanced Concepts Interview Questions

Q1: What is the difference between spread and rest?

Same ... symbol, opposite purposes. Spread expands an array or object into individual items. Rest collects individual items into an array. Spread is used when calling functions or building arrays/objects. Rest is used in function parameters.

```
// Spread — expand:
let arr = [1,2,3];
console.log(Math.max(...arr));    // 3

// Rest — collect:
function sum(...nums) { return nums.reduce((a,b) => a+b, 0); }
console.log(sum(1,2,3,4));        // 10
```

Q2: What is the difference between || and ??

|| returns the right side if left is ANY falsy value (0, '', false, null, undefined).

?? returns the right side ONLY if left is null or undefined. 0 and '' are kept.

```
let score = 0;
console.log(score || 100); // 100 — WRONG for score
console.log(score ?? 100); // 0   — CORRECT for score
```

Q3: What does optional chaining (?.) do?

It safely accesses nested properties. If any part of the chain is null or undefined, it returns undefined instead of throwing a TypeError.

```
let user = null;
console.log(user?.name);         // undefined (no crash)
console.log(user?.address?.city); // undefined (no crash)
```

Q4: What is destructuring?

Destructuring is a syntax that lets you unpack values from arrays or properties from objects into separate variables in one line. It makes code shorter and more readable.

Q5: What is a Generator function?

A generator function (function*) can pause its execution using yield and resume later using .next(). Each call to .next() runs until the next yield, returns the yielded value wrapped in { value, done }, and pauses again. When all yields are done, done becomes true.

Chapter 16: Must-Know Modern JavaScript

> *These are the topics that are asked in almost every modern JavaScript interview in 2024-2025. Some developers skip these — but interviewers love them. Master these and you will stand out.*

Topic 1: Microtask Queue vs Macrotask Queue — The Most Tricky Interview Question

You already know the Event Loop. But there is a deeper layer. JavaScript has TWO separate queues — not one.

Macrotask Queue (also called Task Queue) — holds: setTimeout, setInterval, setImmediate, DOM events, fetch callbacks

Microtask Queue — holds: Promise callbacks (.then, .catch, .finally), queueMicrotask(), MutationObserver

The Critical Rule: After EVERY macrotask, JavaScript drains the ENTIRE microtask queue before picking the next macrotask.

Think of it like this: Microtasks are like urgent WhatsApp replies you must send before going to sleep. Macrotasks are like tomorrow's scheduled meetings. You always finish all your urgent messages before sleeping, no matter how many.

```
console.log("1 — Start");

setTimeout(() => console.log("2 — setTimeout"), 0);  // macrotask

Promise.resolve()
  .then(() => console.log("3 — Promise .then"))      // microtask
  .then(() => console.log("4 — Promise .then 2"));   // microtask

console.log("5 — End");

// Output:
// 1 — Start
// 5 — End
// 3 — Promise .then       (microtask — runs before setTimeout!)
// 4 — Promise .then 2     (microtask — entire queue drained)
// 2 — setTimeout          (macrotask — runs last)
```

Why? Call Stack runs lines 1 and 5 synchronously. setTimeout goes to macrotask queue. Promise .then goes to microtask queue. After Call Stack is empty — microtask queue is FULLY drained first (lines 3 and 4). THEN the macrotask (line 2) runs.

Harder Example — Mix of Promises and setTimeout

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => {
  console.log("C");
  setTimeout(() => console.log("D"), 0);
});

Promise.resolve().then(() => console.log("E"));
```

```
console.log("F");

// Output: A → F → C → E → B → D
```

Step by step: A and F run synchronously. setTimeout B goes to macrotask. Two Promise .thens go to microtask. Microtasks drain: C runs (adds D to macrotask), E runs. Now macrotasks: B runs, then D runs.

> **✅Remember: Promise callbacks (.then) ALWAYS run before setTimeout callbacks — even if setTimeout has 0ms delay. Microtasks have higher priority than macrotasks. This output order question is asked in almost every senior JS interview.**

Topic 2: Type Coercion — == vs === and the Tricky Outputs

JavaScript automatically converts data types when you use == (loose equality). This is called Type Coercion. It causes many surprising results that interviewers love to ask about.

=== (strict equality) — checks value AND type. No conversion. Always predictable.

== (loose equality) — checks value only. Converts types first. Often surprising.

The Rules of == Coercion

Rule 1: If comparing number and string — string is converted to number.

Rule 2: If comparing with boolean — boolean is converted to number (true=1, false=0).

Rule 3: null == undefined is TRUE. null === undefined is FALSE.

Rule 4: null == anything_else is FALSE (except undefined).

Rule 5: NaN == NaN is FALSE. NaN never equals anything, even itself.

```
// === strict — no conversion:
console.log(1   === 1);     // true
console.log(1   === "1");   // false  (different types)
console.log(0   === false); // false  (different types)

// == loose — with conversion:
console.log(1   == "1");    // true   (string "1" → number 1)
console.log(0   == false);  // true   (false → 0)
console.log(0   == "");     // true   ("" → 0)
console.log(1   == true);   // true   (true → 1)
console.log(2   == true);   // false  (true → 1, 2 !== 1)
console.log(null == undefined); // true  (special rule)
console.log(null == 0);     // false  (null only == undefined)
console.log(null == false); // false
console.log(NaN  == NaN);   // false  (NaN is never equal to anything)
```

Truthy and Falsy Values — Complete List

In JavaScript, every value is either truthy or falsy when used in a boolean context (if statement, &&, ||, !).

FALSY values — only these 6 values are falsy:

```
false, 0, -0, 0n, "", null, undefined, NaN
```

TRUTHY values — everything else is truthy, including:

```
"0"          // non-empty string — truthy! Even "false" is truthy
[]           // empty array — truthy!
{}           // empty object — truthy!
"false"      // non-empty string — truthy!
```

```
-1            // any non-zero number — truthy!
Infinity      // truthy!
```

Classic Coercion Traps

```
console.log([] == false);  // true  ([] → "" → 0, false → 0)
console.log([] == ![]);    // true  (![] = false, [] == false → true)
console.log({} == false);  // false ({} → "[object Object]" → NaN)
console.log("" == false);  // true  (both → 0)

// typeof checks:
console.log(typeof null);        // "object"  — famous JS bug!
console.log(typeof undefined);   // "undefined"
console.log(typeof NaN);         // "number"  — surprising!
console.log(typeof []);          // "object"  — arrays are objects
console.log(typeof function(){}); // "function"

// Check for array properly:
console.log(Array.isArray([]));  // true  — correct way
console.log(typeof []);          // "object" — WRONG for array check
```

✅Remember: Always use === in real code. Use == only when you specifically need type coercion. Know these coercion results for interviews — they are classic trick questions.

Topic 3: Map and Set — Modern Data Structures

Map — Like an Object but More Powerful

Map is a collection of key-value pairs. The BIG difference from a regular object: Map can use ANY value as a key — objects, arrays, functions, numbers. An object can only use strings and symbols as keys.

Also — Map remembers the ORDER of insertion. Objects do not guarantee order.

```
// Creating a Map:
let map = new Map();

// Set key-value pairs — key can be ANYTHING:
map.set("name",  "Shaaz");      // string key
map.set(1,       "one");         // number key
map.set(true,    "boolean key"); // boolean key

let obj = { id: 1 };
map.set(obj, "object as key!"); // object key — impossible in regular object

// Get values:
console.log(map.get("name"));  // Shaaz
console.log(map.get(1));       // one
console.log(map.get(obj));     // object as key!

// Check, count, delete:
console.log(map.has("name")); // true
console.log(map.size);        // 4
map.delete("name");
console.log(map.size);        // 3

// Loop through Map:
```

```
map.forEach((value, key) => {
  console.log(key, "→", value);
});

// Convert to array:
let entries = [...map.entries()]; // [[key,val], [key,val]...]
let keys    = [...map.keys()];
let values  = [...map.values()];
```

Object vs Map — When to Use Which

Use a regular Object when: keys are strings, you need JSON serialisation, simple config/data.

Use Map when: keys are not strings (numbers, objects), order matters, you need size easily, frequent add/delete operations.

```
// Object — keys become strings automatically:
let obj = {};
obj[1]   = "one";
obj[true] = "bool";
console.log(Object.keys(obj)); // ["1", "true"]  — converted to strings!

// Map — keys keep their original type:
let map = new Map();
map.set(1,    "one");
map.set(true, "bool");
console.log([...map.keys()]); // [1, true]  — original types kept!
```

Set — Unique Values Collection

Set is like an array but it AUTOMATICALLY removes duplicates. Every value in a Set is unique. Very useful for removing duplicates from arrays.

```
// Creating a Set:
let set = new Set([1, 2, 3, 2, 1, 4, 3]);
console.log(set);        // Set {1, 2, 3, 4}  — duplicates removed!
console.log(set.size);   // 4

// Add, check, delete:
set.add(5);
set.add(1);              // already exists — ignored
console.log(set.has(3)); // true
set.delete(2);
console.log(set.size);   // 4

// Convert Set back to array:
let arr = [...set];       // spread
let arr2 = Array.from(set); // Array.from

// Most common use — remove duplicates from array:
let nums = [1, 2, 2, 3, 3, 3, 4, 5, 5];
let unique = [...new Set(nums)];
console.log(unique); // [1, 2, 3, 4, 5]

// Remove duplicate strings:
let tags = ["js", "css", "js", "html", "css", "js"];
let uniqueTags = [...new Set(tags)];
```

```
console.log(uniqueTags); // ["js", "css", "html"]

// Loop through Set:
set.forEach(value => console.log(value));
```

✅Remember: Remove duplicates in ONE line: [...new Set(array)]. Map is object with superpowers — any key type, insertion order, .size property. These are extremely common interview questions.

Topic 4: ES Modules — import and export (Asked in Every Frontend Interview)

Modules let you split your code into separate files and share only what is needed between them. Before modules, all JavaScript was in one file and every variable was global — causing conflicts and messy code.

ES Modules (ESM) are the modern standard. CommonJS (require/module.exports) is the older Node.js way.

Named Exports — Export Multiple Things

```
// math.js — exporting multiple things:
export const PI = 3.14159;

export function add(a, b) { return a + b; }
export function multiply(a, b) { return a * b; }

export class Calculator {
  square(n) { return n * n; }
}

// OR — export all at bottom:
const subtract = (a, b) => a - b;
const divide    = (a, b) => a / b;
export { subtract, divide };
```

Named Imports — Import Specific Things

```
// main.js — importing:
import { add, multiply, PI } from "./math.js";

console.log(add(2, 3));      // 5
console.log(multiply(4, 5)); // 20
console.log(PI);             // 3.14159

// Rename while importing:
import { add as sum, multiply as times } from "./math.js";
console.log(sum(1, 2));   // 3
console.log(times(3, 4)); // 12

// Import everything as a namespace:
import * as Math from "./math.js";
console.log(Math.add(1, 2));      // 3
console.log(Math.PI);             // 3.14159
```

Default Export — One Main Export per File

```
// user.js — one default export:
```

```
export default class User {
  constructor(name) { this.name = name; }
  greet() { console.log("Hello " + this.name); }
}

// OR:
const config = { theme: "dark", lang: "en" };
export default config;
```

Default Import — Any Name You Want

```
// When importing default — you choose the name:
import User from "./user.js";         // any name works
import MyUser from "./user.js";        // also valid
import TheUser from "./user.js";       // also valid

let u = new User("Shaaz");
u.greet(); // Hello Shaaz

// Mix default and named imports:
import User, { validateEmail, createToken } from "./user.js";
```

Named vs Default Export — Key Differences

Named export — curly braces {} when importing. Exact name must match (or use 'as' to rename). Multiple per file allowed.

Default export — no curly braces. Any name you want. Only ONE per file allowed.

```
import   MyThing  from "./file.js";       // default — no {}
import { myThing } from "./file.js";       // named — with {}
import MyThing, { helper } from "./file.js"; // both together
```

CommonJS vs ES Modules — Node.js Context

CommonJS (older, Node.js): uses require() and module.exports. Loads synchronously.

ES Modules (modern): uses import and export. Loads asynchronously. Now supported in both browser and Node.js.

```
// CommonJS (older Node.js way):
const fs   = require("fs");
const path = require("path");
module.exports = { myFunction, myVariable };
module.exports.default = MyClass;

// ES Modules (modern way):
import fs   from "fs";
import path from "path";
export { myFunction, myVariable };
export default MyClass;
```

✅Remember: Named exports use {}. Default exports don't. You can have many named exports but only ONE default export per file. ES Modules are the modern standard — know both named and default well.

Topic 5: Regular Expressions (Regex) — Pattern Matching

A Regular Expression (regex) is a pattern used to search, match, or replace text. It looks intimidating but once you learn the basic building blocks, it becomes very powerful.

Real life: Regex is like a smart CTRL+F. Instead of searching for exact text, you search for a PATTERN — 'find any 10-digit number' or 'find any email address'.

Creating a Regex

```
// Two ways to create:
let pattern1 = /hello/;           // literal syntax (preferred)
let pattern2 = new RegExp("hello"); // constructor syntax

// Test if pattern exists in string:
console.log(/hello/.test("say hello world")); // true
console.log(/hello/.test("say goodbye"));     // false
```

Flags — Modify How Pattern Works

i — case insensitive (A matches a)

g — global (find ALL matches, not just first)

m — multiline (^ and $ work per line)

```
console.log(/hello/i.test("Hello World"));   // true (case insensitive)
console.log(/hello/i.test("HELLO"));         // true

"hello world hello".match(/hello/);          // ["hello"] — only first
"hello world hello".match(/hello/g);         // ["hello","hello"] — all
```

Special Characters — The Building Blocks

. — any single character except newline

\d — any digit 0-9

\D — any NON-digit

\w — any word character (a-z, A-Z, 0-9, _)

\W — any NON-word character

\s — any whitespace (space, tab, newline)

\S — any NON-whitespace

^ — start of string

$ — end of string

* — 0 or more of previous

+ — 1 or more of previous

? — 0 or 1 of previous (optional)

{n} — exactly n times

{n,m} — between n and m times

[abc] — any one of a, b, c

[^abc] — any character EXCEPT a, b, c

[a-z] — any lowercase letter

[0-9] — any digit (same as \d)

(abc) — group

a|b — a or b

Practical Regex Examples

```
// Email validation:
```

```
let emailRegex = /^[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}$/;
console.log(emailRegex.test("shaaz@gmail.com"));  // true
console.log(emailRegex.test("notanemail"));       // false

// Phone number — 10 digits:
let phoneRegex = /^\d{10}$/;
console.log(phoneRegex.test("9876543210")); // true
console.log(phoneRegex.test("987654"));     // false

// Only letters and spaces:
let nameRegex = /^[a-zA-Z ]+$/;
console.log(nameRegex.test("Shaaz Akhtar")); // true
console.log(nameRegex.test("Shaaz123"));     // false

// Password — min 8 chars, one uppercase, one digit:
let passRegex = /^(?=.*[A-Z])(?=.*\d).{8,}$/;
console.log(passRegex.test("Secure123")); // true
console.log(passRegex.test("weakpass"));  // false
```

Regex Methods on Strings

```
let str = "Hello World, Hello JavaScript";

// test() — returns true/false:
console.log(/hello/i.test(str));          // true

// match() — returns matches:
console.log(str.match(/Hello/g));         // ["Hello", "Hello"]

// replace() — replace matches:
console.log(str.replace(/Hello/g, "Hi")); // "Hi World, Hi JavaScript"

// split() — split by pattern:
console.log("a1b2c3".split(/\d/));       // ["a", "b", "c", ""]

// search() — returns index of first match:
console.log(str.search(/World/));         // 6
```

Tip: For interviews, know the basics: \d, \w, \s, ^, $, +, *, ?, {n}. And know how to write email and phone validation regex — these are the most commonly asked practical tasks.

Topic 6: Prototype Chain — How JavaScript Inheritance Really Works

EVERY object in JavaScript has a hidden link called [[Prototype]] (accessed via __proto__). When you try to access a property on an object and it is not found, JavaScript automatically follows this link to the parent object, then that parent's parent, all the way up to null. This chain is the Prototype Chain.

This is how ALL JavaScript inheritance works under the hood — even classes are just cleaner syntax on top of this.

The Chain Visualised

```
let animal = {
  breathe() { console.log("breathing..."); }
};
```

```
let dog = {
  bark() { console.log("Woof!"); }
};

// Set dog prototype to animal:
Object.setPrototypeOf(dog, animal);
// OR: let dog = Object.create(animal);

dog.bark();    // found on dog itself
dog.breathe(); // NOT on dog → follow chain → found on animal!

// Check prototype:
console.log(Object.getPrototypeOf(dog) === animal); // true
console.log(dog.__proto__ === animal);              // true (same thing)
```

The Full Prototype Chain

```
class Animal { eat() { console.log("eating"); } }
class Dog extends Animal { bark() { console.log("woof"); } }

let d = new Dog();

// d → Dog.prototype → Animal.prototype → Object.prototype → null

d.bark();         // Dog.prototype ✓
d.eat();          // Animal.prototype ✓
d.toString();     // Object.prototype ✓
d.nonExistent();  // null — TypeError ✗

// hasOwnProperty — checks ONLY the object, not chain:
console.log(d.hasOwnProperty("bark")); // false — on prototype not object
console.log(d.hasOwnProperty("name")); // true  — set in constructor
```

Object.create() — Direct Prototype Control

```
let vehicleProto = {
  start()  { console.log(this.brand + " started"); },
  stop()   { console.log(this.brand + " stopped"); }
};

// Create object with vehicleProto as its prototype:
let car = Object.create(vehicleProto);
car.brand = "Toyota";
car.speed = 120;

car.start();  // Toyota started  (from prototype)
car.stop();   // Toyota stopped  (from prototype)

let bike = Object.create(vehicleProto);
bike.brand = "Honda";
bike.start(); // Honda started

// Both share the same methods — memory efficient:
console.log(car.start === bike.start); // true — same function reference!
```

✅Remember: Prototype chain = automatic upward lookup. hasOwnProperty checks only the object. Object.create() sets the prototype directly. Classes are syntactic sugar over prototype chain.

Topic 7: Iterators and for...of — Custom Iteration

for...of is a modern loop that works on anything that is iterable — arrays, strings, Maps, Sets, and custom objects you make iterable yourself.

for...of vs for...in — Very Important Difference

for...in — loops over KEYS (property names). Works on objects. Also loops inherited keys.

for...of — loops over VALUES. Works on iterables (arrays, strings, Map, Set). NOT on plain objects.

```
let arr = ["a", "b", "c"];

// for...in — gives INDEX (key):
for (let key in arr) {
  console.log(key); // 0, 1, 2  (index — not the value!)
}

// for...of — gives VALUE:
for (let val of arr) {
  console.log(val); // a, b, c  (the actual value!)
}

// String iteration:
for (let char of "hello") {
  console.log(char); // h, e, l, l, o
}

// Map iteration:
let map = new Map([["a", 1], ["b", 2]]);
for (let [key, val] of map) {
  console.log(key, val); // a 1, b 2
}

// Set iteration:
let set = new Set([10, 20, 30]);
for (let val of set) {
  console.log(val); // 10, 20, 30
}
```

Making a Custom Object Iterable

Any object can be made iterable by adding a Symbol.iterator method that returns an iterator object with a next() method.

```
let range = {
  from: 1,
  to:   5,

  [Symbol.iterator]() {       // makes this object iterable
    let current = this.from;
    let last     = this.to;
    return {
      next() {
```

```
        if (current <= last) {
          return { value: current++, done: false };
        }
        return { value: undefined, done: true };
      }
    };
  }
};

for (let num of range) {
  console.log(num); // 1, 2, 3, 4, 5
}

console.log([...range]); // [1, 2, 3, 4, 5]  — spread works too!
```

✅Remember: for...in = keys/indexes. for...of = values. for...of works on any iterable. Plain {} objects are NOT iterable by default — use for...in or Object.keys() for them.

Topic 8: Proxy and Reflect — Intercept Object Operations

Proxy lets you intercept and customise fundamental operations on objects — reading properties, setting properties, calling functions, checking existence with 'in', and more. It is like putting a security guard in front of your object.

Real life: A proxy server sits between you and a website — it can intercept your requests, validate them, log them, modify them, or block them. Proxy in JavaScript does the same for object operations.

Basic Proxy

```
let user = { name: "Shaaz", age: 23 };

let proxyUser = new Proxy(user, {
  // Intercept GET operations:
  get(target, property) {
    if (property in target) {
      return target[property];
    }
    return `Property "${property}" does not exist`;
  },

  // Intercept SET operations:
  set(target, property, value) {
    if (property === "age" && typeof value !== "number") {
      throw new TypeError("Age must be a number!");
    }
    if (property === "age" && value < 0) {
      throw new RangeError("Age cannot be negative!");
    }
    target[property] = value;
    return true; // must return true for set
  }
});

console.log(proxyUser.name);   // Shaaz
console.log(proxyUser.email);  // Property "email" does not exist
```

```
proxyUser.age = 25;            // works fine
proxyUser.age = "twenty";      // TypeError: Age must be a number!
proxyUser.age = -5;            // RangeError: Age cannot be negative!
```

Proxy for Logging

```
function createLogged(obj) {
  return new Proxy(obj, {
    get(target, prop) {
      console.log(`GET: ${prop}`);
      return target[prop];
    },
    set(target, prop, value) {
      console.log(`SET: ${prop} = ${value}`);
      target[prop] = value;
      return true;
    }
  });
}

let settings = createLogged({ theme: "dark", lang: "en" });
settings.theme;          // GET: theme
settings.lang = "hi";   // SET: lang = hi
```

Reflect — Clean Way to Call Default Operations

Reflect is a built-in object with methods that mirror Proxy trap names. Inside a Proxy trap, use Reflect to perform the default operation cleanly instead of manually accessing the target.

```
let proxy = new Proxy({}, {
  set(target, prop, value, receiver) {
    console.log(`Setting ${prop} to ${value}`);
    return Reflect.set(target, prop, value, receiver); // default set
  },
  get(target, prop, receiver) {
    console.log(`Getting ${prop}`);
    return Reflect.get(target, prop, receiver); // default get
  }
});

proxy.name = "Shaaz";   // Setting name to Shaaz
proxy.name;             // Getting name → Shaaz
```

Tip: Proxy is used in frameworks like Vue.js to detect when data changes and update the UI automatically. It is also used for validation, logging, and access control.

Topic 9: Newer Array Methods — flat, flatMap, at, findLast, Array.from

flat() — Flatten Nested Arrays

flat() removes nesting from arrays. Pass a depth to control how many levels to flatten.

```
let nested = [1, [2, 3], [4, [5, 6]]];

console.log(nested.flat());    // [1, 2, 3, 4, [5, 6]]  — 1 level
console.log(nested.flat(2));   // [1, 2, 3, 4, 5, 6]    — 2 levels
console.log(nested.flat(Infinity)); // [1,2,3,4,5,6] — all levels
```

```
// Real use — flatten API response:
let pages = [[1,2,3], [4,5,6], [7,8,9]]; // paginated data
let allItems = pages.flat();
console.log(allItems); // [1,2,3,4,5,6,7,8,9]
```

flatMap() — map() + flat(1) in One Step

```
let sentences = ["hello world", "foo bar"];

// map then flat:
let words1 = sentences.map(s => s.split(" ")).flat();

// flatMap — same result, cleaner:
let words2 = sentences.flatMap(s => s.split(" "));

console.log(words2); // ["hello", "world", "foo", "bar"]
```

at() — Access Elements with Negative Index

at() lets you access array elements using negative indices — -1 is the last element, -2 is second from last. Much cleaner than arr[arr.length - 1].

```
let arr = ["a", "b", "c", "d", "e"];

console.log(arr.at(0));   // a   (first)
console.log(arr.at(-1));  // e   (last — no more arr[arr.length-1]!)
console.log(arr.at(-2));  // d   (second from last)
console.log(arr.at(-3));  // c

"hello".at(-1);  // "o"  — works on strings too!
```

findLast() and findLastIndex()

```
let nums = [1, 2, 3, 4, 5, 4, 3];

// find — first match:
console.log(nums.find(n => n > 3));      // 4  (first one)

// findLast — last match:
console.log(nums.findLast(n => n > 3));  // 4  (last occurrence)
console.log(nums.findLastIndex(n => n > 3)); // 5  (index of last match)
```

Array.from() — Create Array from Anything

```
// From string:
console.log(Array.from("hello")); // ["h","e","l","l","o"]

// From Set:
console.log(Array.from(new Set([1,2,2,3]))); // [1,2,3]

// From Map:
let map = new Map([["a",1],["b",2]]);
console.log(Array.from(map)); // [["a",1],["b",2]]

// With mapping function — create array of 5 zeros:
console.log(Array.from({ length: 5 }, () => 0)); // [0,0,0,0,0]
```

```
// Create array 1 to 10:
let oneToTen = Array.from({ length: 10 }, (_, i) => i + 1);
console.log(oneToTen); // [1,2,3,4,5,6,7,8,9,10]

// From NodeList (DOM elements):
let divs = Array.from(document.querySelectorAll("div"));
divs.forEach(div => console.log(div.id));
```

Object.hasOwn() — Modern hasOwnProperty

```
let obj = { name: "Shaaz" };

// Old way:
console.log(obj.hasOwnProperty("name")); // true

// Modern way (ES2022):
console.log(Object.hasOwn(obj, "name")); // true — preferred
console.log(Object.hasOwn(obj, "age"));  // false
```

✅Remember: at(-1) replaces arr[arr.length-1]. flat() removes nesting. flatMap() = map + flat. Array.from() creates arrays from any iterable. These are asked in modern JS interviews.

Topic 10: Modern JavaScript Interview Questions with Solutions

Q1: What is the output? (Microtask vs Macrotask)

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
```

Answer: 1 → 4 → 3 → 2

1 and 4 run synchronously. Promise goes to microtask. setTimeout goes to macrotask. After stack is empty — microtask (3) runs first, then macrotask (2).

Q2: What is the difference between Map and Object?

Object — keys must be strings/symbols. No guaranteed order. No built-in size. Cannot use objects as keys.

Map — keys can be ANY type. Insertion order preserved. Has .size property. Objects and functions can be keys. Better performance for frequent add/delete.

Q3: How do you remove duplicates from an array?

```
let arr = [1,2,2,3,3,4];
let unique = [...new Set(arr)];
console.log(unique); // [1,2,3,4]
```

Q4: What is the difference between for...in and for...of?

for...in — iterates over KEYS (property names) of an object. Also includes inherited properties. Use for plain objects.

for...of — iterates over VALUES of an iterable (array, string, Map, Set). Does not work on plain objects.

Q5: What is the difference between named and default exports?

Named export — export { fn }. Import with exact name in {}. Multiple per file.

Default export — export default fn. Import without {}. Any name you want. Only one per file.

Q6: What is == vs === and when does coercion happen?

=== strict — checks value AND type. No coercion. Recommended always.

== loose — converts types before comparing. 0 == false is true. '1' == 1 is true. null == undefined is true.

Q7: What is typeof null and why?

typeof null returns 'object'. This is a famous JavaScript bug from its very first version. null is NOT an object — it is its own type. But the bug was never fixed to avoid breaking existing code.

```
console.log(typeof null);      // "object"  — bug!
console.log(null === null);    // true  — correct check for null
console.log(value == null);    // true for null AND undefined
```

Q8: What is a Proxy and where is it used?

Proxy wraps an object and intercepts fundamental operations like get, set, delete, has. It is used for validation, logging, reactive data systems (like Vue.js reactivity), and access control. The handler object defines which operations to intercept using traps.

Q9: What is the difference between flat() and flatMap()?

flat(depth) — flattens a nested array by the given depth. No transformation.

flatMap(fn) — maps each element with a function, then flattens ONE level. Equivalent to .map().flat(1) but more efficient.

Q10: What are all the falsy values in JavaScript?

There are exactly 8: false, 0, -0, 0n (BigInt zero), empty string '', null, undefined, NaN. Everything else is truthy — including '0' (string with zero), [] (empty array), {} (empty object).

Chapter 17: Advanced Patterns

These are real-world performance patterns used in every professional JavaScript project. Interviewers love asking about Debounce and Throttle.

Topic 1: Debouncing — Wait Until User Stops

Debounce means: wait until the user has STOPPED doing something for a set time, THEN run the function. If they start again — reset the timer.

Real life: Imagine an elevator. When someone presses the button, the elevator waits a few seconds before closing the door — in case someone else needs to get in. Every time someone presses again, the wait resets. Only when nobody presses for a few seconds does it close and go.

Without debounce: Every keypress → API call → 100 API calls for 'JavaScript tutorial'

With debounce: Wait 500ms after typing stops → 1 API call → much better!

```
function debounce(fn, delay) {
  let timer;                          // closure — timer persists
  return function(...args) {
    clearTimeout(timer);              // cancel previous timer
    timer = setTimeout(() => {
      fn(...args);                    // run ONLY after delay of silence
    }, delay);
  };
}

// Usage — search box:
function searchAPI(query) {
  console.log("Searching for:", query);
}

let debouncedSearch = debounce(searchAPI, 500);

// User types fast — "J", "Ja", "Jav", "Java", "JavaS"...
// Only last value runs after 500ms of silence
debouncedSearch("J");
debouncedSearch("Ja");
debouncedSearch("Jav");
debouncedSearch("Java");   // these cancel each other
debouncedSearch("JavaS");  // only this runs (after 500ms silence)
// Output (after 500ms): Searching for: JavaS

// In real DOM:
document.getElementById("search").addEventListener("input", (e) => {
  debouncedSearch(e.target.value);
});
```

✅Remember: Debounce uses closure to remember the timer between calls. clearTimeout cancels the previous timer. Only the LAST call in a burst actually runs.

Topic 2: Throttling — Run at Most Once Per Interval

Throttle means: no matter how many times an event fires, the function runs at MOST once every N milliseconds.

Real life: A water tap with a flow limiter. No matter how wide you open it, only a fixed amount flows per second.

Without throttle: Scrolling 1 second → scroll event fires 60+ times → 60 function calls

With throttle: Scrolling 1 second → function runs once per 200ms → 5 calls max

```
function throttle(fn, interval) {
  let lastRun = 0;                    // closure — time of last run
  return function(...args) {
    let now = Date.now();
    if (now - lastRun >= interval) {  // enough time passed?
      lastRun = now;                  // update last run time
      fn(...args);                    // run the function
    }
    // else: too soon — ignore this call
  };
}

function onScroll() {
  console.log("Scroll handler ran at:", Date.now());
}

let throttledScroll = throttle(onScroll, 1000); // max once per second

window.addEventListener("scroll", throttledScroll);
// No matter how fast user scrolls — runs max once per second

// Another example — prevent button spam:
let throttledSubmit = throttle(() => {
  console.log("Form submitted!");
}, 3000); // can only submit once every 3 seconds

document.getElementById("btn").addEventListener("click", throttledSubmit);
```

Debounce vs Throttle — When to Use Which

Debounce — use when you want to wait until activity STOPS. Best for: search input, window resize, form validation while typing.

Throttle — use when you want regular execution during continuous activity. Best for: scroll events, mouse move, game loop, API rate limiting.

```
// Memory trick:
// Debounce = "call me when you are DONE talking"
// Throttle = "call me ONCE every minute, no matter what"
```

Topic 3: Currying — One Argument at a Time

Currying transforms a function that takes multiple arguments into a chain of functions that each take ONE argument.

Instead of: add(2, 3, 5) → you call: add(2)(3)(5)

Real life: Ordering a custom pizza step by step — choose size, then toppings, then crust. Each choice is one step.

```
// Normal — all args at once:
function add(a, b, c) { return a + b + c; }
add(2, 3, 5); // 10
```

```
// Curried — one arg at a time:
function add(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}
add(2)(3)(5); // 10

// Arrow function currying — shorter:
const add = a => b => c => a + b + c;
add(2)(3)(5); // 10

// Real power — create specialised functions:
const multiply = a => b => a * b;

let double  = multiply(2);  // a=2 locked
let triple  = multiply(3);  // a=3 locked
let tenX    = multiply(10); // a=10 locked

console.log(double(5));  // 10
console.log(triple(5));  // 15
console.log(tenX(5));    // 50

// Discount calculator:
const applyDiscount = rate => price => price - (price * rate / 100);

let sale10 = applyDiscount(10);
let sale50 = applyDiscount(50);

console.log(sale10(1000)); // 900
console.log(sale50(1000)); // 500
console.log(sale10(500));  // 450
console.log(sale50(200));  // 100
```

Tip: Currying is built on closures — each outer function returns an inner function that remembers the outer argument. Same concept, practical application.

Topic 4: IIFE — Immediately Invoked Function Expression

An IIFE is a function that is defined and called at the SAME time. You write the function and immediately run it with () at the end.

Why? To create a private scope — variables inside the IIFE are not accessible outside. No pollution of global scope.

```
// Normal function — must call separately:
function greet() { console.log("Hello!"); }
greet();

// IIFE — defined and called immediately:
(function() {
  console.log("Hello from IIFE!"); // runs right away
})(  );
```

```
// Arrow function IIFE:
(() => {
  console.log("Arrow IIFE!");
})();

// IIFE with parameters:
(function(name) {
  console.log("Hello, " + name + "!");
})("Shaaz"); // Output: Hello, Shaaz!

// IIFE for private scope — variables cannot leak:
(function() {
  let privateData = "secret";
  console.log(privateData); // secret — accessible inside
})();
console.log(typeof privateData); // undefined — not accessible outside!
```

✅Remember: IIFE = define + call immediately. Used for private scope, one-time setup code, and the module pattern. Before ES6 modules, IIFE was THE way to avoid global variable pollution.

Topic 5: Mixin Pattern — Add Features Without Inheritance

A Mixin is a way to add methods to a class without using inheritance. Instead of extending one parent, you mix in features from multiple sources.

Real life: You are building a user profile. Users can swim. They can also fly (admin feature). They can also sing (premium feature). You do not want one giant parent class — you mix in just what you need.

```
// Mixin objects containing reusable methods:
const Swimmer = {
  swim() { console.log(`${this.name} is swimming!`); }
};

const Singer = {
  sing() { console.log(`${this.name} is singing!`); }
};

const Coder = {
  code() { console.log(`${this.name} is coding!`); }
};

// Base class:
class Person {
  constructor(name) { this.name = name; }
}

// Mix features into different classes:
class Athlete extends Person {}
Object.assign(Athlete.prototype, Swimmer, Singer);

class Developer extends Person {}
Object.assign(Developer.prototype, Coder, Swimmer);

let athlete = new Athlete("Shaaz");
athlete.swim();  // Shaaz is swimming!
```

```
athlete.sing();  // Shaaz is singing!

let dev = new Developer("Aman");
dev.code();  // Aman is coding!
dev.swim();  // Aman is swimming!
```

Tip: Mixins are an alternative to inheritance. Use them when a class needs features from multiple sources — JavaScript only allows single inheritance with extends, but mixins give you multiple inheritance-like behaviour.

Topic 6: Advanced Patterns Interview Questions

Q1: What is debouncing? Give a real example.

Debouncing ensures a function runs only after a user has STOPPED an action for a set time. Real example: Search box — instead of calling the API on every keystroke, wait until the user stops typing for 500ms, then search. Saves many API calls.

Q2: What is the difference between debounce and throttle?

Debounce — waits until the activity STOPS, then runs once. Good for: search input, resize.

Throttle — runs at most once per time interval, even if activity continues. Good for: scroll events, button spam prevention.

Q3: What is currying and why is it useful?

Currying converts a multi-argument function into a chain of single-argument functions. It is useful for creating specialised reusable functions from general ones — like a discount calculator where you pre-set the rate and reuse it for many prices.

Q4: What is an IIFE and why was it used?

IIFE (Immediately Invoked Function Expression) is a function that runs immediately when defined. It was used to create private scopes and avoid global variable pollution — especially before ES6 modules existed. Variables inside an IIFE cannot be accessed outside.

Q5: Implement debounce from scratch.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

Q6: Implement throttle from scratch.

```
function throttle(fn, interval) {
  let lastRun = 0;
  return function(...args) {
    let now = Date.now();
    if (now - lastRun >= interval) {
      lastRun = now;
      fn(...args);
    }
  };
}
```

✅Remember: Implementing debounce and throttle from scratch is a VERY common senior JavaScript interview question. Both use closures to maintain state between calls. Practice writing these from memory.

Chapter 18: Polyfills — Write It From Scratch

How to Define in an Interview

"A Polyfill is a piece of code that implements a feature that the browser or JavaScript engine does not natively support. It fills the gap — if a browser does not have Array.prototype.map(), you write your own version that behaves exactly the same. Polyfills are also the most common JavaScript interview task — interviewers ask you to implement map, filter, reduce, bind, flat, Promise etc. from scratch to test if you truly understand how they work internally, not just how to use them."

Why interviewers ask polyfills: If you can write map() from scratch, you TRULY understand it — not just how to use it.

Polyfill 1: Array.prototype.map()

What map() does: loops through every element, runs your callback on it, collects the result in a NEW array, and returns it. The original array is NEVER changed.

Callback receives 3 arguments: (currentElement, index, originalArray)

Real map() — How It Works

```
let nums = [1, 2, 3, 4];
let doubled = nums.map(function(num, index, array) {
  return num * 2;
});
console.log(doubled); // [2, 4, 6, 8]
console.log(nums);    // [1, 2, 3, 4]  — original unchanged
```

Polyfill — Write map() from Scratch

```
Array.prototype.myMap = function(callback) {
  // 'this' = the array myMap is called on
  let result = [];

  for (let i = 0; i < this.length; i++) {
    // pass: current element, index, original array
    result.push(callback(this[i], i, this));
  }

  return result;   // return the new array
};

// Test it:
let nums = [1, 2, 3, 4];

console.log(nums.myMap(n => n * 2));          // [2, 4, 6, 8]
console.log(nums.myMap(n => n * n));          // [1, 4, 9, 16]
console.log(nums.myMap((n, i) => `${i}:${n}`)); // ['0:1','1:2','2:3','3:4']
console.log(nums);  // [1, 2, 3, 4] — UNCHANGED
```

Note: Key things to remember: 'this' is the array. Create NEW array — do NOT modify original. Pass (element, index, array) to callback. Return the new array at the end.

Polyfill 2: Array.prototype.filter()

What filter() does: loops through every element, runs the callback, and keeps ONLY the elements where the callback returns true. Returns a new array (shorter or equal length, never longer).

Polyfill — Write filter() from Scratch

```
Array.prototype.myFilter = function(callback) {
  let result = [];

  for (let i = 0; i < this.length; i++) {
    if (callback(this[i], i, this)) {   // only keep if truthy
      result.push(this[i]);
    }
  }

  return result;
};

// Test it:
let nums = [1, 2, 3, 4, 5, 6, 7, 8];

console.log(nums.myFilter(n => n % 2 === 0)); // [2, 4, 6, 8]
console.log(nums.myFilter(n => n > 5));       // [6, 7, 8]

// With objects:
let users = [
  { name: 'Shaaz', age: 23, active: true  },
  { name: 'Aman',  age: 17, active: false },
  { name: 'Riya',  age: 25, active: true  },
];

let activeAdults = users.myFilter(u => u.active && u.age >= 18);
// [{ name:'Shaaz'...}, { name:'Riya'...}]
```

Polyfill 3: Array.prototype.reduce()

What reduce() does: goes through every element, ACCUMULATES a single result using a callback. Think of acc as a running total that updates with every element.

Two forms: with initialValue (acc starts as initialValue, loop from 0) and without (acc starts as first element, loop from index 1).

Step-by-Step How reduce() Runs

```
let nums = [1, 2, 3, 4, 5];
let sum = nums.reduce((acc, curr) => acc + curr, 0);
// acc=0,  curr=1  → acc = 1
// acc=1,  curr=2  → acc = 3
// acc=3,  curr=3  → acc = 6
// acc=6,  curr=4  → acc = 10
// acc=10, curr=5  → acc = 15  → returns 15
```

Polyfill — Write reduce() from Scratch

```
Array.prototype.myReduce = function(callback, initialValue) {
  let acc;
  let startIndex;
```

```
  if (initialValue !== undefined) {
    acc        = initialValue;  // use given initial value
    startIndex = 0;             // start loop from index 0
  } else {
    acc        = this[0];       // first element is accumulator
    startIndex = 1;             // start loop from index 1
  }

  for (let i = startIndex; i < this.length; i++) {
    acc = callback(acc, this[i], i, this);
  }

  return acc;
};

// Test it:
let nums = [1, 2, 3, 4, 5];

console.log(nums.myReduce((acc, n) => acc + n, 0));  // 15
console.log(nums.myReduce((acc, n) => acc * n, 1));  // 120
console.log(nums.myReduce((acc, n) => n > acc ? n : acc)); // 5 (max)

// Count occurrences:
let fruits = ['apple','banana','apple','mango','banana','apple'];
let count  = fruits.myReduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
console.log(count); // { apple:3, banana:2, mango:1 }

// Flatten array:
let nested = [[1,2],[3,4],[5,6]];
let flat = nested.myReduce((acc, arr) => acc.concat(arr), []);
console.log(flat); // [1, 2, 3, 4, 5, 6]
```

✅Remember: With initialValue: loop starts at index 0. Without: first element is acc, loop starts at index 1. This distinction is a favourite interview follow-up question.

Polyfill 4: Array.prototype.forEach()

forEach() simply calls the callback for every element. It does NOT return anything — always returns undefined. It cannot be stopped with break.

```
Array.prototype.myForEach = function(callback) {
  for (let i = 0; i < this.length; i++) {
    callback(this[i], i, this);
  }
  // no return statement — forEach always returns undefined
};

let nums = [10, 20, 30];
nums.myForEach((n, i) => console.log(`[${i}] = ${n}`));
// [0] = 10  /  [1] = 20  /  [2] = 30

let result = nums.myForEach(n => n * 2);
```

```
console.log(result); // undefined — forEach always returns undefined
```

Polyfill 5: Array.prototype.find() and findIndex()

find() returns the FIRST element where callback returns true. findIndex() returns the INDEX of that element (-1 if not found). Both short-circuit — stop immediately on first match.

```
Array.prototype.myFind = function(callback) {
  for (let i = 0; i < this.length; i++) {
    if (callback(this[i], i, this)) return this[i]; // stop immediately
  }
  return undefined;
};

Array.prototype.myFindIndex = function(callback) {
  for (let i = 0; i < this.length; i++) {
    if (callback(this[i], i, this)) return i;       // stop immediately
  }
  return -1;  // not found
};

let users = [
  { id: 1, name: 'Shaaz' },
  { id: 2, name: 'Aman'  },
  { id: 3, name: 'Riya'  },
];

console.log(users.myFind(u => u.id === 2));       // { id:2, name:'Aman' }
console.log(users.myFindIndex(u => u.id === 2)); // 1
console.log(users.myFind(u => u.id === 99));      // undefined
console.log(users.myFindIndex(u => u.id === 99)); // -1
```

Polyfill 6: Array.prototype.every() and some()

every() returns true only if callback is true for ALL elements — stops at first false. some() returns true if callback is true for AT LEAST ONE element — stops at first true. Both short-circuit.

```
Array.prototype.myEvery = function(callback) {
  for (let i = 0; i < this.length; i++) {
    if (!callback(this[i], i, this)) return false; // one false = done
  }
  return true;
};

Array.prototype.mySome = function(callback) {
  for (let i = 0; i < this.length; i++) {
    if (callback(this[i], i, this)) return true;  // one true = done
  }
  return false;
};

let nums = [2, 4, 6, 8, 9];
console.log(nums.myEvery(n => n % 2 === 0)); // false (9 fails)
console.log(nums.mySome(n => n % 2 === 0));  // true  (2 passes first)
```

```
let allPos = [1, 5, 10, 20];
console.log(allPos.myEvery(n => n > 0)); // true  — all positive
console.log(allPos.mySome(n => n > 15)); // true  — 20 passes
console.log(allPos.mySome(n => n > 50)); // false — none passes
```

Polyfill 7: Array.prototype.flat()

flat(depth) removes nesting from arrays. Default depth is 1. With Infinity it removes all nesting. Uses recursion — inner arrays call flatten again with reduced depth.

```
Array.prototype.myFlat = function(depth = 1) {
  function flatten(arr, d) {
    let result = [];
    for (let i = 0; i < arr.length; i++) {
      if (Array.isArray(arr[i]) && d > 0) {
        result.push(...flatten(arr[i], d - 1)); // go one level deeper
      } else {
        result.push(arr[i]);
      }
    }
    return result;
  }
  return flatten(this, depth);
};

let nested = [1, [2, 3], [4, [5, [6]]]];
console.log(nested.myFlat());         // [1, 2, 3, 4, [5,[6]]]  depth=1
console.log(nested.myFlat(2));        // [1, 2, 3, 4, 5, [6]]   depth=2
console.log(nested.myFlat(Infinity)); // [1, 2, 3, 4, 5, 6]     all
```

Polyfill 8: Function.prototype.bind() — Most Asked in Interviews!

bind() returns a NEW function with 'this' permanently locked. It can also pre-fill some arguments (partial application). The key insight: save the original function in a variable (as 'this' inside bind is the function itself), then return a new function that calls it with the locked context.

Real bind() — How It Works

```
function greet(greeting, punctuation) {
  return `${greeting} ${this.name}${punctuation}`;
}
let obj = { name: 'Shaaz' };
let fn  = greet.bind(obj, 'Hello');  // pre-fill 'Hello'
console.log(fn('!'));   // Hello Shaaz!
console.log(fn('...')); // Hello Shaaz...
```

Polyfill — Write bind() from Scratch

```
Function.prototype.myBind = function(context, ...boundArgs) {
  let originalFn = this;  // 'this' here = the function bind is called on

  return function(...callArgs) {
    // merge pre-filled args + args passed when calling
    return originalFn.call(context, ...boundArgs, ...callArgs);
  };
};
```

```
// Test it:
function greet(greeting, punctuation) {
  return `${greeting} ${this.name}${punctuation}`;
}
let obj = { name: 'Shaaz' };

let sayHello = greet.myBind(obj, 'Hello');
console.log(sayHello('!'));   // Hello Shaaz!
console.log(sayHello('...')); // Hello Shaaz...

// Partial application:
function multiply(a, b) { return a * b; }
let double = multiply.myBind(null, 2); // pre-fill a=2
let triple = multiply.myBind(null, 3); // pre-fill a=3
console.log(double(5));  // 10
console.log(triple(5));  // 15
```

✅Remember: bind polyfill: save 'this' as originalFn. Return new function. Inside it call originalFn.call(context, ...preFilledArgs, ...newArgs). This is THE most asked senior polyfill question.

Polyfill 9: Function.prototype.call()

call() runs a function immediately with a specific 'this'. The trick: temporarily add the function as a property on the context object, call it as a method (so 'this' becomes context), then delete the temporary property.

```
Function.prototype.myCall = function(context, ...args) {
  context = context || globalThis;

  // Use Symbol to avoid clashing with existing properties:
  let uniqueKey     = Symbol();
  context[uniqueKey] = this;   // attach function temporarily

  let result = context[uniqueKey](...args); // call as method → this = context

  delete context[uniqueKey];  // clean up
  return result;
};

function introduce(city, country) {
  return `I am ${this.name} from ${city}, ${country}`;
}
let person = { name: 'Shaaz' };

console.log(introduce.myCall(person, 'Delhi', 'India'));
// I am Shaaz from Delhi, India
```

□ Tip: Why Symbol()? If you use a hardcoded key like 'fn' or 'temp', it might already exist on the context object and you would overwrite it. Symbol() creates a guaranteed unique key.

Polyfill 10: Function.prototype.apply()

apply() is identical to call() — the only difference is arguments come as an ARRAY instead of comma-separated values. The polyfill is nearly the same as call() with one small change.

```
Function.prototype.myApply = function(context, argsArray) {
  context = context || globalThis;
```

```
  let uniqueKey      = Symbol();
  context[uniqueKey] = this;

  // spread argsArray (it could be undefined — use [] as fallback):
  let result = context[uniqueKey](...(argsArray || []));

  delete context[uniqueKey];
  return result;
};

function introduce(city, country) {
  return `I am ${this.name} from ${city}, ${country}`;
}
let person = { name: 'Shaaz' };

console.log(introduce.myApply(person, ['Delhi', 'India']));
// I am Shaaz from Delhi, India

// Classic apply use — Math.max with array:
let nums = [3, 67, 12, 99, 45];
console.log(Math.max.myApply(null, nums)); // 99
```

Polyfill 11: Promise.all()

Promise.all() takes an array of promises, runs them simultaneously, and resolves with an array of all results when ALL succeed. If even ONE rejects — it rejects immediately with that error.

The tricky part: results must be stored in ORDER (matching input order), even though promises may resolve at different times.

```
function myPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    if (promises.length === 0) { resolve([]); return; }

    let results   = [];
    let completed = 0;

    promises.forEach((promise, index) => {
      Promise.resolve(promise)  // handle non-promise values too
        .then(value => {
          results[index] = value; // preserve ORDER by using index
          completed++;
          if (completed === promises.length) resolve(results);
        })
        .catch(err => reject(err)); // one failure = reject all
    });
  });
}

// Test it:
let p1 = Promise.resolve('one');
let p2 = new Promise(res => setTimeout(() => res('two'), 1000));
let p3 = Promise.resolve('three');
```

```
myPromiseAll([p1, p2, p3]).then(vals => console.log(vals));
// After 1s: ['one', 'two', 'three']  — order preserved!

// Rejection:
myPromiseAll([p1, Promise.reject('FAIL'), p3])
  .catch(err => console.log(err)); // FAIL
```

✅Remember: results[index] = value is the key insight — it preserves the ORDER of results matching input order regardless of which promise resolves first.

Polyfill 12: Promise.allSettled()

Promise.allSettled() waits for ALL promises to finish — whether they resolve or reject. It NEVER rejects. Each result has a status ('fulfilled' or 'rejected') and either a value or reason.

```
function myPromiseAllSettled(promises) {
  return new Promise(resolve => {
    if (promises.length === 0) { resolve([]); return; }

    let results   = [];
    let completed = 0;

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then(value => {
          results[index] = { status: 'fulfilled', value };
        })
        .catch(reason => {
          results[index] = { status: 'rejected', reason };
        })
        .finally(() => {
          completed++;
          if (completed === promises.length) resolve(results);
        });
    });
  });
}

myPromiseAllSettled([
  Promise.resolve('success'),
  Promise.reject('error'),
  Promise.resolve('another success')
]).then(results => console.log(results));
// [
//   { status: 'fulfilled', value: 'success' },
//   { status: 'rejected',  reason: 'error' },
//   { status: 'fulfilled', value: 'another success' }
// ]
```

Polyfill 13: Array.from()

Array.from() creates a new array from any iterable (string, Set, Map, NodeList) or array-like object. Optionally applies a mapping function to each element.

```
function myArrayFrom(iterable, mapFn) {
  let result = [];
  for (let item of iterable) { // for...of works on any iterable
```

```
    result.push(item);
  }
  return mapFn ? result.map(mapFn) : result;
}

console.log(myArrayFrom('hello'));           // ['h','e','l','l','o']
console.log(myArrayFrom(new Set([1,2,3]))); // [1, 2, 3]
console.log(myArrayFrom([1,2,3], n => n*2)); // [2, 4, 6]

// Create array 1 to 5:
console.log(Array.from({ length: 5 }, (_, i) => i + 1)); // [1,2,3,4,5]
```

Polyfill 14: Object.assign()

Object.assign() copies all own enumerable properties from source objects into target. Modifies and returns the target. Later sources overwrite earlier ones on conflict.

```
function myObjectAssign(target, ...sources) {
  if (target == null) throw new TypeError('Target cannot be null');
  let result = Object(target);

  sources.forEach(source => {
    if (source == null) return;          // skip null/undefined sources
    Object.keys(source).forEach(key => {
      result[key] = source[key];         // copy own enumerable props
    });
  });

  return result;
}

let target  = { a: 1, b: 2 };
let source1 = { b: 3, c: 4 };  // b will overwrite target.b
let source2 = { d: 5 };

console.log(myObjectAssign(target, source1, source2));
// { a:1, b:3, c:4, d:5 }

// Shallow copy without mutating original:
let copy = myObjectAssign({}, target);
```

Polyfill 15: Deep Clone — structuredClone() Polyfill

A deep clone creates a completely independent copy — all nested objects and arrays are also copied, not shared. No reference link to the original.

```
function deepClone(value) {
  if (value === null || typeof value !== 'object') return value;

  if (value instanceof Date)  return new Date(value.getTime());

  if (Array.isArray(value))   return value.map(item => deepClone(item));

  // Plain object — recursively clone each own property:
  let cloned = {};
```

```
  for (let key in value) {
    if (value.hasOwnProperty(key)) {
      cloned[key] = deepClone(value[key]);
    }
  }
  return cloned;
}

let original = {
  name:    'Shaaz',
  scores:  [95, 87, 92],
  address: { city: 'Delhi', pin: '110001' }
};

let clone = deepClone(original);
clone.name         = 'Aman';
clone.scores.push(100);
clone.address.city = 'Mumbai';

console.log(original.name);         // Shaaz   ✓ unchanged
console.log(original.scores);       // [95,87,92] ✓ unchanged
console.log(original.address.city); // Delhi   ✓ unchanged
```

Polyfill 16: pipe() and compose() — Functional Programming

pipe() chains functions left to right — output of one becomes input of next. compose() does the same but right to left. Both are built using reduce/reduceRight.

```
// pipe — left to right:
function pipe(...fns) {
  return function(value) {
    return fns.reduce((acc, fn) => fn(acc), value);
  };
}

// compose — right to left:
function compose(...fns) {
  return function(value) {
    return fns.reduceRight((acc, fn) => fn(acc), value);
  };
}

const double  = x => x * 2;
const addTen  = x => x + 10;
const square  = x => x * x;

const transform = pipe(double, addTen, square);
console.log(transform(3));
// double(3)=6 → addTen(6)=16 → square(16)=256

const same = compose(square, addTen, double);
console.log(same(3)); // same result: 256 (runs right to left)
```

Tip: pipe and compose are functional programming patterns built entirely on reduce. Seeing them in an interview shows you understand both higher-order functions AND reduce deeply.

Polyfill 17: once() — Run Function Only Once

once() returns a wrapper function that calls the original function ONLY on the first call. All subsequent calls return the cached result without calling the original again.

```
function once(fn) {
  let called = false;
  let result;
  return function(...args) {
    if (!called) {
      called = true;
      result = fn(...args);
    }
    return result;   // always return first result
  };
}

let init = once(() => {
  console.log('Initialised!');
  return 42;
});

console.log(init()); // Initialised! → 42
console.log(init()); // 42  (no log — fn not called again)
console.log(init()); // 42
```

Polyfill 18: memoize() — Cache Function Results

memoize() wraps a function and caches its results. If the same arguments are passed again, it returns the cached result instantly without re-running the expensive function.

```
function memoize(fn) {
  let cache = {};  // private cache — closure!

  return function(...args) {
    let key = JSON.stringify(args);  // convert args to string key

    if (key in cache) {
      console.log('From cache:', key);
      return cache[key];
    }

    console.log('Computing:', key);
    cache[key] = fn(...args);
    return cache[key];
  };
}

// Expensive function:
function slowSquare(n) {
  // imagine this takes a long time
  return n * n;
}

let fastSquare = memoize(slowSquare);
```

```
fastSquare(5);  // Computing: [5]  → 25
fastSquare(5);  // From cache: [5] → 25  (instant!)
fastSquare(6);  // Computing: [6]  → 36
fastSquare(6);  // From cache: [6] → 36  (instant!)

// Works with multiple args:
let memoAdd = memoize((a, b) => a + b);
memoAdd(2, 3); // Computing: [2,3] → 5
memoAdd(2, 3); // From cache → 5
```

Polyfill Interview Questions with Solutions

Q1: What is a polyfill? Why is it needed?

A polyfill is code that implements a modern JavaScript feature for browsers/environments that don't support it natively. It fills the gap. For example, older browsers may not have Array.prototype.flat() — a polyfill provides the same behaviour. Also used in interviews to test deep understanding of built-in methods.

Q2: Write map polyfill — clean one-liner style

```
Array.prototype.myMap = function(cb) {
  let res = [];
  for (let i = 0; i < this.length; i++) res.push(cb(this[i], i, this));
  return res;
};
```

Q3: What is the difference between map, filter, and reduce?

map — transforms EVERY element → returns same-length new array.

filter — keeps elements that PASS a test → returns shorter or equal new array.

reduce — accumulates ALL elements into ONE single value (number, object, array, anything).

Q4: Write bind polyfill — the most asked one

```
Function.prototype.myBind = function(ctx, ...boundArgs) {
  let fn = this;
  return function(...callArgs) {
    return fn.call(ctx, ...boundArgs, ...callArgs);
  };
};
```

Q5: Why use Symbol() in call/apply polyfill?

To avoid overwriting existing properties on the context object. If the object already has a property called 'fn' or 'temp', using that name would corrupt it. Symbol() creates a guaranteed unique key that cannot clash with anything on the object.

Q6: What is the tricky part in Promise.all polyfill?

Preserving ORDER of results. Promises may resolve at different times, but results must match the ORDER of input. Using results[index] = value instead of results.push(value) ensures order is preserved regardless of resolution order.

Q7: What is the difference between reduce with and without initialValue?

With initialValue: acc starts as initialValue, loop runs from index 0. Required when reducing into a different type (e.g. array → object).

Without initialValue: first element becomes acc, loop runs from index 1. Works for same-type reductions like summing numbers.

Q8: Implement debounce — another common polyfill question

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

Q9: How does memoize() work? What is its internal mechanism?

memoize() uses a closure to maintain a private cache object between calls. On each call, it serialises the arguments into a string key using JSON.stringify. If the key exists in cache — returns cached result instantly. If not — calls the original function, stores the result in cache, and returns it.

Q10: Implement a curry() function that converts any function to curried form

```
function curry(fn) {
  return function curried(...args) {
    // if enough args given — call the original function:
    if (args.length >= fn.length) {
      return fn(...args);
    }
    // otherwise — return a function waiting for more args:
    return function(...moreArgs) {
      return curried(...args, ...moreArgs);
    };
  };
}

function add(a, b, c) { return a + b + c; }

let curriedAdd = curry(add);
console.log(curriedAdd(1)(2)(3));   // 6
console.log(curriedAdd(1, 2)(3));   // 6
console.log(curriedAdd(1)(2, 3));   // 6
console.log(curriedAdd(1, 2, 3));   // 6
```

Tip: fn.length is the number of parameters the function expects. This is how curry knows when it has received enough arguments.

Quick Reference — All Polyfills at a Glance

myMap(cb) — loop + push callback result → new array

myFilter(cb) — loop + push if callback true → new array

myReduce(cb, iv) — loop + accumulate → single value

myForEach(cb) — loop + call callback → undefined

myFind(cb) — first element where callback true

myFindIndex(cb) — index of first match, -1 if not found

myEvery(cb) — false at first failure, else true
mySome(cb) — true at first success, else false
myFlat(depth) — recursive flatten by given depth
myBind(ctx,...args) — new fn with locked this + partial args
myCall(ctx,...args) — Symbol attach fn to ctx, call, delete
myApply(ctx,[args]) — same as call but args as array
myPromiseAll — resolve when all done, reject on first fail
myPromiseAllSettled — wait for all, never rejects, status per item
myArrayFrom — for...of → array, optional mapFn
myObjectAssign — copy own enumerable keys from sources
deepClone — recursive full copy, no shared references
pipe(...fns) — left to right chain using reduce
compose(...fns) — right to left chain using reduceRight
once(fn) — closure flag, run fn only on first call
memoize(fn) — cache by JSON.stringify(args) key
curry(fn) — args.length vs fn.length, wait for all args

✅Remember: The most asked polyfills in order of frequency: bind() — almost always. map/filter/reduce — very common. Promise.all — senior interviews. debounce/throttle — frontend roles. curry/pipe/compose — functional programming rounds. once/memoize — tricky closure rounds. Practice ALL of these from memory.

Chapter 19: Machine Coding Round — Complete Practice Guide

How to Define in an Interview

"A Machine Coding Round tests whether you can write clean, working JavaScript in a time-pressured environment. Unlike theory questions, the interviewer gives you a real-world problem and watches you build it live. This chapter covers every type of question asked — output-based traps, function implementations, full system builds, data structures, algorithms, and advanced JS patterns. The goal is not just to know the answer but to be able to code it from scratch in under 30 minutes."

How to use this chapter: Cover the solution first. Try to write it yourself. Then reveal and compare. The struggle before seeing the answer is what builds real interview skill.

SECTION A — Output Based Questions

The most common trick in output questions: JavaScript does NOT always run top to bottom. Async, hoisting, coercion, and closures all change execution order. Think before running.

A1 — Hoisting and var/let/const

Q1 — Basic Hoisting

```
console.log(a);
console.log(b);
var a = 10;
let b = 20;
```

Output: undefined — ReferenceError

Why: var is hoisted and initialised as undefined. let is hoisted but stays in the Temporal Dead Zone (TDZ) — accessing it before declaration throws ReferenceError.

Q2 — Function Hoisting

```
greet();
sayHi();

function greet() { console.log('Hello'); }
var sayHi = function() { console.log('Hi'); };
```

Output: Hello — TypeError: sayHi is not a function

Why: Function declarations are hoisted FULLY (name + body). Function expressions (var sayHi = function) are hoisted only as var sayHi = undefined. Calling undefined() throws TypeError.

Q3 — Hoisting Inside Function

```
var x = 1;
function test() {
  console.log(x);
  var x = 2;
  console.log(x);
}
test();
```

Output: undefined — 2

Why: Inside test(), var x is hoisted to TOP of the function (not global). At first log, local x exists but is undefined. After assignment it becomes 2.

Q4 — var in Loop (The Classic Trap)

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Output: 3 3 3

Why: var has no block scope — ONE shared i. By the time callbacks run, loop is done and i=3. All 3 callbacks print 3.

Q5 — let in Loop (The Fix)

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

Output: 0 1 2

Why: let creates a NEW binding for each iteration. Each callback closes over its own separate i.

Q6 — const Object Mutation

```
const obj = { name: 'Shaaz' };
obj.name = 'Aman';
obj = {};
console.log(obj.name);
```

Output: TypeError on line 3 — never reaches console.log

Why: const prevents REASSIGNMENT of the variable (obj = {} fails). It does NOT prevent mutating the object's properties. obj.name = 'Aman' works fine.

A2 — Closures Output Questions

Q7 — Separate Closures

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const c1 = makeCounter();
const c2 = makeCounter();
console.log(c1()); // ?
console.log(c1()); // ?
console.log(c2()); // ?
```

Output: 1 — 2 — 1

Why: c1 and c2 are separate closures with their own private count. c2 starts fresh — its count is independent of c1.

Q8 — Closure Captures Reference

```
function outer() {
  let x = 10;
  function inner() { console.log(x); }
  x = 20;
  return inner;
}
outer()();
```

Output: 20

Why: Closure captures a REFERENCE to x, not its value at creation time. When inner runs, x has already been changed to 20.

Q9 — Classic Closure + var Bug

```
function make() {
  const fns = [];
  for (var i = 0; i < 3; i++) {
    fns.push(() => i);
  }
  return fns;
}
const fns = make();
console.log(fns[0]()); // ?
console.log(fns[1]()); // ?
```

Output: 3 — 3

Why: All three functions share the same var i. Loop ends with i=3. Fix: use let, or wrap each in an IIFE.

Q10 — IIFE Fix

```
function make() {
  const fns = [];
  for (var i = 0; i < 3; i++) {
    fns.push((function(j) {
      return () => j;
    })(i));
  }
  return fns;
}
const fns = make();
console.log(fns[0]()); // ?
console.log(fns[1]()); // ?
```

Output: 0 — 1

Why: IIFE creates a NEW scope for each iteration, capturing the current value of i as j. Now each function has its own j.

A3 — 'this' Keyword Output Questions

Q11 — Method Extraction Bug

```
const person = {
  name: 'Shaaz',
  greet() { console.log(this.name); }
};
person.greet();
const fn = person.greet;
fn();
```

Output: Shaaz — undefined (or '' in browser)

Why: person.greet() — this is person. fn() — this is window/global. window.name is '' by default in browsers.

Q12 — Arrow vs Regular in Object

```
const obj = {
```

```
  val: 100,
  getVal() {
    const inner = () => this.val;
    return inner();
  },
  getVal2: () => this.val
};
console.log(obj.getVal());  // ?
console.log(obj.getVal2()); // ?
```

Output: 100 — undefined

Why: getVal uses regular method — this is obj. Arrow inner inherits this from getVal — still obj. getVal2 is an arrow function defined at module level — this is the module scope (undefined or window), NOT obj.

Q13 — this in setTimeout

```
function Timer() {
  this.seconds = 0;
  setInterval(function() {
    this.seconds++;
    console.log(this.seconds);
  }, 1000);
}
new Timer();
```

Output: NaN (every second)

Why: Regular function in setInterval — this is window, not Timer. window.seconds is undefined. undefined++ = NaN. Fix: use arrow function.

Q14 — new Binding

```
function Person(name) {
  this.name = name;
  this.greet = () => console.log('Hello ' + this.name);
}
const p1 = new Person('Shaaz');
const p2 = new Person('Aman');
p1.greet.call(p2);
```

Output: Hello Shaaz

Why: greet is an arrow function — its this is permanently bound to the Person instance created by new. .call(p2) has no effect on arrow function's this. Arrow this cannot be changed.

A4 — Async, Promise, and Event Loop Output Questions

Q15 — Microtask vs Macrotask

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

Output: A — D — C — B

Why: A and D are synchronous. Promise.then goes to microtask queue. setTimeout goes to macrotask. Microtasks always run BEFORE macrotasks. So C before B.

Q16 — async/await Execution Order

```
async function getData() {
  console.log('1');
  await Promise.resolve();
  console.log('2');
}
console.log('start');
getData();
console.log('end');
```

Output: start — 1 — end — 2

Why: start runs. getData() starts and prints 1. await pauses getData and yields control back. end runs. Then microtask queue resumes getData — prints 2.

Q17 — Nested Promises

```
Promise.resolve(1)
  .then(v => { console.log(v); return 2; })
  .then(v => { console.log(v); return 3; })
  .then(v => console.log(v));
```

Output: 1 — 2 — 3

Why: Each .then receives the return value of the previous one and passes its own return to the next.

Q18 — Promise Error Propagation

```
Promise.resolve()
  .then(() => { throw new Error('boom'); })
  .then(() => console.log('then'))
  .catch(e => console.log('catch:', e.message))
  .then(() => console.log('after catch'));
```

Output: catch: boom — after catch

Why: When .then throws, it skips to the next .catch. After .catch handles the error, the chain CONTINUES normally — 'after catch' runs.

Q19 — Async Function Return Value

```
async function fail() {
  throw new Error('oops');
}
fail()
  .then(v  => console.log('then:', v))
  .catch(e => console.log('catch:', e.message));
```

Output: catch: oops

Why: async function that throws returns a REJECTED promise. .then is skipped. .catch handles it.

Q20 — setTimeout Order

```
console.log('1');
setTimeout(() => console.log('2'), 1000);
setTimeout(() => console.log('3'), 0);
console.log('4');
```

Output: 1 — 4 — 3 — 2

Why: 1 and 4 are synchronous — run immediately. setTimeout 0ms runs before 1000ms. Both are macrotasks so they run after all synchronous code.

A5 — Type Coercion and Tricky Comparisons

Q21 — Addition Traps

```
console.log(1  + '2');
console.log('3' - 1);
console.log(true + true);
console.log([] + []);
console.log([] + {});
```

Output: '12' — 2 — 2 — '' — '[object Object]'

Why: + with string = concatenation. - is always numeric. true=1. []+[] = ''+'' = ''. []+{} = '' + '[object Object]'.

Q22 — Equality Traps

```
console.log(null == undefined);
console.log(null === undefined);
console.log(NaN === NaN);
console.log(typeof null);
console.log(typeof NaN);
console.log(0 == false);
console.log('' == false);
console.log('0' == false);
```

Output: true — false — false — 'object' — 'number' — true — true — true

Why: null==undefined is a JS special rule. NaN never equals itself. typeof null='object' is a JS bug. 0, '', '0' all coerce to falsy. '0' == false: '0' → 0, false → 0, 0===0 is true.

Q23 — Floating Point

```
console.log(0.1 + 0.2 === 0.3);
console.log(0.1 + 0.2);
```

Output: false — 0.30000000000000004

Why: Computers store decimals in binary — 0.1 and 0.2 cannot be represented exactly. Fix: use Math.abs(a - b) < Number.EPSILON

Q24 — Short Circuit Evaluation

```
console.log(false || 'hello');
console.log(true  || 'hello');
console.log(null  ?? 'default');
console.log(0     ?? 'default');
console.log(0     || 'default');
```

Output: 'hello' — true — 'default' — 0 — 'default'

Why: || returns first truthy. ?? returns first non-null/undefined (0 is NOT null/undefined so ?? returns 0). || sees 0 as falsy and returns 'default'.

A6 — Prototype and OOP Output Questions

Q25 — hasOwnProperty

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function() { return this.name; };
const a = new Animal('Dog');
console.log(a.speak());
console.log(a.hasOwnProperty('speak'));
console.log(a.hasOwnProperty('name'));
```

Output: Dog — false — true

Why: speak is on Animal.prototype, not on a directly. name is set inside the constructor on 'this', so it IS directly on a.

Q26 — instanceof

```
class A {}
class B extends A {}
const b = new B();
console.log(b instanceof B);
console.log(b instanceof A);
console.log(b instanceof Object);
```

Output: true — true — true

Why: instanceof walks the prototype chain. b → B.prototype → A.prototype → Object.prototype → null. Every class in the chain returns true.

SECTION B — Function Implementation Questions

Write the function from scratch. These test depth of understanding — not just usage but actual internal logic.

B1 — Array and String Manipulation

Q27 — Flatten Nested Array (without flat())

Input: [1, [2, [3, [4]], 5]] Output: [1, 2, 3, 4, 5]

```
function flatten(arr) {
  return arr.reduce((acc, item) =>
    acc.concat(Array.isArray(item) ? flatten(item) : item), []);
}
console.log(flatten([1,[2,[3,[4]],5]])); // [1,2,3,4,5]
```

Q28 — Group Array of Objects by Property

Input: [{name:'A',dept:'Dev'},{name:'B',dept:'Design'},{name:'C',dept:'Dev'}]

Output: {Dev:[{name:'A'...},{name:'C'...}], Design:[{name:'B'...}]}

```
function groupBy(arr, key) {
  return arr.reduce((acc, item) => {
    const group = item[key];
    if (!acc[group]) acc[group] = [];
    acc[group].push(item);
    return acc;
  }, {});
}
```

Q29 — Remove Duplicates from Array of Objects by Key

Input: [{id:1,name:'A'},{id:2,name:'B'},{id:1,name:'A'}] → Keep unique by id

```
function uniqueBy(arr, key) {
  const seen = new Set();
  return arr.filter(item => {
    if (seen.has(item[key])) return false;
    seen.add(item[key]);
    return true;
  });
```

```
}
```

Q30 — Chunk Array into Groups of Size n

Input: [1,2,3,4,5,6,7], n=3 Output: [[1,2,3],[4,5,6],[7]]

```
function chunk(arr, size) {
  const result = [];
  for (let i = 0; i < arr.length; i += size) {
    result.push(arr.slice(i, i + size));
  }
  return result;
}
```

Q31 — Find Most Frequent Element

Input: [1,2,3,2,3,3,4] Output: 3

```
function mostFrequent(arr) {
  const freq = {};
  let max = 0, result;
  for (let item of arr) {
    freq[item] = (freq[item] || 0) + 1;
    if (freq[item] > max) { max = freq[item]; result = item; }
  }
  return result;
}
```

Q32 — Anagram Check

Two strings are anagrams if they have the same characters in different order.

```
function isAnagram(a, b) {
  if (a.length !== b.length) return false;
  const sort = s => s.toLowerCase().split('').sort().join('');
  return sort(a) === sort(b);
}
console.log(isAnagram('listen', 'silent')); // true
```

Q33 — Deep Equal — Compare Two Objects

Return true if both objects have the same keys and values at ALL nesting levels.

```
function deepEqual(a, b) {
  if (a === b) return true;
  if (a === null || b === null) return false;
  if (typeof a !== 'object' || typeof b !== 'object') return false;

  const keysA = Object.keys(a);
  const keysB = Object.keys(b);
  if (keysA.length !== keysB.length) return false;

  return keysA.every(key => deepEqual(a[key], b[key]));
}
console.log(deepEqual({a:1,b:{c:2}},{a:1,b:{c:2}})); // true
console.log(deepEqual({a:1},{a:2}));                  // false
```

Q34 — String: Count Word Frequency

Input: 'hello world hello js world world' Output: {hello:2, world:3, js:1}

```
function wordFrequency(str) {
```

```
  return str.split(' ').reduce((acc, word) => {
    acc[word] = (acc[word] || 0) + 1;
    return acc;
  }, {});
}
```

Q35 — Reverse Words in a String

Input: 'Hello World JS' Output: 'JS World Hello'

```
function reverseWords(str) {
  return str.split(' ').reverse().join(' ');
}
console.log(reverseWords('Hello World JS')); // 'JS World Hello'
```

Q36 — Check Balanced Brackets

Input: '({[]})' → true Input: '({[)' → false

```
function isBalanced(str) {
  const stack = [];
  const open  = '({[';
  const close = ')}]';
  const match = { ')':'(', '}':'{', ']':'[' };

  for (let ch of str) {
    if (open.includes(ch)) {
      stack.push(ch);
    } else if (close.includes(ch)) {
      if (stack.pop() !== match[ch]) return false;
    }
  }
  return stack.length === 0;
}
console.log(isBalanced('({[]})')); // true
console.log(isBalanced('({[)'));   // false
```

Q37 — Longest Substring Without Repeating Characters

Input: 'abcabcbb' Output: 3 (abc)

```
function lengthOfLongestSubstring(s) {
  let map   = new Map(); // char → last seen index
  let left  = 0;
  let maxLen = 0;

  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    if (map.has(ch) && map.get(ch) >= left) {
      left = map.get(ch) + 1; // shrink window
    }
    map.set(ch, right);
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
console.log(lengthOfLongestSubstring('abcabcbb')); // 3
console.log(lengthOfLongestSubstring('pwwkew'));   // 3
```

B2 — Closure-Based Function Implementations

Q38 — Counter with increment, decrement, reset

```
function createCounter(start = 0) {
  let count = start;
  return {
    increment() { return ++count; },
    decrement() { return --count; },
    reset()     { count = start; return count; },
    value()     { return count; },
  };
}
const c = createCounter(10);
console.log(c.increment()); // 11
console.log(c.increment()); // 12
console.log(c.reset());     // 10
```

Q39 — Function that runs only N times

```
function limitCalls(fn, n) {
  let count = 0;
  return function(...args) {
    if (count < n) { count++; return fn(...args); }
    console.log('Limit reached!');
  };
}
const log3 = limitCalls(console.log, 3);
log3('a'); log3('b'); log3('c'); log3('d'); // Limit reached!
```

Q40 — Retry — call async fn up to N times until it succeeds

```
async function retry(fn, retries) {
  for (let i = 0; i <= retries; i++) {
    try {
      return await fn();
    } catch (err) {
      if (i === retries) throw err;
      console.log(`Attempt ${i+1} failed, retrying...`);
    }
  }
}
```

Q41 — Debounce from Scratch

Call the function only after the user has stopped calling it for `delay` ms.

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
// Usage:
const search = debounce(query => fetchResults(query), 500);
input.addEventListener('input', e => search(e.target.value));
```

Q42 — Throttle from Scratch

Call the function at most once every `limit` ms no matter how many times it is triggered.

```
function throttle(fn, limit) {
  let lastRun = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastRun >= limit) {
      lastRun = now;
      return fn.apply(this, args);
    }
  };
}
// Usage:
const onScroll = throttle(() => updatePosition(), 200);
window.addEventListener('scroll', onScroll);
```

B3 — Object Transformation

Q43 — Flatten a Nested Object

Input: {a:1, b:{c:2, d:{e:3}}} Output: {'a':1, 'b.c':2, 'b.d.e':3}

```
function flattenObj(obj, prefix = '') {
  return Object.keys(obj).reduce((acc, key) => {
    const fullKey = prefix ? `${prefix}.${key}` : key;
    if (typeof obj[key] === 'object' && obj[key] !== null && !Array.isArray(obj[key]))
{
      Object.assign(acc, flattenObj(obj[key], fullKey));
    } else {
      acc[fullKey] = obj[key];
    }
    return acc;
  }, {});
}
console.log(flattenObj({a:1,b:{c:2,d:{e:3}}}));
// {a:1, 'b.c':2, 'b.d.e':3}
```

Q44 — Unflatten Dot-Notation Object

Input: {'a':1,'b.c':2,'b.d.e':3} Output: {a:1, b:{c:2, d:{e:3}}}

```
function unflattenObj(obj) {
  const result = {};
  for (let key in obj) {
    const parts = key.split('.');
    let curr = result;
    for (let i = 0; i < parts.length - 1; i++) {
      if (!curr[parts[i]]) curr[parts[i]] = {};
      curr = curr[parts[i]];
    }
    curr[parts[parts.length - 1]] = obj[key];
  }
  return result;
}
```

Q45 — pick() — Extract Only Specified Keys

Input: {a:1,b:2,c:3,d:4}, keys=['a','c'] Output: {a:1,c:3}

```
function pick(obj, keys) {
```

```
  return keys.reduce((acc, key) => {
    if (key in obj) acc[key] = obj[key];
    return acc;
  }, {});
}
```

Q46 — omit() — Exclude Specified Keys

Input: {a:1,b:2,c:3,d:4}, keys=['b','d'] Output: {a:1,c:3}

```
function omit(obj, keys) {
  return Object.keys(obj).reduce((acc, key) => {
    if (!keys.includes(key)) acc[key] = obj[key];
    return acc;
  }, {});
}
```

Q47 — Implement _.get() — Safe Deep Property Access

_.get(obj, 'a.b.c', 'default') — returns obj.a.b.c or default if any part is undefined.

```
function get(obj, path, defaultValue = undefined) {
  const keys = path.split('.');
  let result = obj;
  for (let key of keys) {
    if (result === null || result === undefined) return defaultValue;
    result = result[key];
  }
  return result !== undefined ? result : defaultValue;
}
const obj = { user: { name: 'Shaaz', address: { city: 'Delhi' } } };
console.log(get(obj, 'user.name'));              // Shaaz
console.log(get(obj, 'user.age', 0));            // 0
console.log(get(obj, 'user.address.city'));       // Delhi
console.log(get(obj, 'user.address.pin', 'N/A')); // N/A
```

B4 — Async and Promise Implementations

Q48 — Run Promises Sequentially

Given array of async functions — run them one by one in order, not in parallel.

```
async function runSequential(asyncFns) {
  const results = [];
  for (let fn of asyncFns) {
    results.push(await fn()); // wait for each before next
  }
  return results;
}
```

Q49 — Promise Timeout Wrapper

If the promise does not resolve within N ms — reject with 'Timeout' error.

```
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Timeout!')), ms)
  );
  return Promise.race([promise, timeout]);
}
const slow = new Promise(res => setTimeout(() => res('data'), 3000));
```

```
withTimeout(slow, 1000).catch(e => console.log(e.message)); // Timeout!
```

Q50 — Promisify — Convert Callback to Promise

Node.js callbacks: fn(arg, callback) where callback = (error, result). Convert to promise.

```
function promisify(fn) {
  return function(...args) {
    return new Promise((resolve, reject) => {
      fn(...args, (err, result) => {
        if (err) reject(err);
        else resolve(result);
      });
    });
  };
}
// Usage:
const readFile = promisify(fs.readFile);
readFile('data.txt', 'utf8').then(console.log).catch(console.error);
```

Q51 — Batch API Calls with Concurrency Limit

Run n async tasks but at most `limit` at the same time. When one finishes, start the next.

```
function batchWithLimit(tasks, limit) {
  return new Promise(resolve => {
    let index = 0, running = 0, done = 0;
    const results = [];

    function runNext() {
      while (running < limit && index < tasks.length) {
        const i = index++;
        running++;
        tasks[i]().then(result => {
          results[i] = result;
          running--;
          done++;
          if (done === tasks.length) resolve(results);
          else runNext();
        });
      }
    }
    runNext();
  });
}
```

B5 — DOM Implementation Questions

Q52 — Event Delegation on Dynamic List

Add ONE listener on parent that handles ALL current AND future child items.

```
const list = document.getElementById('list');

// ONE listener — works for all current AND future li items:
list.addEventListener('click', function(e) {
  if (e.target.tagName === 'LI') {
    e.target.classList.toggle('selected');
    console.log('Clicked:', e.target.textContent);
```

```
  }
});

// Add new item dynamically — listener still works!
function addItem(text) {
  const li = document.createElement('li');
  li.textContent = text;
  list.appendChild(li);
}
```

Q53 — Custom Drag and Drop

Make an element draggable and droppable using mousedown, mousemove, mouseup.

```
function makeDraggable(el) {
  let offsetX, offsetY;

  el.addEventListener('mousedown', e => {
    offsetX = e.clientX - el.getBoundingClientRect().left;
    offsetY = e.clientY - el.getBoundingClientRect().top;
    el.style.position = 'fixed';

    function onMove(e) {
      el.style.left = (e.clientX - offsetX) + 'px';
      el.style.top  = (e.clientY - offsetY) + 'px';
    }

    function onUp() {
      document.removeEventListener('mousemove', onMove);
      document.removeEventListener('mouseup',  onUp);
    }

    document.addEventListener('mousemove', onMove);
    document.addEventListener('mouseup',  onUp);
  });
}

makeDraggable(document.getElementById('box'));
```

Q54 — Implement IntersectionObserver-based Lazy Load

Load images only when they enter the viewport — not all at once on page load.

```
function lazyLoadImages() {
  const images = document.querySelectorAll('img[data-src]');

  const observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        const img = entry.target;
        img.src = img.dataset.src;  // swap data-src → src
        img.removeAttribute('data-src');
        observer.unobserve(img);    // stop watching once loaded
      }
    });
  }, { threshold: 0.1 });
```

```
  images.forEach(img => observer.observe(img));
}
lazyLoadImages();
```

SECTION C — Full Machine Coding Challenges

These are the complete system builds asked in real interviews at product companies. Each one typically takes 30-60 minutes. Read the requirements fully, plan on paper, then code.

C1 — Build an Event Emitter (Pub/Sub System)

Companies: Flipkart, Swiggy, Razorpay, Uber

on(event,fn) — register. off(event,fn) — remove. emit(event,...args) — call all. once(event,fn) — fire once then auto-remove.

```
class EventEmitter {
  constructor() { this.events = {}; }

  on(event, fn) {
    if (!this.events[event]) this.events[event] = [];
    this.events[event].push(fn);
    return this;
  }

  off(event, fn) {
    if (!this.events[event]) return this;
    this.events[event] = this.events[event].filter(f => f !== fn);
    return this;
  }

  emit(event, ...args) {
    (this.events[event] || []).forEach(fn => fn(...args));
    return this;
  }

  once(event, fn) {
    const wrapper = (...args) => {
      fn(...args);
      this.off(event, wrapper);
    };
    return this.on(event, wrapper);
  }
}

// Test:
const em = new EventEmitter();
em.on('data', d => console.log('got:', d));
em.once('connect', () => console.log('connected!'));
em.emit('data', 'hello');   // got: hello
em.emit('connect');          // connected!
em.emit('connect');          // nothing — already removed
```

C2 — Build an LRU Cache

Companies: Amazon, Google, Microsoft, Atlassian

LRU = Least Recently Used. When cache is full, evict the item that was used least recently.

get(key) → value or -1. put(key,value) → add/update. If at capacity, remove LRU first.

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache    = new Map(); // Map preserves insertion order
  }

  get(key) {
    if (!this.cache.has(key)) return -1;
    const val = this.cache.get(key);
    this.cache.delete(key);
    this.cache.set(key, val); // move to end = most recently used
    return val;
  }

  put(key, value) {
    if (this.cache.has(key)) this.cache.delete(key);
    else if (this.cache.size >= this.capacity) {
      // First item = least recently used → delete it
      this.cache.delete(this.cache.keys().next().value);
    }
    this.cache.set(key, value);
  }
}

const lru = new LRUCache(3);
lru.put('a',1); lru.put('b',2); lru.put('c',3);
console.log(lru.get('a')); // 1 — 'a' is now MRU
lru.put('d',4);            // full — evict 'b' (LRU)
console.log(lru.get('b')); // -1 — evicted
```

C3 — Build a Debounced Search Bar with Loading State

Companies: Zomato, Nykaa, Swiggy, any product company

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const input   = document.getElementById('search');
const results = document.getElementById('results');
const loader  = document.getElementById('loader');

async function searchAPI(query) {
  if (!query.trim()) { results.innerHTML = ''; return; }
  loader.style.display = 'block';
  try {
    const res  = await fetch(`/api/search?q=${encodeURIComponent(query)}`);
```

```
    const data = await res.json();
    results.innerHTML = data.map(d => `<li>${d.name}</li>`).join('');
  } catch (e) {
    results.innerHTML = '<li>Error loading results</li>';
  } finally {
    loader.style.display = 'none';
  }
}

input.addEventListener('input', debounce(e => searchAPI(e.target.value), 500));
```

C4 — Build a Task Queue with Concurrency Limit

Companies: Uber, Ola, Razorpay, PayTM

Given N async tasks and a limit C — run at most C at a time. As each finishes, start the next.

```
function taskQueue(tasks, concurrency) {
  return new Promise(resolve => {
    let index = 0, running = 0, completed = 0;
    const results = [];

    function runNext() {
      while (running < concurrency && index < tasks.length) {
        const i = index++;
        running++;
        tasks[i]().then(result => {
          results[i] = result;
          running--;
          completed++;
          if (completed === tasks.length) resolve(results);
          else runNext();
        });
      }
    }
    runNext();
  });
}
```

C5 — Build a Redux-Style Store

Companies: Meesho, Cred, ShareChat, any React-heavy company

A store holds state, takes a reducer (pure function), and notifies subscribers on every change.

```
function createStore(reducer, initialState) {
  let state = initialState;
  let listeners = [];

  function getState() { return state; }

  function dispatch(action) {
    state = reducer(state, action); // reducer computes new state
    listeners.forEach(fn => fn(state)); // notify all subscribers
  }

  function subscribe(fn) {
```

```
    listeners.push(fn);
    return () => { listeners = listeners.filter(l => l !== fn); }; // unsubscribe
  }

  return { getState, dispatch, subscribe };
}

// Reducer — pure function:
function counterReducer(state = { count: 0 }, action) {
  switch (action.type) {
    case 'INCREMENT': return { count: state.count + 1 };
    case 'DECREMENT': return { count: state.count - 1 };
    case 'RESET':     return { count: 0 };
    default:          return state;
  }
}

const store = createStore(counterReducer);

const unsubscribe = store.subscribe(state => {
  console.log('State changed:', state);
});

store.dispatch({ type: 'INCREMENT' }); // State changed: {count:1}
store.dispatch({ type: 'INCREMENT' }); // State changed: {count:2}
store.dispatch({ type: 'DECREMENT' }); // State changed: {count:1}
unsubscribe(); // stop listening
store.dispatch({ type: 'RESET' }); // no log — unsubscribed
console.log(store.getState());      // {count:0}
```

C6 — Build a Rate Limiter

Companies: Razorpay, PhonePe, Zepto

Allow at most `limit` calls per `windowMs` milliseconds. Reject calls that exceed this.

```
function createRateLimiter(limit, windowMs) {
  const calls = []; // timestamps of recent calls

  return function isAllowed() {
    const now  = Date.now();
    const cutoff = now - windowMs;

    // Remove expired calls outside the window:
    while (calls.length && calls[0] < cutoff) calls.shift();

    if (calls.length < limit) {
      calls.push(now);
      return true;  // allowed
    }
    return false; // rate limit exceeded
  };
}

const limiter = createRateLimiter(3, 1000); // 3 per second
```

```
console.log(limiter()); // true
console.log(limiter()); // true
console.log(limiter()); // true
console.log(limiter()); // false — exceeded!
setTimeout(() => console.log(limiter()), 1100); // true — window reset
```

C7 — Build a Carousel / Image Slider

Companies: Myntra, Nykaa, Flipkart, Meesho

```
class Carousel {
  constructor(container, images) {
    this.images    = images;
    this.current   = 0;
    this.container = container;
    this.autoTimer = null;
    this.render();
  }

  render() {
    this.container.innerHTML = `
      <div class='slide'>
        <img src='${this.images[this.current]}' alt='slide ${this.current}' />
        <span>${this.current + 1} / ${this.images.length}</span>
      </div>
      <button id='prev'>&#8249;</button>
      <button id='next'>&#8250;</button>
    `;
    this.container.querySelector('#prev').onclick = () => this.prev();
    this.container.querySelector('#next').onclick = () => this.next();
  }

  next() {
    this.current = (this.current + 1) % this.images.length;
    this.render();
  }

  prev() {
    this.current = (this.current - 1 + this.images.length) % this.images.length;
    this.render();
  }

  autoPlay(interval = 3000) {
    this.autoTimer = setInterval(() => this.next(), interval);
    return this;
  }

  stop() {
    clearInterval(this.autoTimer);
    return this;
  }
}

const carousel = new Carousel(
  document.getElementById('slider'),
```

```
  ['img1.jpg', 'img2.jpg', 'img3.jpg']
);
carousel.autoPlay(2000);
```

C8 — Build a Star Rating Component

Companies: Amazon, Flipkart, MakeMyTrip, Zomato

```
class StarRating {
  constructor(container, max = 5) {
    this.container = container;
    this.max       = max;
    this.selected  = 0;
    this.hovered   = 0;
    this.render();
  }

  render() {
    this.container.innerHTML = '';
    for (let i = 1; i <= this.max; i++) {
      const star = document.createElement('span');
      star.textContent = '★';
      const filled = i <= (this.hovered || this.selected);
      star.style.color    = filled ? 'gold' : 'grey';
      star.style.fontSize = '2rem';
      star.style.cursor   = 'pointer';

      star.addEventListener('click',      () => { this.selected = i; this.render();
});
      star.addEventListener('mouseenter', () => { this.hovered  = i; this.render();
});
      star.addEventListener('mouseleave', () => { this.hovered  = 0; this.render();
});

      this.container.appendChild(star);
    }
    const label = document.createElement('p');
    label.textContent = this.selected ? `${this.selected} / ${this.max}` : 'Not
rated';
    this.container.appendChild(label);
  }
}

new StarRating(document.getElementById('rating'), 5);
```

C9 — Build an Accordion / FAQ Component

Companies: Swiggy, Zepto, Urban Company

```
class Accordion {
  constructor(container, items) {
    // items = [{question:'...', answer:'...'}, ...]
    this.container = container;
    this.items     = items;
    this.openIndex = null;
    this.render();
  }
```

```
  render() {
    this.container.innerHTML = this.items.map((item, i) => `
      <div class='accordion-item'>
        <button class='question' data-index='${i}'>
          ${item.question}
          <span>${this.openIndex === i ? '▲' : '▼'}</span>
        </button>
        <div class='answer' style='display:${this.openIndex === i ? 'block' :
'none'}'>
          ${item.answer}
        </div>
      </div>
    `).join('');

    // Event delegation — one listener for all buttons:
    this.container.addEventListener('click', e => {
      const btn = e.target.closest('.question');
      if (!btn) return;
      const index = Number(btn.dataset.index);
      this.openIndex = this.openIndex === index ? null : index;
      this.render();
    });
  }
}
```

C10 — Build a Todo App with Local Storage

Companies: Any company with frontend round — this is a very common starter build.

```
class TodoApp {
  constructor() {
    this.todos = JSON.parse(localStorage.getItem('todos') || '[]');
    this.render();
    this.bindEvents();
  }

  save() {
    localStorage.setItem('todos', JSON.stringify(this.todos));
  }

  add(text) {
    this.todos.push({ id: Date.now(), text, done: false });
    this.save();
    this.render();
  }

  toggle(id) {
    const todo = this.todos.find(t => t.id === id);
    if (todo) { todo.done = !todo.done; this.save(); this.render(); }
  }

  delete(id) {
    this.todos = this.todos.filter(t => t.id !== id);
    this.save();
```

```
    this.render();
  }

  render() {
    const list = document.getElementById('todo-list');
    list.innerHTML = this.todos.map(t => `
      <li class='${t.done ? 'done' : ''}'>
        <input type='checkbox' ${t.done ? 'checked' : ''} data-id='${t.id}' />
        <span>${t.text}</span>
        <button data-id='${t.id}' class='delete'>✕</button>
      </li>
    `).join('');
  }

  bindEvents() {
    // Add:
    document.getElementById('add-btn').addEventListener('click', () => {
      const input = document.getElementById('todo-input');
      if (input.value.trim()) {
        this.add(input.value.trim());
        input.value = '';
      }
    });

    // Toggle + Delete via delegation:
    document.getElementById('todo-list').addEventListener('click', e => {
      const id = Number(e.target.dataset.id);
      if (e.target.type === 'checkbox') this.toggle(id);
      if (e.target.classList.contains('delete')) this.delete(id);
    });
  }
}

new TodoApp();
```

SECTION D — Data Structures Implemented in JavaScript

Interviewers at product companies often ask you to implement fundamental data structures using JavaScript. Know these cold.

D1 — Stack (using Array)

```
class Stack {
  constructor() { this.items = []; }
  push(item)  { this.items.push(item); }
  pop()       { return this.items.pop(); }
  peek()      { return this.items[this.items.length - 1]; }
  isEmpty()   { return this.items.length === 0; }
  size()      { return this.items.length; }
}

const s = new Stack();
s.push(1); s.push(2); s.push(3);
```

```
console.log(s.peek()); // 3
console.log(s.pop());  // 3
console.log(s.size()); // 2
```

D2 — Queue (using Array)

```
class Queue {
  constructor() { this.items = []; }
  enqueue(item) { this.items.push(item); }
  dequeue()     { return this.items.shift(); }
  front()       { return this.items[0]; }
  isEmpty()     { return this.items.length === 0; }
  size()        { return this.items.length; }
}

const q = new Queue();
q.enqueue('a'); q.enqueue('b'); q.enqueue('c');
console.log(q.dequeue()); // 'a'
console.log(q.front());   // 'b'
```

D3 — Queue using Two Stacks

Implement a Queue using only Stack operations (push/pop). Classic interview question.

```
class QueueFromStacks {
  constructor() {
    this.inbox  = []; // for enqueue
    this.outbox = []; // for dequeue
  }

  enqueue(item) {
    this.inbox.push(item);
  }

  dequeue() {
    if (!this.outbox.length) {
      // Transfer all from inbox to outbox (reverses order):
      while (this.inbox.length) {
        this.outbox.push(this.inbox.pop());
      }
    }
    return this.outbox.pop();
  }
}

const q = new QueueFromStacks();
q.enqueue(1); q.enqueue(2); q.enqueue(3);
console.log(q.dequeue()); // 1 (FIFO — first in, first out)
console.log(q.dequeue()); // 2
```

D4 — Binary Search

Find the index of a target in a SORTED array. O(log n) — much faster than linear search.

```
function binarySearch(arr, target) {
  let left  = 0;
```

```
  let right = arr.length - 1;

  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if      (arr[mid] === target) return mid;  // found
    else if (arr[mid] < target)  left  = mid + 1; // go right
    else                         right = mid - 1; // go left
  }
  return -1; // not found
}

const sorted = [1, 3, 5, 7, 9, 11, 13, 15];
console.log(binarySearch(sorted, 7));  // 3
console.log(binarySearch(sorted, 6));  // -1
```

D5 — Linked List (Singly)

```
class Node {
  constructor(val) { this.val = val; this.next = null; }
}

class LinkedList {
  constructor() { this.head = null; }

  append(val) {
    const node = new Node(val);
    if (!this.head) { this.head = node; return; }
    let curr = this.head;
    while (curr.next) curr = curr.next;
    curr.next = node;
  }

  prepend(val) {
    const node = new Node(val);
    node.next  = this.head;
    this.head  = node;
  }

  delete(val) {
    if (!this.head) return;
    if (this.head.val === val) { this.head = this.head.next; return; }
    let curr = this.head;
    while (curr.next && curr.next.val !== val) curr = curr.next;
    if (curr.next) curr.next = curr.next.next;
  }

  print() {
    const vals = [];
    let curr = this.head;
    while (curr) { vals.push(curr.val); curr = curr.next; }
    console.log(vals.join(' → '));
  }
}
```

```
const ll = new LinkedList();
ll.append(1); ll.append(2); ll.append(3);
ll.prepend(0);
ll.print();   // 0 → 1 → 2 → 3
ll.delete(2);
ll.print();   // 0 → 1 → 3
```

D6 — Reverse a Linked List

Reverse the linked list in place — O(n) time, O(1) space.

```
function reverseList(head) {
  let prev = null;
  let curr = head;

  while (curr) {
    const next = curr.next; // save next
    curr.next  = prev;       // reverse link
    prev       = curr;       // move prev
    curr       = next;       // move curr
  }
  return prev; // new head
}
```

SECTION E — Algorithm Questions in JavaScript

E1 — Two Sum

Return indices of the two numbers that add up to target. O(n) using Map.

```
function twoSum(nums, target) {
  const map = new Map(); // value → index
  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (map.has(complement)) return [map.get(complement), i];
    map.set(nums[i], i);
  }
  return [];
}
console.log(twoSum([2,7,11,15], 9)); // [0,1]
console.log(twoSum([3,2,4], 6));      // [1,2]
```

E2 — Valid Palindrome

Return true if the string reads the same forward and backward (ignore non-alphanumeric).

```
function isPalindrome(s) {
  const clean = s.toLowerCase().replace(/[^a-z0-9]/g, '');
  return clean === clean.split('').reverse().join('');
}
console.log(isPalindrome('A man, a plan, a canal: Panama')); // true
console.log(isPalindrome('race a car')); // false
```

E3 — Maximum Subarray — Kadane's Algorithm

Find the contiguous subarray with the largest sum. O(n) using Kadane's algorithm.

```
function maxSubarray(nums) {
  let maxSum  = nums[0];
  let currSum = nums[0];
  for (let i = 1; i < nums.length; i++) {
    currSum = Math.max(nums[i], currSum + nums[i]);
    maxSum  = Math.max(maxSum, currSum);
  }
  return maxSum;
}
console.log(maxSubarray([-2,1,-3,4,-1,2,1,-5,4])); // 6  ([4,-1,2,1])
```

E4 — Fibonacci with Memoization

```
function fib(n, memo = {}) {
  if (n in memo) return memo[n];
  if (n <= 1)    return n;
  memo[n] = fib(n-1, memo) + fib(n-2, memo);
  return memo[n];
}
console.log(fib(10)); // 55
console.log(fib(50)); // 12586269025
```

E5 — Find All Permutations of a String

```
function permutations(str) {
  if (str.length <= 1) return [str];
  const result = [];
  for (let i = 0; i < str.length; i++) {
    const char = str[i];
    const rest = str.slice(0,i) + str.slice(i+1);
    permutations(rest).forEach(p => result.push(char + p));
  }
  return result;
}
console.log(permutations('abc')); // ['abc','acb','bac','bca','cab','cba']
```

E6 — Valid Parentheses / Brackets

Already covered in B1 Q36. Practice it again — it appears in almost every round.

```
function isBalanced(s) {
  const stack = [];
  const match = { ')':'(', '}':'{', ']':'[' };
  for (let ch of s) {
    if ('({['.includes(ch)) stack.push(ch);
    else if (stack.pop() !== match[ch]) return false;
  }
  return stack.length === 0;
}
```

E7 — Find Duplicate in Array (O(n), no extra space)

Array has n+1 integers in range [1,n]. Find the duplicate.

```
// XOR approach:
function findDuplicate(nums) {
  const freq = {};
```

```
  for (let n of nums) {
    if (freq[n]) return n;
    freq[n] = true;
  }
}
console.log(findDuplicate([1,3,4,2,2])); // 2
```

E8 — Sort Array of 0s, 1s, and 2s (Dutch National Flag)

Sort without using sort() — O(n) one pass. Classic interview question.

```
function sortColors(nums) {
  let low = 0, mid = 0, high = nums.length - 1;
  while (mid <= high) {
    if (nums[mid] === 0) {
      [nums[low], nums[mid]] = [nums[mid], nums[low]];
      low++; mid++;
    } else if (nums[mid] === 1) {
      mid++;
    } else {
      [nums[mid], nums[high]] = [nums[high], nums[mid]];
      high--;
    }
  }
  return nums;
}
console.log(sortColors([2,0,2,1,1,0])); // [0,0,1,1,2,2]
```

SECTION F — Tricky JS-Specific Builds

F1 — Implement map() using reduce()

```
Array.prototype.mapWithReduce = function(cb) {
  return this.reduce((acc, item, i, arr) => {
    acc.push(cb(item, i, arr));
    return acc;
  }, []);
};
console.log([1,2,3].mapWithReduce(x => x*2)); // [2,4,6]
```

F2 — Implement Object.create() from scratch

```
function myObjectCreate(proto) {
  function F() {}
  F.prototype = proto;
  return new F();
}
const animal = { breathe() { console.log('breathing'); } };
const dog = myObjectCreate(animal);
dog.breathe(); // breathing — inherited via prototype
```

F3 — Implement an Observable / Reactive Variable

When a variable changes, automatically notify all watchers — like Vue's reactivity.

```
function observable(value) {
  let watchers = [];
  return {
    get()      { return value; },
    set(val)   {
      const old = value;
      value = val;
      watchers.forEach(fn => fn(val, old));
    },
    watch(fn)  { watchers.push(fn); }
  };
}

const name = observable('Shaaz');
name.watch((next, prev) => console.log(`${prev} → ${next}`));
name.set('Aman'); // Shaaz → Aman
name.set('Riya'); // Aman  → Riya
```

F4 — Implement Promisify

```
function promisify(fn) {
  return (...args) => new Promise((resolve, reject) => {
    fn(...args, (err, result) => err ? reject(err) : resolve(result));
  });
}
```

F5 — Implement pipe() with Error Handling

```
function safePipe(...fns) {
  return value => fns.reduce((acc, fn) => {
    if (acc instanceof Error) return acc;
    try   { return fn(acc); }
    catch (e) { return e; }
  }, value);
}

const transform = safePipe(
  x => x * 2,
  x => { if (x > 10) throw new Error('too big'); return x; },
  x => x + 1
);
console.log(transform(3));  // 7  (3*2=6, 6+1=7)
console.log(transform(6));  // Error: too big
```

F6 — Implement a Simple Template Engine

Replace {{key}} placeholders in a template string with values from a data object.

```
function template(str, data) {
  return str.replace(/\{\{(\w+)\}\}/g, (match, key) => {
    return data[key] !== undefined ? data[key] : match;
  });
}

const tmpl = 'Hello {{name}}, you are {{age}} years old!';
console.log(template(tmpl, { name: 'Shaaz', age: 23 }));
```

```
// Hello Shaaz, you are 23 years old!

console.log(template('Dear {{title}} {{name}}', { title: 'Mr', name: 'Aman' }));
// Dear Mr Aman
```

F7 — Infinite Scroll Implementation

Load more data when user scrolls to 80% of the page height.

```
let page    = 1;
let loading = false;

async function loadMore() {
  if (loading) return;
  loading = true;
  const data = await fetchPage(page++);
  appendItems(data);
  loading = false;
}

function throttle(fn, ms) {
  let last = 0;
  return () => { if (Date.now()-last >= ms) { last=Date.now(); fn(); } };
}

window.addEventListener('scroll', throttle(() => {
  const { scrollTop, scrollHeight, clientHeight } = document.documentElement;
  if ((scrollTop + clientHeight) / scrollHeight > 0.8) loadMore();
}, 300));
```

Machine Coding Round — Master Checklist

Before your machine coding round — verify you can write ALL of these from memory without help:

OUTPUT QUESTIONS — Know cold:

› var hoisting → undefined. let TDZ → ReferenceError
› var in loop + setTimeout → all print last value (3,3,3). let → 0,1,2
› Closure captures REFERENCE not value — outer()() prints updated x
› Arrow function this cannot be changed by call/apply/bind
› Promise.then runs BEFORE setTimeout even with 0ms delay
› async/await: code after await runs after current synchronous code finishes
› typeof null → 'object'. typeof NaN → 'number'. Both are JS bugs
› 0.1 + 0.2 !== 0.3 — floating point precision
› null == undefined is true. null === undefined is false
› NaN === NaN is false. Use Number.isNaN() to check for NaN
› [] + [] = ''. [] + {} = '[object Object]'
› 0, '', null, undefined, NaN, false — the 6 falsy values. '0', [], {} are truthy

FUNCTION IMPLEMENTATIONS — Write from memory:

› flatten nested array → recursion + reduce
› groupBy array of objects → reduce into {}
› chunk array → slice in loop
› isBalanced brackets → stack + match object
› debounce → clearTimeout + setTimeout in closure
› throttle → Date.now() + lastRun in closure
› deep clone → recursive type check (null, Array, Date, Object)
› deep equal → recursive keys comparison
› deepGet (_.get) → split path, traverse with reduce
› flatten object → recursive with prefix

SYSTEM BUILDS — Practice these fully:

› EventEmitter → on/off/emit/once with events:{} object
› LRU Cache → Map (preserves order) + delete + re-set
› Redux Store → reducer + listeners[] + dispatch + subscribe
› Rate Limiter → timestamps array + window filter
› Todo App → render, add, toggle, delete + localStorage
› Search bar → debounce + fetch + loading state
› Task Queue → concurrency counter + runNext pattern
› Carousel → current index + next/prev + autoPlay with setInterval

DATA STRUCTURES — Know all of these:

› Stack → push, pop, peek, isEmpty
› Queue → enqueue, dequeue, front
› Queue from 2 Stacks → inbox/outbox pattern
› Linked List → append, prepend, delete, print
› Reverse Linked List → prev/curr/next pointer pattern
› Binary Search → left/right/mid while loop

ALGORITHMS — Practice these until instant:

› Two Sum → Map of complement → index
› Valid Palindrome → clean + reverse compare
› Max Subarray → Kadane's algorithm
› Fibonacci → memoization with {} cache
› Permutations → recursive character extraction
› Sort 0s 1s 2s → Dutch National Flag — three pointers
› Anagram → sort both strings, compare
› Most Frequent Element → frequency map + track max

✅Remember: The #1 tip for machine coding rounds: Plan before you type. Spend 3-5 minutes writing the approach on paper or commenting it in code. Interviewers value structured thinking over speed. Talk out loud about what you are doing — they want to see your mind, not just your fingers.

You Made It!

Good Luck with Your Interview

Remember:

Code every day. Explain out loud. Teach others what you learn.

— Syed Shaaz Akhtar